



KUKA System Technology
KUKA.RobotSensorInterface 5.0
Für KUKA System Software 8.7



Stand: 11.11.2024
KST RSI 5.0 V3
KUKA Deutschland GmbH

© Copyright 2024

KUKA Deutschland GmbH
Zugspitzstraße 140
D-86165 Augsburg
Deutschland

Diese Dokumentation darf – auch auszugsweise – nur mit ausdrücklicher Genehmigung der KUKA Deutschland GmbH vervielfältigt oder Dritten zugänglich gemacht werden.

Es können weitere, in dieser Dokumentation nicht beschriebene Funktionen in der Steuerung lauffähig sein. Es besteht jedoch kein Anspruch auf diese Funktionen bei Neulieferung oder im Servicefall.

Wir haben den Inhalt der Druckschrift auf Übereinstimmung mit der beschriebenen Hard- und Software geprüft. Dennoch können Abweichungen nicht ausgeschlossen werden, so dass wir für die vollständige Übereinstimmung keine Gewähr übernehmen. Die Angaben in dieser Druckschrift werden jedoch regelmäßig überprüft und notwendige Korrekturen sind in der nachfolgenden Auflage enthalten.

Technische Änderungen ohne Beeinflussung der Funktion vorbehalten.

KIM-PS5-DOC

Original-Dokumentation

Publikation: Pub KST RSI 5.0 (PDF) de
PB13326

Buchstruktur: KST RSI 5.0 V3.1
BS12316

Version: KST RSI 5.0 V3

Inhaltsverzeichnis

1	Einleitung.....	7
1.1	Zielgruppe.....	7
1.2	Dokumentation des Industrieroboters.....	7
1.3	Darstellung von Hinweisen.....	7
1.4	Verwendete Begriffe.....	8
1.5	Marken.....	10
1.6	Lizenzbedingungen.....	10
2	Produktbeschreibung.....	13
2.1	Produktbeschreibung.....	13
2.2	Übersicht Datenaustausch.....	13
2.2.1	Datenaustausch über E/A-System von RSI.....	14
2.2.2	Datenaustausch über E/A-System der Robotersteuerung.....	15
2.2.3	Datenaustausch über Ethernet.....	15
2.3	Signalverarbeitung.....	17
2.4	Übersicht Sensorkorrektur.....	18
2.4.1	Korrekturtyp.....	19
2.4.1.1	Achsweise Korrektur.....	20
2.4.1.2	Kartesische Korrektur.....	20
2.4.2	Korrekturmodus.....	22
2.4.2.1	Absolutkorrektur.....	22
2.4.2.2	Relativkorrektur.....	22
2.4.3	Korrekturmethode.....	24
2.4.4	Sensorkorrektur im RoboTeam-Betrieb.....	26
2.4.5	Sensorkorrektur im ConveyorTech-Betrieb.....	27
2.5	Bestimmungsgemäße Verwendung und Fehlanwendung.....	29
3	Sicherheit.....	31
3.1	Sicherheitshinweise.....	31
4	Installation.....	33
4.1	Systemvoraussetzungen.....	33
4.2	Installation über WorkVisual.....	34
4.2.1	KUKA.RobotSensorInterface 5.0 installieren oder updaten.....	34
4.2.2	Optionspaket deinstallieren.....	35
4.3	Installation über smartHMI.....	35
4.3.1	KUKA.RobotSensorInterface 5.0 installieren oder updaten.....	35
4.3.2	KUKA.RobotSensorInterface 5.0 deinstallieren.....	37
5	Bedienung.....	39
5.1	Benutzerrechte anzeigen.....	39
6	Konfiguration.....	41
6.1	Ein-/Ausgänge von RSI in WorkVisual verschalten.....	41
6.2	Ethernet über KLI-Netzwerkconfiguration konfigurieren.....	43
6.3	Ethernet über KONI-Netzwerkconfiguration konfigurieren.....	44
6.4	Globale Variablen ändern.....	45
6.5	RSI-Kontexte über Kontextverwaltung parametrieren.....	45

7	Programmierung.....	49
7.1	Übersicht RSI-Befehle.....	49
7.1.1	Zeichen und Schriftarten.....	49
7.1.2	RSI_CREATE: RSI-Container erstellen.....	50
7.1.3	RSI_DELETE: RSI-Container löschen.....	51
7.1.4	RSI_ON: Signalverarbeitung einschalten.....	51
7.1.5	RSI_OFF: Signalverarbeitung ausschalten.....	53
7.1.6	RSI_MOVECORR: Sensorgeführte Bewegung einschalten.....	53
7.1.7	RSI_GETPUBLICPAR: RSI-Objektparameterwert lesen.....	54
7.1.8	RSI_SETPUBLICPAR: RSI-Objektparameter neuen Wert zuweisen.....	55
7.1.9	RSI_RESET: Signalverarbeitung und alle RSI-Objekte löschen	56
7.1.10	RSI_CHECKID: RSI-Container-ID prüfen.....	56
7.1.11	RSI_ENABLE: RSI-Container aktivieren.....	57
7.1.12	RSI_DISABLE: RSI-Container deaktivieren.....	57
7.2	Verhalten der Befehle RSI_DISABLE und RSI_ENABLE.....	58
7.3	Verhalten der Befehle RSI_ON und RSI_OFF.....	59
7.4	Signalverarbeitung programmieren.....	59
7.4.1	RSI-Kontext in KRL-Programm einbinden.....	60
7.4.2	Zugriff auf RSI-Objektparameter über KRL.....	60
7.5	XML-Datei für Ethernet-Verbindung konfigurieren.....	61
7.5.1	XML-Struktur für Verbindungseigenschaften.....	62
7.5.2	XML-Struktur für den Datenversand.....	62
7.5.3	XML-Struktur für den Datenempfang.....	64
7.5.4	Konfiguration nach XML-Schema.....	66
7.5.5	Schlüsselwörter zum Daten lesen.....	67
7.5.6	Schlüsselwörter zum Daten schreiben.....	68
7.5.7	Ethernet-Verbindung in WorkVisual konfigurieren.....	69
8	Expertenprogrammierung mit RSIVisual.....	71
8.1	Übersicht RSIVisual.....	71
8.2	RSI-Kontext anlegen.....	71
8.3	RSI-Kontextdatei importieren und konvertieren.....	72
8.4	RSI-Kontext im Kontext-Editor öffnen.....	73
8.4.1	RSI-Element in Editor einfügen.....	74
8.4.2	RSI-Objektparameter bearbeiten.....	75
8.4.2.1	RSI-Objektparameter freischalten.....	75
8.4.3	Ein-/Ausgänge der RSI-Elemente verbinden.....	76
8.4.4	Informationen zu RSI-Elementen anzeigen.....	77
8.4.5	RSI-Elemente automatisch anordnen.....	77
8.4.6	RSI-Elemente gruppieren.....	77
8.4.7	Weitere Bearbeitungsfunktionen.....	79
8.5	Fenster Werkzeugkasten einblenden.....	79
8.5.1	Benutzerdefinierten Werkzeugkasten erstellen.....	81
8.5.2	Kategorie für benutzerdefinierten Werkzeugkasten erstellen.....	82
8.5.3	Gruppe in benutzerdefiniertem Werkzeugkasten hinzufügen.....	82
8.5.4	Funktionsblock in benutzerdefiniertem Werkzeugkasten erstellen.....	82
8.5.5	Funktionsblock in RSI-Kontext aktualisieren.....	83
8.5.6	Elemente im benutzerdefinierten Werkzeugkasten kopieren, ausschneiden und einfügen.....	83

8.5.7	Benutzerdefinierten Werkzeugkasten exportieren.....	83
8.5.8	Benutzerdefinierten Werkzeugkasten importieren.....	84
8.5.9	Benutzerdefinierten Werkzeugkasten im Optionspaketeditor verwenden.....	84
8.6	Funktionsblockprogrammierung mit RSIVisual.....	85
8.6.1	Funktionsblock im Funktionsblock-Editor öffnen.....	85
8.6.2	Funktionsblock programmieren.....	87
8.6.3	Ein- und Ausgänge von Funktionsblöcken definieren.....	88
8.6.4	Funktionsblockeigenschaften definieren.....	89
8.7	Zugriff auf Funktionsblockparameter über KRL.....	91
8.8	Hilfdatei.....	92
9	Beispielkonfigurationen und Beispielprogramme.....	93
9.1	Server-Programm und Beispieldateien einbinden.....	93
9.1.1	Dateien über Projekt in WorkVisual auf Steuerung übertragen.....	94
9.1.2	Dateien über KRC-Explorer von WorkVisual auf Steuerung übertragen.....	94
9.2	Bedienoberfläche Server-Programm.....	95
9.2.1	Kommunikationsparameter im Server-Programm einstellen.....	96
9.3	Beispiel RSI_Ethernet: kartesische Korrektur über Ethernet.....	97
9.4	Beispiel RSI_CircleCorr: sensorgeführte Kreisbewegung.....	99
9.5	Beispiel RSI_DistanceCtrl: Bahnkorrektur zur Abstandssteuerung.....	102
9.6	Beispiel RSI_SigTransformation: Transformation in neues Koordinatensystem..	105
10	Diagnose.....	111
10.1	RSI-Monitor anzeigen.....	111
10.1.1	Signaleigenschaften definieren.....	112
10.1.2	Signalverlauf anzeigen und aufzeichnen.....	113
10.1.3	Signalaufzeichnung speichern.....	114
10.1.4	Signalaufzeichnung in den RSI-Monitor laden.....	114
10.2	Diagnosedaten anzeigen.....	114
10.3	Fehlerprotokoll (LOG-Datei).....	115
10.3.1	LOG-Level konfigurieren.....	115
11	Meldungen.....	117
11.1	Informationen zu den Meldungen.....	117
11.2	Systemmeldungen aus Modul: CrossMeld (KSS).....	117
11.2.1	KSS29000.....	117
11.2.2	KSS29001.....	119
11.2.3	KSS29002.....	119
11.2.4	KSS29004.....	120
11.2.5	KSS29006.....	121
11.2.6	KSS29007.....	123
11.2.7	KSS29008.....	125
11.2.8	KSS29009.....	126
11.2.9	KSS29010.....	127
11.2.10	KSS29011.....	128
11.2.11	KSS29012.....	129
11.2.12	KSS29013.....	130
12	Anhang.....	131
12.1	Speicher erhöhen.....	131

12.2	RSI-Objektbibliothek.....	131
12.2.1	RSI-Objekte zur Kommunikation.....	131
12.2.2	RSI-Objekte für logische Verknüpfungen.....	132
12.2.3	RSI-Objekte für mathematische Operationen.....	132
12.2.4	RSI-Objekte für mathematische Vergleiche.....	133
12.2.5	RSI-Objekte zur Koordinatentransformation.....	134
12.2.6	RSI-Objekte zur Bewegungskorrektur.....	134
12.2.7	RSI-Objekte zur Korrekturüberwachung.....	134
12.2.8	RSI-Objekte zum Lesen von Roboterdaten.....	134
12.2.9	RSI-Objekte zur Signalverarbeitung.....	135
13	KUKA Service.....	137
13.1	Support-Anfrage.....	137
13.2	KUKA Customer Support.....	137
	Index	139

1 Einleitung

1.1 Zielgruppe

Diese Dokumentation richtet sich an Benutzer mit folgenden Kenntnissen:

- Fortgeschrittene KRL-Programmierkenntnisse
- Fortgeschrittene Systemkenntnisse der Robotersteuerung
- Fortgeschrittene Kenntnisse über Bussysteme
- Grundkenntnisse in XML
- Grundkenntnisse in der Digitaltechnik



Für den optimalen Einsatz unserer Produkte empfehlen wir unseren Kunden eine Schulung im KUKA College. Informationen zum Schulungsprogramm sind unter www.kuka.com oder direkt bei den Niederlassungen zu finden.

1.2 Dokumentation des Industrieroboters

Die Dokumentation zum Industrieroboter besteht aus folgenden Teilen:

- Dokumentation für die Robotermechanik
- Dokumentation für die Robotersteuerung
- Dokumentation für das smartPAD-2 oder smartPAD pro (wenn verwendet)
- Dokumentation für die Systemsoftware
- Anleitungen zu Optionen und Zubehör
- Anhaltewege und Anhaltezeiten in der Dokumentation für die Robotermechanik oder in KUKA Xpert
- Ersatzteilübersicht in KUKA Xpert

Jede Anleitung ist ein eigenes Dokument.

1.3 Darstellung von Hinweisen

Sicherheit

Diese Hinweise dienen der Sicherheit und **müssen** beachtet werden.



GEFAHR

Diese Hinweise bedeuten, dass Tod oder schwere Verletzungen sicher oder sehr wahrscheinlich eintreten **werden**, wenn keine Vorsichtsmaßnahmen getroffen werden.



WARNUNG

Diese Hinweise bedeuten, dass Tod oder schwere Verletzungen eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.



VORSICHT

Diese Hinweise bedeuten, dass leichte Verletzungen eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.

HINWEIS

Diese Hinweise bedeuten, dass Sachschäden eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.



Diese Hinweise enthalten Verweise auf sicherheitsrelevante Informationen oder allgemeine Sicherheitsmaßnahmen. Diese Hinweise beziehen sich nicht auf einzelne Gefahren oder einzelne Vorsichtsmaßnahmen.

Dieser Hinweis macht auf Vorgehensweisen aufmerksam, die der Vorbeugung oder Behebung von Not- oder Störfällen dienen:

SICHERHEITSANWEISUNG

Die folgende Vorgehensweise genau einhalten!

Mit diesem Hinweis gekennzeichnete Vorgehensweisen **müssen** genau eingehalten werden.

Hinweise

Diese Hinweise dienen der Arbeitserleichterung oder enthalten Verweise auf weiterführende Informationen.



Hinweis zur Arbeitserleichterung oder Verweis auf weiterführende Informationen

1.4 Verwendete Begriffe**Allgemeine Begriffe**

Begriff	Beschreibung
CCS	Correction Coordinate System Das Korrekturkoordinatensystem ist für die kartesische Sensorkorrektur. Der Ursprung des Korrekturkoordinatensystems ist der aktuelle TCP, die Orientierung wird von einem Referenzkoordinatensystem übernommen.
Ethernet	Datennetztechnologie für lokale Datennetze Ethernet ermöglicht den Datenaustausch in Form von Datenrahmen zwischen den verbundenen Teilnehmern.
IP	Internet Protocol Das Internet-Protokoll hat die Aufgabe, Subnetze über physikalische MAC-Adressen zu definieren.
KLI	KUKA Line Interface Ethernet-Schnittstelle der Robotersteuerung für die externe Kommunikation (nicht echtzeitfähig)
KONI	KUKA.OptionalNetworkInterface Optionale Ethernet-Schnittstelle der Robotersteuerung für die externe Kommunikation
KR C	KUKA Robot Control Robotersteuerung

Sensormodus	Modus, in dem Sensorsignale verarbeitet werden • Sensormodus #IPO: Signalverarbeitung im Sensortakt von 12 ms • Sensormodus #IPO_FAST: Signalverarbeitung im Sensortakt von 4 ms
smartHMI	smart Human-Machine Interface Bedienoberfläche der Systemsoftware auf dem Handbediengerät
smartPAD	Programmierhandgerät für die Robotersteuerung Das smartPAD hat alle Bedien- und Anzeigemöglichkeiten, die für die Bedienung und Programmierung benötigt werden. Es existieren folgende Modelle: • KUKA smartPAD • KUKA smartPAD-2 • KUKA smartPAD pro Für Robotersteuerungen der Serie KR C5 mit KUKA System Software oder VW System Software kommt nur das Modell KUKA smartPAD-2 zum Einsatz Für Robotersteuerungen der Serie KR C5 mit KUKA iiQKA.OS kommt nur das Modell KUKA smartPAD pro zum Einsatz. Für andere Robotersteuerungen bezieht sich die Bezeichnung "KUKA smartPAD" oder "smartPAD" immer auf alle für diese Steuerung möglichen Modelle, sofern diese nicht explizit unterschieden werden.
TCP	Tool Center Point Arbeitspunkt eines Werkzeugs und Ursprung des TOOL-Koordinatensystems
TTS	Toolbased Technological System Koordinatensystem, das auf der Bahn mitfährt Koordinatensystem, das auf der Bahn mitfährt, wird bei jeder LIN-, CIRC-, SLIN und SCIRC Bewegung berechnet.
UDP	User Datagram Protocol Verbindungsloses Protokoll für den Datenaustausch zwischen den Geräten eines Netzwerks. UDP stellt keine Verbindung zum Netzwerk her.
XML	Extensible Markup Language Standard zur Erstellung maschinen- und menschenlesbarer Dokumente in Form einer vorgegebenen Baumstruktur

RSI-Begriffe

Begriff	Beschreibung
---------	--------------

RSI	RobotSensorInterface Schnittstelle zur Kommunikation zwischen Industrieroboter und Sensorsystem Das RSI ist die Basistechnologie zur Realisierung sensibler Applikationen, die eine zyklische Signalverarbeitung und Bewegungsbeeinflussung des Roboters in Echtzeit erfordern.
RSI-Container	Container im KRL-Programm, in den ein mit RSIVisual erstellter RSI-Kontext geladen wird
RSI-Container-ID	Kennung, die beim Anlegen des RSI-Containers im KRL-Programm automatisch vergeben wird
RSI-Kontext	Mit RSIVisual programmierte Signalverarbeitung Ein RSI-Kontext besteht aus RSI-Objekten und Verknüpfungen zwischen den RSI-Objekten.
RSI-Monitor	Monitor zur Online-Visualisierung von RSI-Signalen
RSI-Objekt	Funktionsbaustein zur Programmierung eines RSI-Kontexts RSI-Objekte verfügen über objektspezifische Signaleingänge/-ausgänge, durch die sie miteinander verknüpft werden können.
RSI-Objektbibliothek	Bibliothek der RSI-Objekte, die zur Programmierung eines RSI-Kontexts zur Verfügung stehen
RSI-Objektparameter	Parameter des RSI-Objekts Die Parameter bestimmen die Funktionalität des RSI-Objekts. Die Anzahl der Parameter ist spezifisch für jedes RSI-Objekt.
RSIVisual	Grafischer Editor zur Programmierung der Signalverarbeitung (RSI-Kontext)

1.5 Marken



EtherCAT® ist eine eingetragene Marke und patentierte Technologie, lizenziert durch die Beckhoff Automation GmbH, Deutschland.

.NET Framework ist eine Marke der Microsoft Corporation.

Windows ist eine Marke der Microsoft Corporation.

1.6 Lizenzbedingungen

Die KUKA Lizenzbedingungen und die Lizenzbedingungen verwendeter Open-Source-Software sind in folgenden Ordnern zu finden:

- Auf dem Datenträger mit den Installationsdateien der KUKA Software unter `.LICENSE`
- Auf der KUKA smartHMI im Hauptmenü unter **Hilfe > Info**, Registerkarte **Lizenzen**
- Auf der Robotersteuerung unter `D:\KUKA_OPT\Optionspaketname\LICENSE`
- In WorkVisual im Katalog **Optionen** im Lizenzenordner unter dem Namen des Optionspakets



Weitere Informationen zu Open-Source-Lizenzen können unter folgender Adresse angefordert werden: opensource@kuka.com

2 Produktbeschreibung

2.1 Produktbeschreibung

Beschreibung

KUKA.RobotSensorInterface 5.0 ist ein Optionspaket, mit dem Sensorapplikationen realisiert werden können, die eine Echtzeitverarbeitung von Sensordaten erfordern.

Die Programmierung der Signalverarbeitung erfolgt in WorkVisual mithilfe des graphischen Editors RSIVisual. In RSIVisual können hierfür sog. RSI-Kontexte erstellt werden, in denen vordefinierte Funktionsbausteine, sog. RSI-Objekte, verknüpft werden.

Die programmierte Signalverarbeitung kann die ursprünglich definierte Roboterbahn korrigieren.

Funktionen

- Datenaustausch zwischen Robotersteuerung und Sensorsystem:
 - Über das EA-System von RSI, das EA-System der Robotersteuerung oder über Ethernet
- Zyklische Signalverarbeitung und -auswertung im Sensortakt
- Roboterbahn durch die Verarbeitung von Sensorsignalen korrigieren
- Online-Visualisierung der RSI-Signale mit RSI-Monitor
- RSI-Kontexte in RSIVisual erstellen:
 - Werkzeugkasten mit über 75 RSI-Objekten
 - Benutzerdefinierte Werkzeugkästen erstellen, exportieren und importieren
 - Funktionsblöcke für benutzerdefinierte Werkzeugkästen erstellen
 - Funktionsblöcke in eigenem Funktionsblockeditor programmieren
 - Funktionsblöcke in RSI-Kontexten verwenden

2.2 Übersicht Datenaustausch

Robotersteuerung und Sensorsystem können auf verschiedene Arten miteinander kommunizieren und Daten austauschen:

- Über das E/A-System von RSI

Das E/A-System von RSI ist unabhängig vom E/A-System der Robotersteuerung und kann sowohl mit Feldbusteilnehmern als auch mit anderen E/A-Systemen kommunizieren.

(>>> [2.2.1 "Datenaustausch über E/A-System von RSI" Seite 14](#))
- Über das E/A-System der Robotersteuerung

(>>> [2.2.2 "Datenaustausch über E/A-System der Robotersteuerung" Seite 15](#))
- Über Ethernet

Die Robotersteuerung kommuniziert mit dem Sensorsystem über eine echtzeitfähige Netzwerkverbindung. Die Daten werden über das Ethernet UDP/IP-Protokoll übertragen.

Bei Verwendung des Ethernet UDP/IP-Protokolls sind folgende Punkte zu beachten:

 - UDP ist ein verbindungsloses Netzwerkprotokoll für den Austausch von Datenpaketen. Der Datenaustausch über UDP ist unzuverlässig

sig und ungesichert. Es kann z. B. nicht garantiert werden, dass gesendete Pakete zuverlässig ankommen oder in der Reihenfolge ankommen, in der sie gesendet wurden.

- Der Datenaustausch über Ethernet eignet sich für Anwendungen, die unempfindlich auf verlorengegangene oder in falscher Reihenfolge empfangene Pakete reagieren. Toleriert eine Anwendung dies nicht, muss der Programmierer geeignete Maßnahmen ergreifen. Z. B. im Programm überprüfen, ob alle Pakete angekommen sind und bei Bedarf die Pakete neu anfordern.

(>>> 2.2.3 "Datenaustausch über Ethernet" Seite 15)

2.2.1 Datenaustausch über E/A-System von RSI

Beschreibung

- Die Daten und Signale des Sensorsystems werden über das E/A-System von RSI gelesen und geschrieben.
- RSI besitzt 8192 Bits für Ein-/Ausgänge. Diese Ein-/Ausgänge können in WorkVisual mit den Signalen eines Feldbusteilnehmers oder eines anderen E/A-Systems verschaltet werden.

Der Zugriff auf die verschalteten Signale findet im RSI-Kontext statt. Folgende RSI-Objekte werden dafür verwendet:

- INPUT greift lesend auf die digitalen RSI-Eingänge zu. Auf RSI-Eingänge können die Ausgangssignale eines Feldbusteilnehmers oder eines anderen E/A-Systems verschaltet werden.
- OUTPUT schreibt auf die digitalen RSI-Ausgänge. Die RSI-Ausgänge können auf die Eingänge eines Feldbusteilnehmers oder eines anderen E/A-Systems verschaltet werden.



Die Verschaltung von RSI-Ausgängen auf RSI-Eingängen ist unzulässig und wird daher unterbunden.

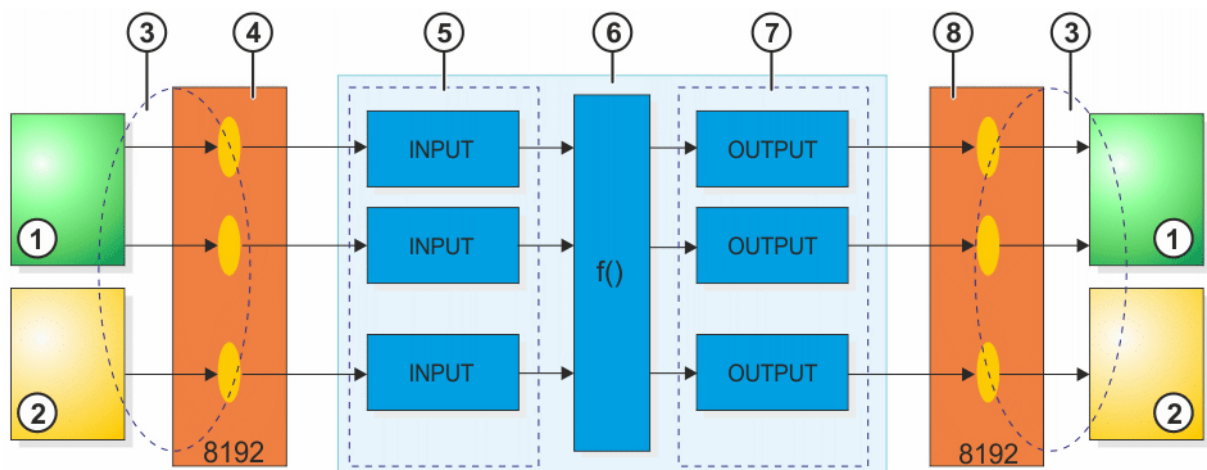


Abb. 2-1: Datenaustausch über E/A-System von RSI

- 1 Feldbusteilnehmer mit seinen E/As (z. B. Sensor am PROFINET)
- 2 Feldbusteilnehmer mit seinen E/As (z. B. Sensor am EtherCAT)
- 3 E/A-Verschaltung über WorkVisual
- 4 Eingänge des E/A-Systems von RSI
- 5 INPUT-Objekte für Zugriff auf RSI-Eingänge
- 6 RSI-Kontext zur Signalverarbeitung

- 7 OUTPUT-Objekte für Zugriff auf RSI-Ausgänge
- 8 Ausgänge des E/A-Systems von RSI

2.2.2 Datenaustausch über E/A-System der Robotersteuerung

Beschreibung

- Die Daten und Signale des Sensorsystems werden über das E/A-System der Robotersteuerung gelesen (\$IN, \$ANIN).
- RSI greift auf die Daten und Signale zu und verarbeitet sie.
- Die verarbeiteten Daten und Signale werden über das E/A-System der Robotersteuerung wieder an das Sensorsystem zurückgegeben (\$OUT, \$ANOUT).
- Die Signale werden über ein Bussystem mit dem E/A-System der Robotersteuerung verschaltet.

Die Signale werden im Sensortakt gelesen und geschrieben. Folgende RSI-Objekte werden dafür verwendet:

- ANIN und DIGIN greifen lesend auf das E/A-System der Robotersteuerung zu und übergeben die Daten und Signale des Sensorsystems an die Signalverarbeitung
- MAP2ANOUT und MAP2DIGOUT greifen auf die verarbeiteten Signale zu und schreiben diese auf das E/A-System der Robotersteuerung.

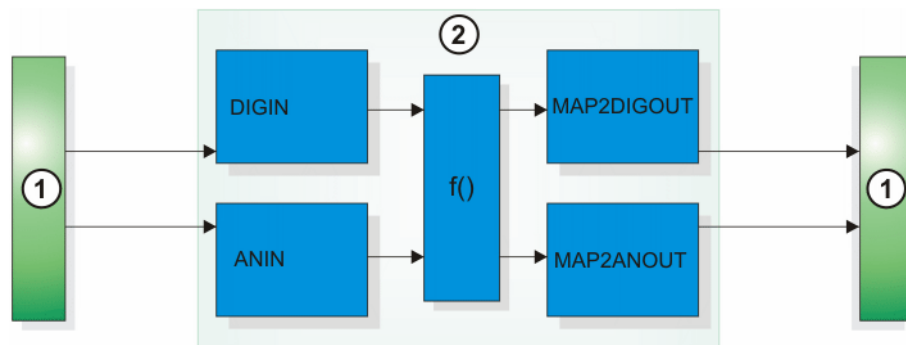


Abb. 2-2: Datenaustausch über E/A-System der Robotersteuerung

- 1 E/A-System der Robotersteuerung
- 2 RSI-Kontext

2.2.3 Datenaustausch über Ethernet

Beschreibung

Beim Datenaustausch über Ethernet ist kein fester Datenrahmen vorgegeben. Der Benutzer muss den Datensatz in einer xml-Datei konfigurieren.

Eigenschaften:

- Zyklische Datenübertragung von der Robotersteuerung an ein Sensorsystem parallel zum Programmablauf (z. B. Positionsdaten, Achswinkel, Betriebsart etc.)
- Zyklische Datenübertragung von einem Sensorsystem an die Robotersteuerung parallel zum Programmablauf (z. B. Sensormesswerte)

Der Datenaustausch über Ethernet wird über das RSI-Objekt ETHERNET realisiert.

Für ETHERNET können bis zu 64 Ein- und Ausgänge definiert werden. Die Signale an den Eingängen werden an das Sensorsystem gesendet. An den Ausgängen stehen die vom Sensorsystem empfangenen Daten zur Verfügung.

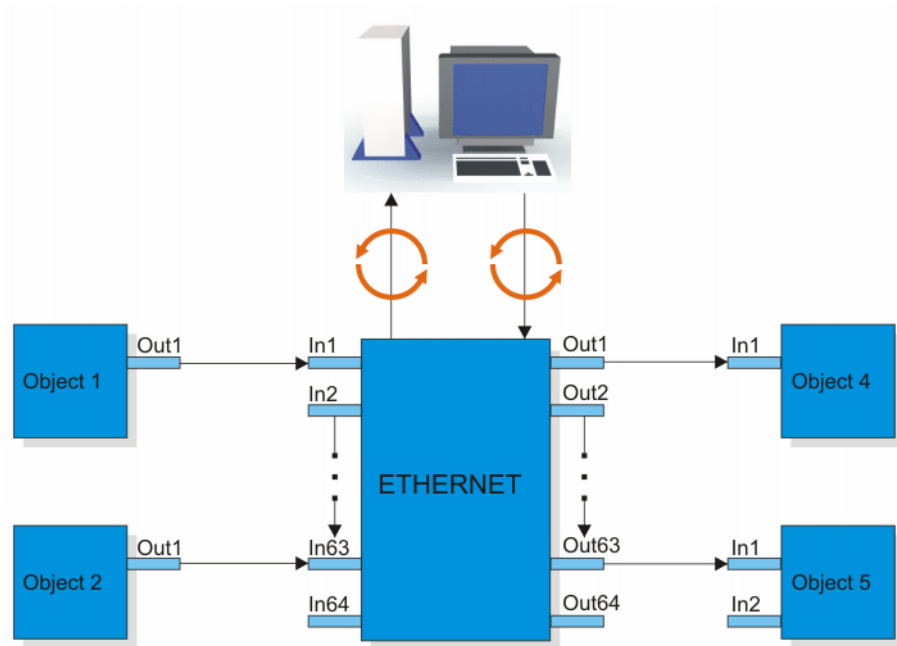


Abb. 2-3: Datenaustausch über Ethernet (Funktionsprinzip)

Mit dem Einschalten der Signalverarbeitung wird ein Kanal vorbereitet, der über das UDP/IP-Protokoll Daten an das Sensorsystem sendet. Die Robotersteuerung leitet den Datenaustausch mit einem Datenpaket ein und sendet im Sensortakt weitere Datenpakete an das Sensorsystem. Das Sensorsystem muss die erhaltenen Datenpakete mit einem eigenen Datenpaket beantworten.

Bei eingeschalteter Signalverarbeitung sendet und empfängt ETHERNET im Sensortakt einen benutzerdefinierten Datensatz im XML-Format. Dieser Datensatz muss in einer xml-Datei konfiguriert werden. Der Name der xml-Datei wird im ETHERNET-Objekt angegeben.

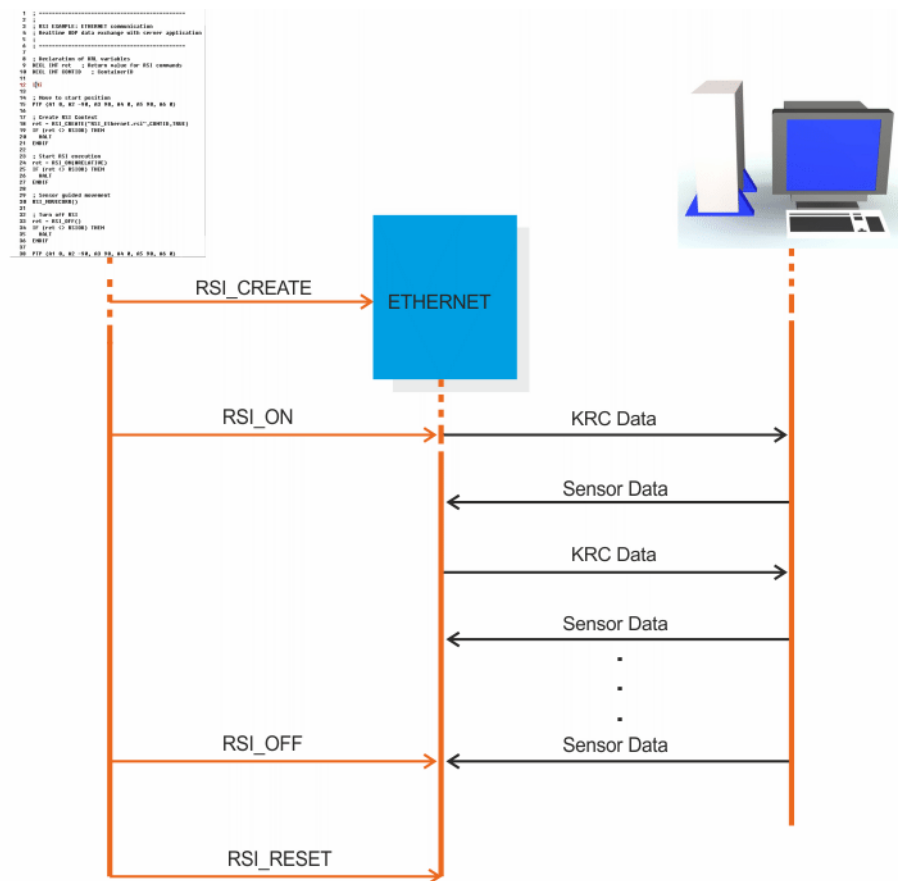


Abb. 2-4: Datenaustausch über Ethernet (Ablauf)

Echtzeitanforderung

Ein am Sensorsystem eintreffendes Datenpaket muss innerhalb des Sensortakts beantwortet werden. Zu spät eingetroffene Pakete werden verworfen.

Wenn die maximale Anzahl der zu spät beantworteten Datenpakete überschritten ist, stoppt der Roboter. Wenn die Signalverarbeitung ausgeschaltet wird, stoppt auch der Datenaustausch.

2.3 Signalverarbeitung

Beschreibung

Eine Signalverarbeitung wird mit RSI-Objekten aufgebaut. Ein RSI-Objekt führt mit seinen Signaleingängen eine bestimmte Funktionalität aus und stellt das Ergebnis an den Signalausgängen bereit.

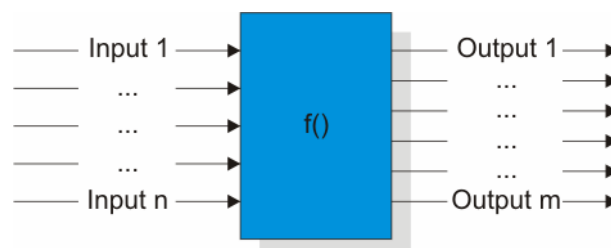


Abb. 2-5: Schematischer Aufbau eines RSI-Objekts

RobotSensorInterface stellt dem Benutzer in einer Bibliothek einen umfangreichen Satz an RSI-Objekten zur Verfügung. Durch die Verknüpfung

der Signal-Ein- und Ausgänge mehrerer RSI-Objekte entsteht ein Signalfluss. Der gesamte Signalfluss wird als RSI-Kontext bezeichnet.

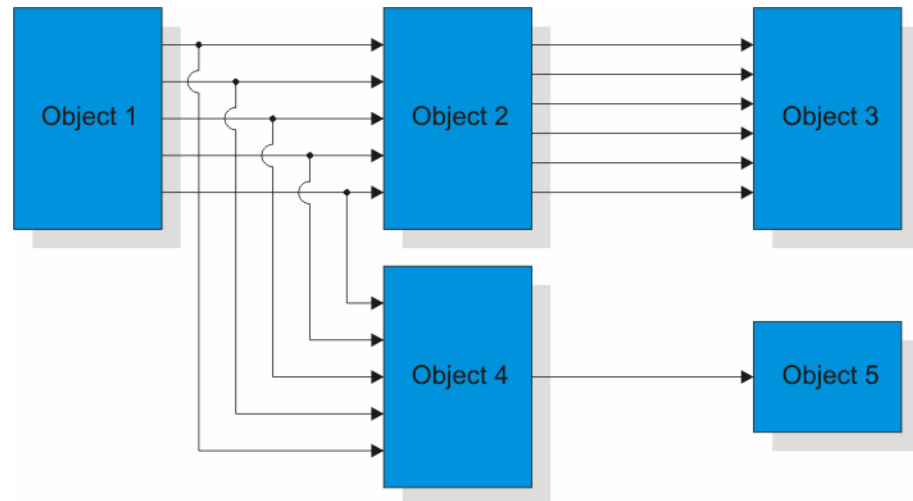


Abb. 2-6: Schematischer Aufbau eines RSI-Kontexts

Der RSI-Kontext wird mit dem grafischen Editor RSIVisual definiert und gespeichert. Im KRL-Programm kann der RSI-Kontext geladen und die zum Programmablauf parallele Signalverarbeitung ein- und ausgeschaltet werden. Die Signalverarbeitung wird im Sensortakt berechnet. Je nach Modus ist der Sensortakt 12 ms (Sensormodus #IPO) oder 4 ms (Sensormodus #IPO_FAST).

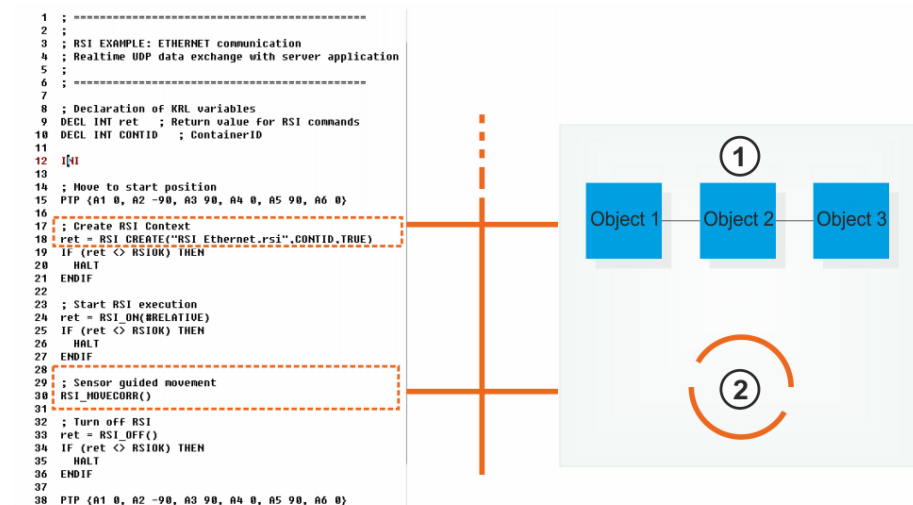


Abb. 2-7: Zusammenspiel KRL-Programm - Signalverarbeitung

1 RSI-Kontext

2 Sensortakt

2.4 Übersicht Sensorkorrektur

Beschreibung

KUKA.RobotSensorInterface 5.0 ermöglicht die kontinuierliche Einflussnahme auf die Roboterbewegung über Sensordaten. Dabei wird im Sensortakt ein Korrekturwert in die aktuelle Sollposition eingerechnet. Die dadurch erzielte Abweichung von der Sollposition ist die Sensorkorrektur.



Die Sensorkorrektur kann nicht für asynchrone Achsen verwendet werden.

HINWEIS**Sachschaden durch Korrekturvorgaben des Sensors**

Sensorkorrekturen beeinflussen die Roboterbewegung direkt. Nicht der Industrieroboter gibt die Bahn vor, sondern der Sensor.

- Die Korrekturvorgaben des Sensors signaltechnisch so aufbereiten, dass am Robotersystem keine mechanischen Beschädigungen, z. B. durch Schwingungen, auftreten können.

Übersicht

Korrekturmöglichkeiten	Beschreibung
Korrekturtyp	Die Korrekturtypen geben an, ob die Sensorkorrektur kartesisch oder achsspezifisch erfolgt. (>>> 2.4.1 "Korrekturtyp" Seite 19)
Korrekturmodus	Der Korrekturmodus gibt an, ob die Sensorkorrektur absolut oder relativ erfolgt. (>>> 2.4.2 "Korrekturmodus" Seite 22)
Korrekturmethode	Die Korrekturmethode gibt an, ob die Sensorkorrektur rein sensorgeführt oder überlagert zu einer geplanten Bahn erfolgt. (>>> 2.4.3 "Korrekturmethode" Seite 24)

Spezialfälle

KUKA.RobotSensorInterface 5.0 kann zusammen mit den folgenden Optionspaketen betrieben werden, es sind jedoch Einschränkungen zu beachten:

- KUKA.RoboTeam 3.2
(>>> [2.4.4 "Sensorkorrektur im RoboTeam-Betrieb" Seite 26](#))
- KUKA.ConveyorTech 8.1
(>>> [2.4.5 "Sensorkorrektur im ConveyorTech-Betrieb" Seite 27](#))

2.4.1 Korrekturtyp**Beschreibung**

Der Korrekturtyp gibt an, ob die Sensorkorrektur kartesisch oder achsspezifisch erfolgt. Zur Einflussnahme auf die Roboterbewegung oder die Bewegung von Einzelachsen stehen verschiedene RSI-Objekte mit unterschiedlichem Korrekturtyp zur Verfügung:

- Achsweise Korrektur
 - AXISCORR zur Korrektur der Roboterachsen A1 bis A6
 - AXISCORREXT zur Korrektur der Zusatzachsen E1 bis E6
- Kartesische Korrektur
 - POSCORR zur Korrektur der Roboterposition

Die zulässige Korrektur wird über RSI-Objektparameter in beide Richtungen begrenzt.

2.4.1.1 Achsweise Korrektur

Die achsweise Korrektur ermöglicht eine Einflussnahme auf die Stellung einzelner Achsen über die Sensorkorrektur. Es können sowohl translatorische als auch rotatorische Achsen korrigiert werden. Je nach Achstyp wird die Sensorkorrektur in Grad oder in Millimeter interpretiert.

2.4.1.2 Kartesische Korrektur

Beschreibung

Die kartesische Korrektur ermöglicht eine Einflussnahme auf die kartesische Roboterposition einzelner Freiheitsgrade über die Sensorkorrektur. Die Sensorkorrektur wird hierzu in einem Korrekturkoordinatensystem (CCS) interpretiert.

Korrekturkoordinatensystem

Der Ursprung des Korrekturkoordinatensystems liegt immer im aktiven TCP, die Orientierung kann von folgenden Referenzkoordinatensystemen übernommen werden:

- BASE
- ROBROOT
- TOOL
- WORLD
- TTS

Beispiel 1

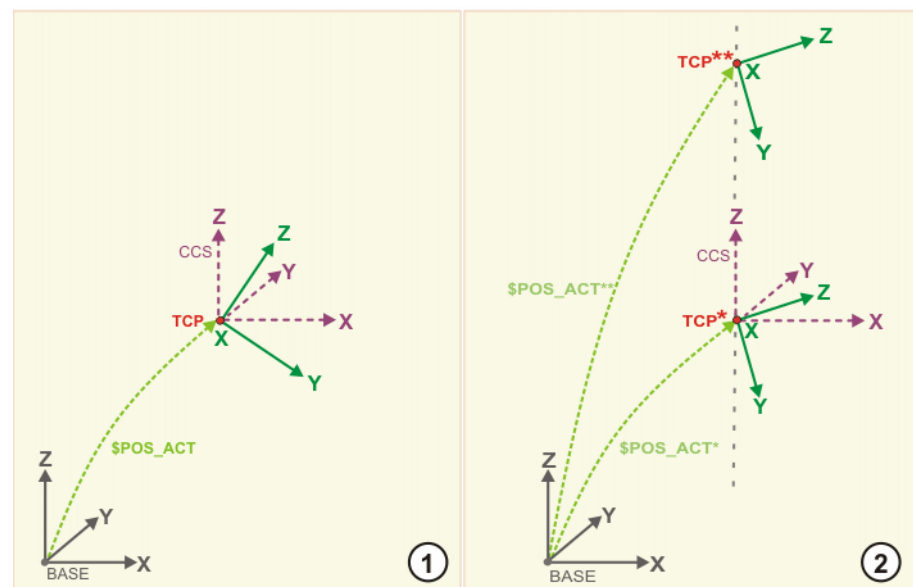
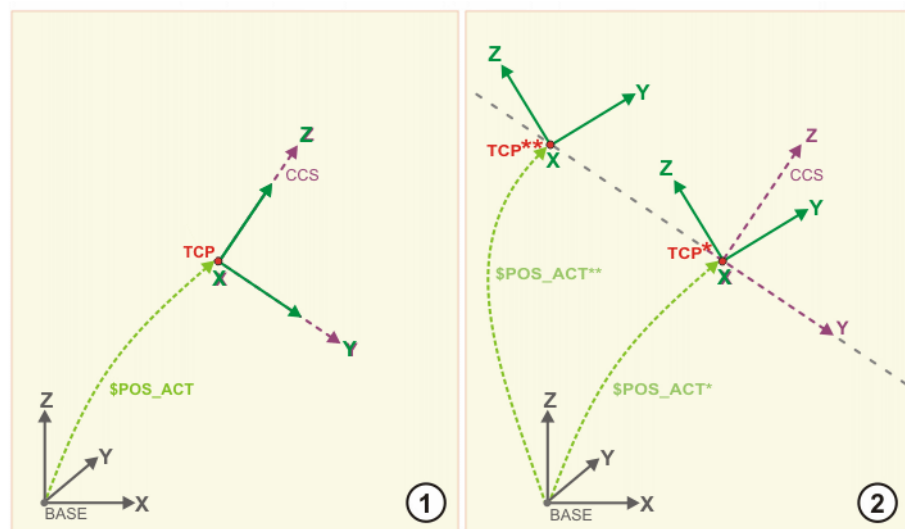


Abb. 2-8: Kartesische Korrektur bezogen auf BASE

Pos.	Beschreibung
1	Ausgangsposition für die kartesische Korrektur • \$POS_ACT: Kartesische Roboterposition • CCS: Korrekturkoordinatensystem im TCP mit der Orientierung von BASE
2	Kartesische Korrektur mit BASE als Korrekturkoordinatensystem • \$POS_ACT*: Um den Korrekturwert verdrehte kartesische Roboterposition – TCP*: Der TCP ist im Korrekturkoordinatensystem um +B verdreht. • \$POS_ACT**: Um den Korrekturwert verschobene und verdrehte kartesische Roboterposition – TCP**: Der TCP ist im Korrekturkoordinatensystem in Richtung +Z verschoben und um +B verdreht.

Beispiel 2**Abb. 2-9: Kartesische Korrektur bezogen auf TOOL**

Pos.	Beschreibung
1	Ausgangsposition für die kartesische Korrektur • \$POS_ACT: Kartesische Roboterposition • CCS: Korrekturkoordinatensystem im TCP mit der Orientierung von TOOL
2	Kartesische Korrektur mit TOOL als Korrekturkoordinatensystem • \$POS_ACT*: Um den Korrekturwert verdrehte kartesische Roboterposition – TCP*: Der TCP ist im Korrekturkoordinatensystem um +C verdreht. • \$POS_ACT**: Um den Korrekturwert verschobene und verdrehte kartesische Roboterposition – TCP**: Der TCP ist im Korrekturkoordinatensystem in Richtung -Y verschoben und um +C verdreht.

2.4.2 Korrekturmodus

Beschreibung

Der Korrekturmodus gibt an, in welcher Form die Sensorkorrektur mit der aktuellen Position des Roboters oder einzelner Achsen verrechnet wird. Er wird über einen Parameter im Befehl RSI_ON() ausgewählt.

Folgende Korrekturmodi stehen zur Verfügung:

- Absolutkorrektur
- Relativkorrektur

2.4.2.1 Absolutkorrektur

Bei der Absolutkorrektur wird die Sensorkorrektur als Absolutwert mit der aktuellen Sollposition des Roboters oder einzelner Achsen verrechnet. Die neue Position ergibt sich aus der Verschiebung der programmierten Sollposition um den aktuellen Korrekturwert.

2.4.2.2 Relativkorrektur

Beschreibung

Bei der Relativkorrektur wird die Sensorkorrektur inkrementell mit der Position des Roboters oder einzelner Achsen verrechnet. Die neue Position ergibt sich aus der Verschiebung der programmierten Sollposition um alle bisherigen Korrekturen und der weiteren Verschiebung um den aktuellen Korrekturwert.

Die Gesamtverschiebung wird mit jedem Sensortakt gespeichert.

Bei einer kartesischen Relativkorrektur erfolgt dies in einem speziellen Koordinatensystem, dem Integrationskoordinatensystem.

Integrationskoordinatensystem

Bei kartesischen Relativkorrekturen wird die inkrementell entstehende Gesamtkorrektur in einem speziellen Integrationskoordinatensystem gespeichert. Die Relativkorrekturen aller aktiven Korrekturobjekte werden dabei zusammengeführt.

In jedem Sensortakt wird hierzu die neu zu verrechnende Relativkorrektur vom Korrekturkoordinatensystem in das Integrationskoordinatensystem umgerechnet und auf die bisher gespeicherte Gesamtkorrektur addiert.

Die Orientierung des Integrationskoordinatensystems kann von folgenden Referenzkoordinatensystemen übernommen werden:

- BASE
- ROBROOT
- TOOL
- WORLD
- TTS



Das Integrationssystem kann nicht verändert werden, wenn die Signalverarbeitung bereits aktiv ist.

Verändert das Integrationskoordinatensystem während der Signalverarbeitung die Orientierung, so wird die bisher erfolgte Gesamtkorrektur entsprechend mitgeführt.

Die Orientierung des Integrationskoordinatensystems wird in jedem Sensortakt ermittelt. Die tatsächliche Position des Roboters in Bezug auf das

Weltkoordinatensystem ergibt sich aus der Verschiebung der für diesen Sensortakt geltenden Sollposition um die Gesamtkorrektur im für diesen Sensortakt geltenden Integrationskoordinatensystem.

Die Wahl des Integrationskoordinatensystems hat somit je nach Applikation direkten Einfluss auf die tatsächliche Position oder Bahn, die durch Einsatz der Sensorkorrektur entsteht. Dies gilt insbesondere für sensitive Bahnapplikationen.

Als Integrationssystem sollte daher das Referenzkoordinatensystem gewählt werden, auf das sich die Sensorkorrektur im realen Prozess bezieht. In den meisten Fällen bietet es sich an, für das Integrationssystem dasselbe Referenzkoordinatensystem zu wählen, das auch für das Korrekturkoordinatensystem gewählt wurde.

Beispiel 1

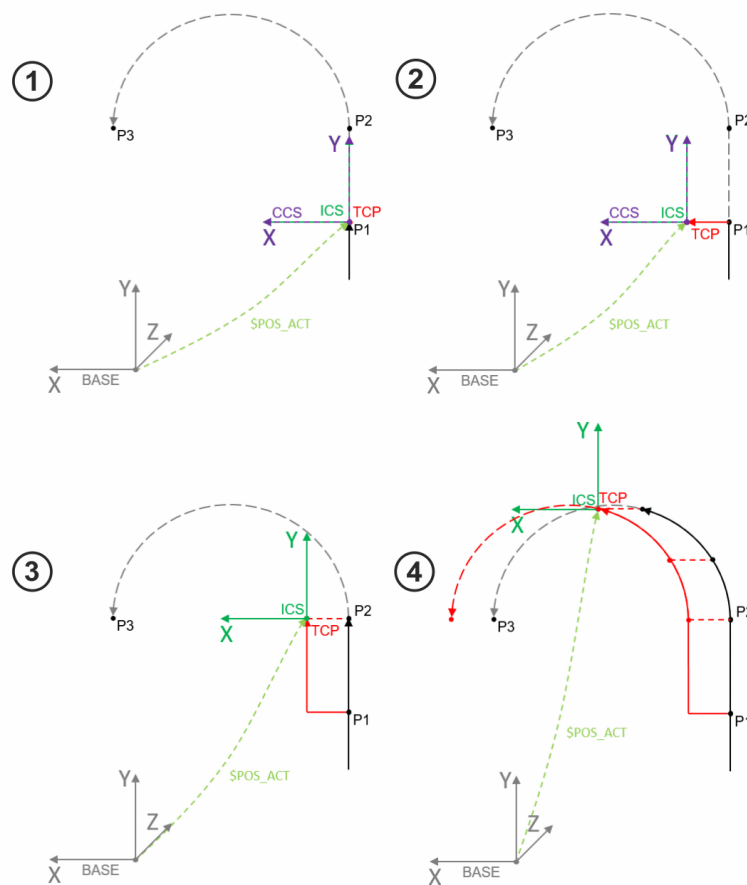
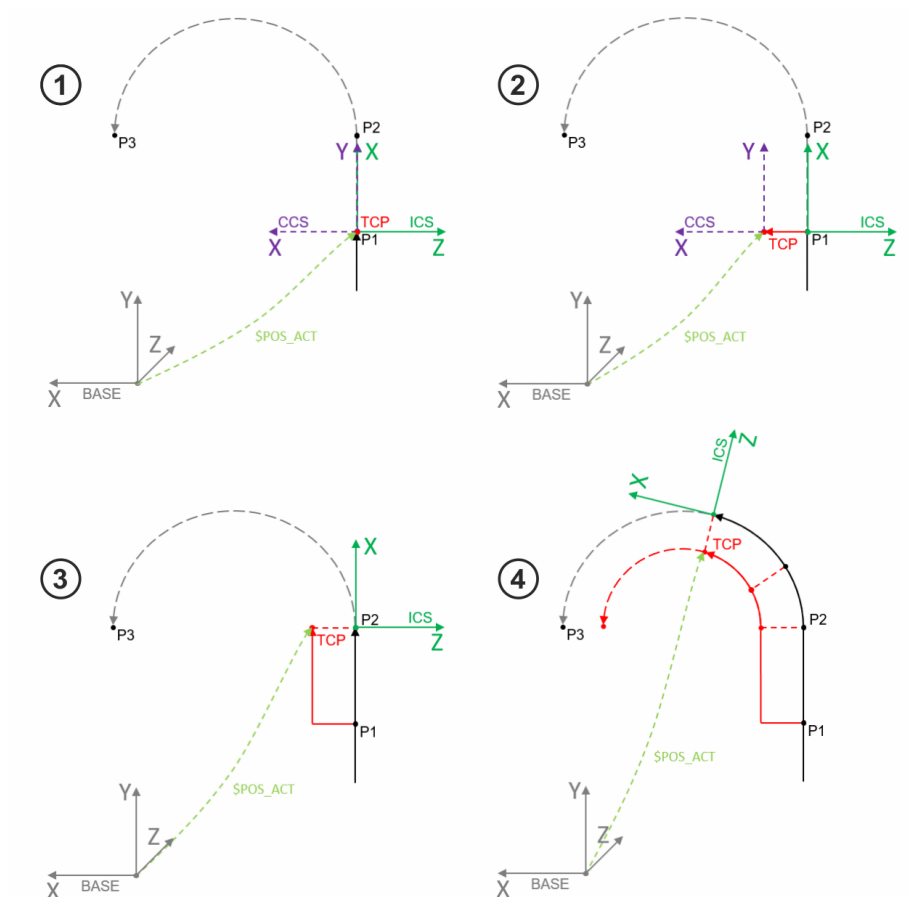


Abb. 2-10: Relativkorrektur Beispiel 1

Pos.	Beschreibung
1	Der Roboter folgt einer geplanten Bahn (grau gestrichelte Linie). - Korrektursystem #BASE (violett gestrichelte Linie) - Integrationssystem #BASE (grüne Linie)
2	Es wird in positiver X_{BASE} -Richtung korrigiert und im selben Koordinatensystem (BASE) gespeichert.
3	Der Roboter folgt der geplanten Bahn mit der Korrektur im Integrationssystem. Die Korrektur bleibt in X_{BASE} erhalten.
4	Bei einem Kreis wird #BASE nicht implizit umorientiert. Der Kreis wird deshalb mit der gespeicherten Korrektur gefahren.

Beispiel 2**Abb. 2-11: Relativkorrektur Beispiel 2**

Pos.	Beschreibung
1	Der Roboter folgt einer geplanten Bahn (grau gestrichelte Linie). - Korrektursystem #BASE (violett gestrichelte Linie) - Integrationssystem #TTS (grüne Linie)
2	Es wird in positiver X_{BASE} -Richtung korrigiert und in negativer Z_{TTS} -Richtung gespeichert.
3	Der Roboter folgt der geplanten Bahn mit der Korrektur im Integrationssystem. Die Korrektur bleibt in Z_{TTS} erhalten.
4	Bei einem Kreis sorgt die Umlagerung von #TTS dafür, dass die Roboterposition parallel zur Kreisbahn verläuft.

2.4.3 Korrekturmethode**Beschreibung**

Die Korrekturmethode gibt an, ob die Sensorkorrektur rein sensorgeführt oder überlagert zu einer geplanten Bahn erfolgt. Erfolgt nach dem Befehl `RSI_ON()` ein Bewegungsbefehl, erfolgt die Sensorkorrektur überlagert zur programmierten Bewegung. Soll eine rein sensorgeführte Sensorkorrektur des Roboters erfolgen, ist der Befehl `RSI_MOVECORR()` zu verwenden.

Überlagerte Sensorkorrektur

Die Korrekturwerte werden auf die Stützpunkte einer programmierten Bahn aufgeschaltet. Der Bahnverlauf kann auf der Basis von absoluten oder relativen Korrekturwerten korrigiert werden.



Wenn die Signale im IPO-Modus verarbeitet werden, kann der Bahnverlauf nur auf der Basis von LIN- und CIRC-Bewegungen korrigiert werden.

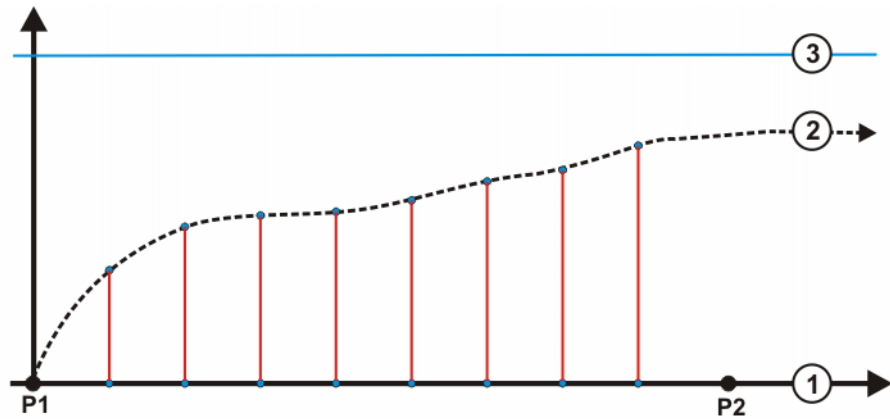


Abb. 2-12: Bahnkorrektur auf Basis von Absolutwerten

- 1 Programmierte Bahn
- 2 Korrigierte Bahn
- 3 Maximale Gesamtkorrektur
- Rot Absoluter Korrekturwert

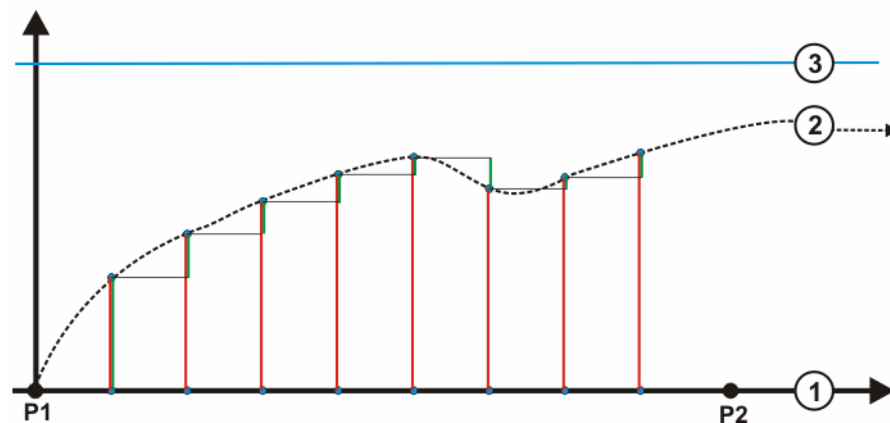


Abb. 2-13: Bahnkorrektur auf Basis von Relativwerten

- 1 Programmierte Bahn
- 2 Korrigierte Bahn
- 3 Maximale Gesamtkorrektur
- Rot Gesamtkorrektur
- Grün Relativer Korrekturwert

Sensorgeführte Bewegung

Mithilfe des Befehls RSI_MOVECORR() kann eine sensorgeführte Bewegung programmiert werden. Ausgehend von einem Startpunkt fährt der Roboter keinen definierten Zielpunkt an, sondern wird rein korrekturgesteuert auf der Basis von Sensordaten verfahren.

Die sensorgeführte Bewegung kann auf der Basis von absoluten oder relativen Korrekturwerten ausgeführt werden.

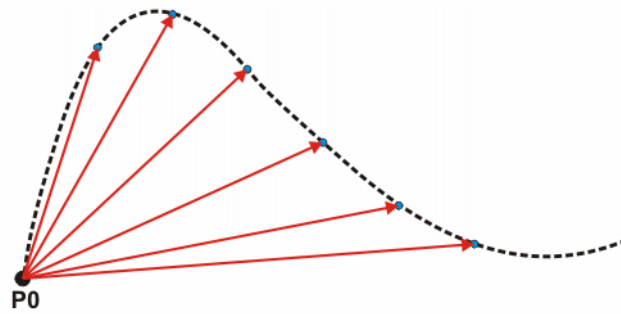


Abb. 2-14: Sensorgeführte Bewegung auf Basis von Absolutwerten

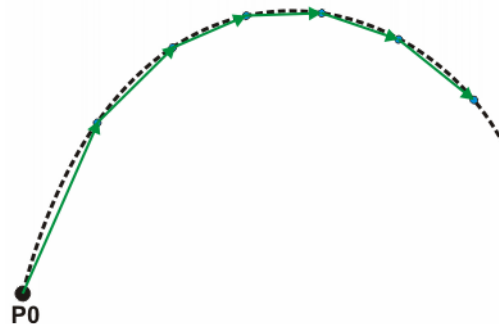


Abb. 2-15: Sensorgeführte Bewegung auf Basis von Relativwerten

2.4.4 Sensorkorrektur im RoboTeam-Betrieb

RobotSensorInterface kann zusammen mit RoboTeam betrieben werden, es gelten jedoch Einschränkungen:

- Die Sensorkorrektur einer Master-Kinematik ($\$LK_Master[]$) ist im Sensormodus $\#IPO$ und $\#IPO_FAST$ möglich.
- Die Sensorkorrektur einer Slave-Kinematik ($\$LK_Slave[]$) ist ausschließlich im Sensormodus $\#IPO$ möglich.
- Eine Sensorkorrektur ist im Sensormodus $\#IPO$ nur für CP-Bewegungen möglich. Ist eine Sensorkorrektur für Spline und PTP-Befehle erforderlich, muss der Sensormodus $\#IPO_FAST$ verwendet werden.
- Eine Sensorkorrektur der Slave-Kinematik im Sensormodus $\#IPO_FAST$ ist nicht möglich. Wird versucht die Slave-Kinematik im Sensormodus $\#IPO_FAST$ zu betreiben, wird folgende Meldung angezeigt:
 - *Aktive Sensorik ist während {Unverträgliche Funktionalität} nicht möglich.* Im genannten Fall ist die {Unverträgliche Funktionalität} die Funktion $LK()$.
- Die Sensorkorrektur einer Slave-Kinematik hat in keinem Fall Auswirkungen auf die Bewegung der Master-Kinematik.

Die Sensorkorrektur der Master-Kinematik beeinflusst die Bewegung der Slave-Kinematik in Abhängigkeit davon, welcher Sensormodus auf der Master-Steuerung aktiv ist:

	LK-Master ohne Sensorik	LK-Master im Sensormodus #IPO	LK-Master im Sensormodus #IPO_FAST
LK-Slave ohne Sensorik	(gewöhnliche Robo-Team-Anwendung ohne Signalverarbeitung durch RSI)	Der LK-Slave folgt der LK-Base (geplante Bahn) und der zusätzlichen Sensorkorrektur des LK-Masters.	- Der LK-Slave folgt nur der LK-Base (geplante Bahn) des LK-Masters. - Der LK-Slave folgt nicht der zusätzlichen Sensorkorrektur des LK-Masters.
LK-Slave im Sensormodus #IPO	- Der LK-Slave (geplante Bahn) folgt der LK-Base des LK-Masters. - Eine Sensorkorrektur des LK-Slaves ist möglich.	- Der LK-Slave folgt der LK-Base (geplante Bahn) und der zusätzlichen Sensorkorrektur des LK-Masters. - Eine Sensorkorrektur des LK-Slaves ist möglich und überlagert sich mit der des LK-Masters.	- Der LK-Slave folgt der LK-Base (geplante Bahn) des LK-Masters. - Der LK-Slave folgt nicht der zusätzlichen Sensorkorrektur des LK-Masters. - Eine Sensorkorrektur des LK-Slaves ist möglich.

2.4.5 Sensorkorrektur im ConveyorTech-Betrieb

Beschreibung

Es ist möglich, die Signalverarbeitung von RSI auszuführen, während der Roboter aktiv ein Bauteil auf einem Conveyor verfolgt. Da das Basiskoordinatensystem während der Bauteilverfolgung kontinuierlich in Bewegung sein kann, müssen einige Rahmenbedingungen eingehalten werden:

- Auswahl des Integrationskoordinatensystems:
Die Koordinatensysteme BASE und TTS können nicht als Referenzkoordinatensystem für das Integrationskoordinatensystem verwendet werden. Das Referenzkoordinatensystem TOOL kann nur verwendet werden, wenn sich die Tooldaten während der Kraft-Momenten-Aufgabe bis zum Absynchronisieren des Conveyors nicht ändern.
- Beenden der Kraft-Momenten-Aufgabe:
Die Signalverarbeitung darf nicht beendet werden, solange der Roboter ein Bauteil verfolgt. Die Ausführung eines RSI-Kontexts darf ausschließlich deaktiviert werden. Das Beenden der Signalverarbeitung darf erst erfolgen, wenn die Conveyor-Basis verlassen wurde.

Programmierbeispiel

In diesem Beispiel werden nacheinander zwei Sensorikaufgaben durchgeführt.

```

1 DEF CONV_MOV (Z_CONV_NBR:IN, Z_MOV_NBR:IN)
2 INT ret
3 INT contId1
4 INT contId2
5 INI CONV_MOV
6
7 CONVEYER 1 MOVEMENT GROUP 1
8
9 ; synchronize with conveyor

```

```

10 LIN P2 Vel=0.5 m/s CPDAT3 Tool[1]:Tool1 Base[1]:Flange
   Base (Conv1)
11
12 ; load and activate RSI-Job 1
13 ret = RSI_CREATE("Context1", contId1, TRUE)
14 IF (ret <> RSIOK) THEN
15     halt
16 ENDIF
17 ret = RSI_ON()
18 IF (ret <> RSIOK) THEN
19     halt
20 ENDIF
21
22 LIN P3 Vel=0.2 m/s CPDAT4 Tool[1]:Tool1 Base[1]:Flange
   Base (Conv1)
23
24 ; stop RSI-Job 1
25 ret = RSI_Disable(contId1)
26 IF (ret <> RSIOK) THEN
27     halt
28 ENDIF
29
30 ; load and activate RSI-Job 2
31 ret = RSI_Create("Context2", contId2, TRUE)
32 IF (ret <> RSIOK) THEN
33     halt
34 ENDIF
35
36 LIN P4 CONT Vel=0.2 m/s CPDAT5 Tool[1]:Tool1
   Base[1]:Flange Base (Conv1)
37 LIN P5 CONT Vel=0.2 m/s CPDAT6 Tool[1]:Tool1
   Base[1]:Flange Base (Conv1)
38
39 ; stop RSI-Job 2
40 ret = RSI_DISABLE(contId2)
41 IF (ret <> RSIOK) THEN
42     halt
43 ENDIF
44
45 ; desynchronize from conveyor
46 LIN P6 CONT Vel=0.5 m/s CPDAT7 Tool[1]:Tool1 Base[0]
47
48 ; deactivate RSI execution
49 ret = RSI_OFF()
50 IF (ret <> RSIOK) THEN
51     halt
52 ENDIF
53
54 ; delete all RSI jobs
55 ret = RSI_RESET()
56 IF (ret <> RSIOK) THEN
57     halt
58 ENDIF

```

Zeile	Beschreibung
10	Aufsynchronisieren auf die Conveyor-Basis
13 ... 20	Starten der Sensorik
22	Durchführen der ersten Sensorikaufgabe
24 ... 28	Deaktivieren der Sensorik

Zeile	Beschreibung
30 ... 43	Durchführen der zweiten Sensorikaufgabe
46	Absynchronisieren von Conveyor-Basis
48 ... 58	Abschalten der Sensorik

Auswahl der Korrektur- und Integrationssysteme

Im Sensormodus #IPO_FAST ist die Auswahl an Korrektur- und Integrationssystemen eingeschränkt.

Für das Integrationssystem dürfen nur Koordinatensysteme gewählt werden, die unabhängig von der Conveyor-Base sind und sich nicht sprunghaft ändern. Dies sind:

- #WORLD
- #ROBROOT
- #TCP (wenn kein Toolwechsel erfolgt)

Für das Korrektursystem sind die zulässigen Koordinatensysteme abhängig vom gewählten Korrekturmodus.

Bei der Absolutkorrektur können folgende Systeme gewählt werden, sofern sich diese nicht sprunghaft ändern:

- #WORLD
- #ROBROOT
- #TCP (wenn kein Toolwechsel erfolgt)

Bei der Relativkorrektur können alle Systeme gewählt werden:

- #WORLD
- #ROBROOT
- #TCP
- #BASE
- #TTS

2.5 Bestimmungsgemäße Verwendung und Fehlanwendung

Verwendung

Das Optionspaket KUKA.RobotSensorInterface 5.0 (RSI) dient als universelle Schnittstelle zur Realisierung von Industrieroboterapplikationen im Bereich Industrie und Forschung. Ziel ist es, durch eine Erweiterung der Basisfunktionalität der System Software Applikationen zu ermöglichen, die eine Echtzeitverarbeitung von Daten externer Systeme und die kontinuierliche Einflussnahme auf die Roboterbewegung erfordern. RSI darf hierzu für folgende Vorgänge eingesetzt werden:

- Zyklische Datenkommunikation zwischen einem externen System und der Signalverarbeitung von RSI zur Programmlaufzeit des Roboter-Interpreters
Der zyklische Austausch erfolgt in einem festgelegten Zeitraster.
- Aufbereiten der Daten durch die Signalverarbeitung von RSI
- Einflussnahme auf die Roboterbewegung oder den Programmablauf durch die aufbereiteten Sensordaten der Signalverarbeitung

Bei Verwendung des Optionspakets sind folgende Punkte zu beachten:

- Das Optionspaket darf ausschließlich unter den dafür spezifizierten Systemvoraussetzungen betrieben werden.

- Sensorkorrekturen beeinflussen die Roboterbewegung direkt. Die sich aus der üblichen Bahnplanung ergebende Bewegung des Industrieroboters wird in Abhängigkeit von Sensormesswerten während der Bewegung verändert.

Der Anwender ist dafür verantwortlich, dass die Korrekturvorgaben des Sensors signaltechnisch so aufbereitet werden, dass am Robotersystem keine mechanischen Beschädigungen, z. B. durch Schwingungen, auftreten können.

- Das Optionspaket kann für Applikationen im Bereich der Mensch-Roboter-Kollaboration (MRK) eingesetzt werden.

Für MRK-Applikationen gelten spezielle Sicherheitsvorschriften. Der Anwender ist dafür verantwortlich, dass diese Sicherheitsvorschriften eingehalten werden.

- Die Kommunikation über Ethernet ist kein vollständiger Ersatz für ein Feldbussystem.

Zur bestimmungsgemäßen Verwendung gehört auch die Beachtung der Inbetriebnahme- und Konfigurationsanweisung in der Dokumentation des Optionspakets sowie die Beachtung der Montage- und Betriebsanleitungen der eingesetzten Zellenkomponenten.

Fehlanwendung

Alle von der bestimmungsgemäßen Verwendung abweichenden Anwendungen gelten als Fehlanwendung und sind unzulässig. Sie führen zum Verlust von Gewährleistungs- und Haftungsansprüchen. Für Schäden, die aus einer Fehlanwendung resultieren, haftet KUKA nicht.

3 Sicherheit

Diese Dokumentation enthält Sicherheitshinweise, die sich spezifisch auf das hier beschriebene Optionspaket beziehen.

Die grundlegenden Sicherheitsinformationen zum Industrieroboter sind im Kapitel "Sicherheit" der Bedien- und Programmieranleitung der Systemsoftware zu finden.



Sicherheitsrelevante Informationen beachten

Die sichere Nutzung dieses Produkts erfordert die Kenntnis und Einhaltung grundlegender Sicherheitsmaßnahmen. Tod, schwere Verletzungen oder Sachschäden können sonst die Folge sein.

- Das Kapitel "Sicherheit" in der Bedien- und Programmieranleitung der Systemsoftware muss beachtet werden.

3.1 Sicherheitshinweise

Sensorbetrieb

- Bei falscher Anwendung der RobotSensorInterface kann es zu Personen- und Sachschäden kommen.
- Im Sensorbetrieb kann sich der Roboter in folgenden Fällen unerwartet bewegen:
 - Bei falsch parametrisierten RSI Objekten
 - Bei einem Hardware-Fehler (z. B. falsche Verkabelung, Kabelbruch in der Sensorleitung oder Fehlfunktion des Sensors)
- Bei unerwarteten Bewegungen kann es zu schweren Verletzungen und erheblichen Sachschäden kommen. Der Systemintegrator ist verpflichtet, das Verletzungsrisiko für sich und andere Personen sowie Sachschaden durch den Einsatz geeigneter Sicherheitsmaßnahmen zu minimieren, z. B. durch die Begrenzung des Arbeitsraumes.
- Beim Start der Signalverarbeitung mit RobotSensorInterface gibt die Sicherheitssteuerung in der Betriebsart T1 oder T2 eine Quittiermeldung aus:

!!! Achtung –Sensorkorrektur wird aktiviert !!!

Arbeitsraumbegrenzung

- Die Achsbereiche aller Roboterachsen sind über einstellbare Software-Endschalter begrenzt. Diese Software-Endschalter sind so einzustellen, dass der Arbeitsraum des Roboters auf den für den Prozess minimal notwendigen Bereich begrenzt ist.
- Mit der Systemsoftware können maximal 8 kartesische und 8 achsspezifische Arbeitsräume konfiguriert werden. Der Systemintegrator muss die Arbeitsräume so konfigurieren, dass sie auf den für den Prozess minimal notwendigen Bereich begrenzt sind. Damit wird die Gefahr von Schäden durch unerwartete Bewegungen im Sensorbetrieb auf ein Minimum reduziert.



Weitere Informationen zur Konfiguration von Arbeitsräumen sind in der Bedien- und Programmieranleitung für Systemintegratoren zu finden.

Sensorkorrektur

RobotSensorInterface überwacht und begrenzt die maximale Sensorkorrektur. Es kann jedes einzelne Korrekturobjekt überwacht werden sowie die Gesamtkorrektur über alle Korrekturobjekte.

Objektbezogene Sensorkorrekturen sind standardmäßig auf maximal ± 5 mm oder 5° begrenzt, die Gesamtkorrektur auf maximal ± 6 mm oder 6° .

Bei Überschreiten einer objektbezogenen Korrektur läuft die Signalverarbeitung weiter und die Korrektur wird automatisch auf die zulässige Maximalkorrektur begrenzt. Bei Überschreiten der zulässigen Gesamtkorrektur wird die Signalverarbeitung gestoppt.

4 Installation

Das Optionspaket kann entweder über die smarHMI oder über WorkVisual auf der Robotersteuerung installiert werden.

4.1 Systemvoraussetzungen

Hardware

- Robotersteuerung KR C5
- Für den Datenaustausch über Ethernet:
 - Prozessorgestütztes externes System mit echtzeitfähigem Betriebssystem und echtzeitfähiger Netzwerkkarte mit 100 MBit im Voll duplexmodus
 - Mikroprozessorgestützter Sensor mit echtzeitfähiger Netzwerkkarte bei Sensorikapplikationen
 - Netzkabel für Switch, Hub oder gekreuztes Netzkabel zur direkten Verbindung
- Für den Datenaustausch über ein E/A-System:
 - Bussystem, z. B. PROFINET
- Empfohlene Roboter:
 - KUKA 6-Achs-Roboter
 - Asynchrone Achsen werden nicht unterstützt.
 - Den Einsatz anderer Roboter nur nach Rücksprache mit der KUKA Deutschland GmbH planen. (>>> [13 "KUKA Service" Seite 137](#))
- Laptop/PC zum Arbeiten mit WorkVisual

Software

Laptop/PC:

- KUKA.WorkVisual 6.0.x

Robotersteuerung:

- KUKA System Software 8.7.x

Die Nummern der erforderlichen Releases/Versionen müssen der Datei ReleaseNotes.txt entnommen werden. Die Datei befindet sich auf dem Datenträger mit KUKA.RobotSensorInterface 5.0.

KRL-Ressourcen:

Für RSI-Korrekturen im IPO-Modus müssen folgende KRL-Ressourcen frei sein:

KRL-Ressource	Nummer
Funktionsgenerator	1

Kompatibilität

KUKA.RobotSensorInterface 5.0 darf nicht zusammen mit einem oder mehreren der folgenden Optionspakete auf derselben Robotersteuerung installiert sein:

- KUKA.ServoGun Basic
- KUKA.ServoGun Advanced
- KUKA.EqualizingTech

KUKA.RobotSensorInterface 5.0 kann zusammen mit den folgenden Optionspaketen betrieben werden, es sind jedoch Einschränkungen zu beachten:

- KUKA.RoboTeam
(>>> [2.4.4 "Sensorkorrektur im RoboTeam-Betrieb" Seite 26](#))
- KUKA.ConveyorTech
(>>> [2.4.5 "Sensorkorrektur im ConveyorTech-Betrieb" Seite 27](#))

4.2 Installation über WorkVisual

4.2.1 KUKA.RobotSensorInterface 5.0 installieren oder updaten

Beschreibung

Das Optionspaket wird in WorkVisual installiert und dem Projekt hinzugefügt. Beim Übertragen des Projekts auf die Robotersteuerung wird das Optionspaket dort automatisch installiert.

Bei einem Update muss die vorherige Version des Optionspakets in WorkVisual zuerst deinstalliert werden.



Es wird empfohlen, vor dem Update einer Software alle zugehörigen Daten zu archivieren.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.
- Netzwerkverbindung zur Robotersteuerung
- Optionspaket liegt als KOP-Datei vor.

Vorgehensweise

1. Nur bei einem Update: Die vorherige Version des Optionspakets KUKA.RobotSensorInterface 5.0 in WorkVisual deinstallieren.
2. Das Optionspaket KUKA.RobotSensorInterface 5.0 in WorkVisual installieren.
3. Das aktive Projekt von der Robotersteuerung laden.
4. Das Optionspaket KUKA.RobotSensorInterface 5.0 in das Projekt einfügen.
5. Das Optionspaket wie benötigt in WorkVisual konfigurieren.
(>>> [6.1 "Ein-/Ausgänge von RSI in WorkVisual verschalten" Seite 41](#))
6. Das Projekt von WorkVisual auf die Robotersteuerung übertragen und aktivieren.
7. Auf der smartHMI wird die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* angezeigt. Bei der Aktivierung wird das aktive Projekt überschrieben. Wenn kein relevantes Projekt überschrieben wird: Die Abfrage mit **Ja** bestätigen.
8. Auf der smartHMI wird eine Übersicht mit den Änderungen und eine Sicherheitsabfrage angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird installiert und die Robotersteuerung führt einen Neustart durch.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

LOG-Datei

Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.2.2 Optionspaket deinstallieren**Beschreibung**

Das Optionspaket kann über WorkVisual deinstalliert werden.

Es wird empfohlen, vor der Deinstallation einer Software alle zugehörigen Daten zu archivieren.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.
- Netzwerkverbindung zur Robotersteuerung

Vorgehensweise

1. Das Projekt von der Robotersteuerung laden.
2. Das Optionspaket aus dem Projekt entfernen. Ein Fenster mit Änderungen wird angezeigt.
3. Das Projekt von WorkVisual auf die Robotersteuerung übertragen und aktivieren.
4. Die Sicherheitsabfrage auf der smarHMI *Wollen Sie die Aktivierung des Projektes [...] zulassen?* mit **Ja** bestätigen.
5. Auf der smarHMI wird eine Übersicht mit den Änderungen und eine Sicherheitsabfrage angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird deinstalliert und die Robotersteuerung führt einen Neustart durch.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

LOG-Datei

Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.3 Installation über smarHMI**4.3.1 KUKA.RobotSensorInterface 5.0 installieren oder updaten**

Es wird empfohlen, vor dem Update einer Software alle zugehörigen Daten zu archivieren.



Bei einem Update wird die bestehende Konfiguration automatisch übernommen. Wenn dies nicht gewünscht ist, muss die bestehende Version erst deinstalliert werden.

Voraussetzung

- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
Mindestens jedoch Benutzergruppe Experte
- Betriebsart T1 oder T2

- Es ist kein Programm angewählt.
- USB-Stick mit dem Optionspaket (KOP-Datei)

HINWEIS**Datenverlust durch USB-Sticks von Fremdherstellern**

Wenn bei Tätigkeiten an der Robotersteuerung USB-Sticks von anderen Herstellern als KUKA verwendet werden, kann dies zu Datenverlust führen.

- Für Tätigkeiten an der Robotersteuerung, die einen USB-Stick erfordern, einen KUKA USB-Stick verwenden.
Die KUKA USB-Sticks sind für den Einsatz an der Robotersteuerung validiert.

Vorgehensweise

1. Den USB-Stick an der Robotersteuerung oder am smartPAD anstecken.
2. Im Hauptmenü **Inbetriebnahme** > **Zusatzsoftware** wählen.
3. Auf **Neue Software** drücken: In der Spalte **Name** muss der Eintrag "KUKA.RobotSensorInterface 5.0" angezeigt werden und in der Spalte **Pfad** das Laufwerk **E:** oder **K:**.
Wenn nicht, auf **Aktualisieren** drücken.
4. Wenn die genannten Einträge jetzt angezeigt werden, weiter mit Schritt 5.
Wenn nicht, muss erst der Pfad, von dem installiert werden soll, konfiguriert werden:
 - a. Auf die Schaltfläche **Konfigurieren** drücken.
 - b. Im Bereich **Installationspfade für Optionen** eine Zeile markieren.
Hinweis: Wenn die Zeile bereits einen Pfad enthält, wird dieser überschrieben.
 - c. Auf **Pfadauswahl** drücken. Die vorhandenen Laufwerke werden angezeigt.
 - d. Wenn der Stick an der Robotersteuerung angesteckt ist: Auf **E:** das Verzeichnis, in dem die Software liegt, markieren.
Wenn der Stick am smartPAD angesteckt ist: **K:** statt **E:**
 - e. Auf **Speichern** drücken. Der Bereich **Installationspfade für Optionen** wird wieder angezeigt. Er enthält jetzt den neuen Pfad.
 - f. Die Zeile mit dem neuen Pfad markieren und nochmal auf **Speichern** drücken.
5. Bei "KUKA.RobotSensorInterface 5.0" das Häkchen setzen und auf **Installieren** drücken. Den Installationshinweis mit **OK** bestätigen.
6. Die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* wird angezeigt. Bei der Aktivierung wird das aktive Projekt überschrieben. Wenn kein relevantes Projekt überschrieben wird: Die Abfrage mit **Ja** bestätigen.
7. Eine Übersicht mit den Änderungen und eine Sicherheitsabfrage wird angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird installiert und die Robotersteuerung führt einen Neustart durch.
8. Den Stick abziehen.

LOG-Datei

Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.3.2 KUKA.RobotSensorInterface 5.0 deinstallieren



Es wird empfohlen, vor der Deinstallation einer Software alle zugehörigen Daten zu archivieren.

Voraussetzung

- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
Mindestens jedoch Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.

Vorgehensweise

1. Im Hauptmenü **Inbetriebnahme** > **Zusatzsoftware** wählen.
2. Bei KUKA.RobotSensorInterface 5.0 das Häkchen setzen und auf **Deinstallieren** drücken. Die Sicherheitsabfrage mit **Ja** beantworten.
3. Die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* mit **Ja** bestätigen.
4. Eine Übersicht mit den Änderungen und eine Sicherheitsabfrage wird angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird deinstalliert und die Robotersteuerung führt einen Neustart durch.

LOG-Datei

Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

5 Bedienung

5.1 Benutzerrechte anzeigen

Beschreibung

Die Funktionen einer Software sind verschiedenen Funktionsgruppen zugeordnet. Über die Rechteverwaltung ist für jede Funktionsgruppe festgelegt, welche Benutzergruppe mindestens erforderlich ist, um eine Funktion aus dieser Gruppe auszuführen.

- Der Administrator kann den Funktionsgruppen andere Benutzergruppen zuordnen.
- Die Zuordnung der einzelnen Funktionen zu den Funktionsgruppen kann nicht geändert werden.
- Jeder Benutzer kann anzeigen, welche Benutzergruppe aktuell welcher Funktionsgruppe zugeordnet ist.



Weitere Informationen zu Funktionsgruppen und Benutzerrechten sind in der Bedien- und Programmieranleitung der KUKA System Software (KSS) zu finden.

Wenn für eine Vorgehensweise eine Funktionsgruppe relevant ist, ist sie jeweils unter **Voraussetzung** angegeben.

Vorgehensweise

Die Benutzergruppen, die den Funktionsgruppen zugeordnet sind, können angezeigt werden:

1. Im Hauptmenü **Inbetriebnahme** > **Rechteverwaltung** wählen.
Das Fenster **Rechteverwaltung** öffnet sich.
2. Die Registerkarte **Funktionsgruppen** wählen.

6 Konfiguration

6.1 Ein-/Ausgänge von RSI in WorkVisual verschalten

Beschreibung

Die Ein-/Ausgänge von RSI werden im Verschaltungseditor von WorkVisual mit den Ein-/Ausgängen eines Feldbusteilnehmers oder mit den Ein-/Ausgängen eines anderen E/A-Systems verschaltet.



Die folgende Vorgehensweise beschreibt die Verschaltung eines Ein-/Ausgangs von RSI mit dem Ein-/Ausgang eines Feldbusteilnehmers. Bei Verschaltung mit einem anderen E/A-System weicht die Vorgehensweise geringfügig ab. Weitere Informationen zur E/A-Verschaltung sind in der Dokumentation zu WorkVisual zu finden.

Voraussetzung

- Die Robotersteuerung ist aktiv gesetzt.
- Der Busaufbau in WorkVisual stimmt mit dem Aufbau des realen Busses überein.
- Die Feldbusgeräte sind konfiguriert.
- Fenster **EA-Verschaltung** ist geöffnet.

Vorgehensweise

1. In der linken Hälfte des Fensters **EA-Verschaltung** in der Registerkarte **RSI** den Bereich von RSI markieren, der verschaltet werden soll, z. B. **Digitale Eingänge**.
Die RSI-Signale werden im unteren Bereich des Fensters **EA-Verschaltung** angezeigt.
2. In der rechten Hälfte des Fensters **EA-Verschaltung** in der Registerkarte **Feldbusse** das gewünschte Gerät markieren.
Die Signale des Geräts werden im unteren Bereich des Fensters **EA-Verschaltung** angezeigt.
3. Das RSI-Signal per Drag&Drop auf den Eingang oder Ausgang des Geräts ziehen. (Oder auch umgekehrt den Eingang oder Ausgang des Geräts auf das RSI-Signal ziehen.)
Der Dialog **Signal Gruppierung** öffnet sich.
4. Den Dialog mit **Ja** bestätigen. Das Signal wird gruppiert und anschließend verschaltet.



Die Datentypen der Signale in WorkVisual haben keinen Einfluss auf die im RSI-Kontext verwendeten Datentypen. Sie dienen lediglich der Gruppierung. Die Datentypen der Feldbusteilnehmer müssen im RSI-Kontext zugewiesen werden.

Beispiel

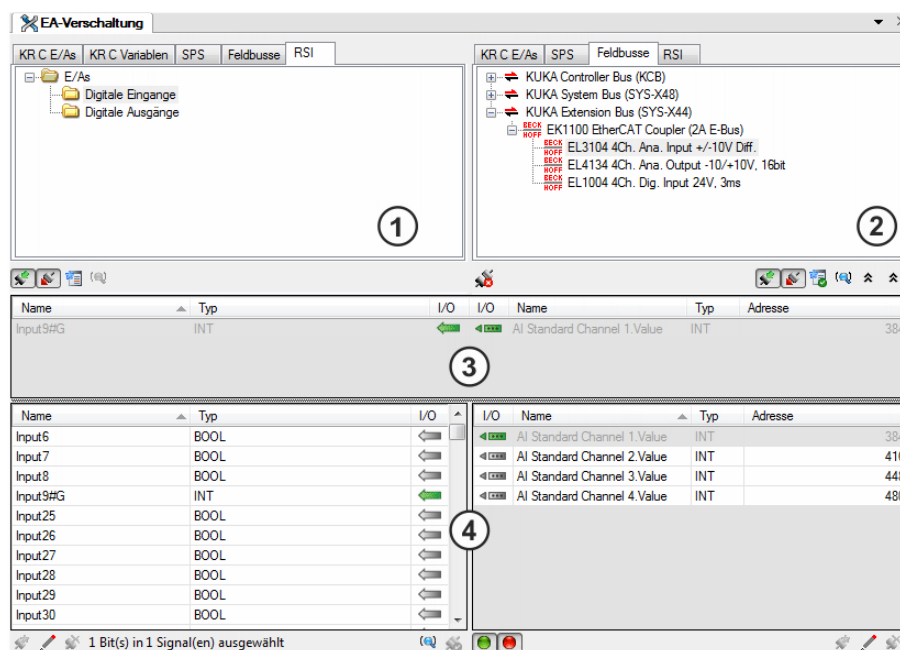


Abb. 6-1: Fenster EA-Verschaltung

Pos.	Beschreibung
1	Anzeige der Ein-/Ausgangstypen von RSI Hier wählt man den zu verschaltenden Bereich aus. Die Signale des markierten Bereichs werden in der unteren Anzeige angezeigt.
2	Anzeige der Feldbusgeräte Hier wählt man den zu verschaltenden Feldbusteilnehmer aus. Die Signale des markierten Feldbusteilnehmers werden in der unteren Anzeige angezeigt.
3	Anzeige der verschalteten Signale
4	Anzeige aller Signale Hier werden die Ein-/Ausgänge verschaltet.

In diesem Beispiel wurde der analoge Eingang eines EtherCAT-Teilnehmers mit einem digitalen RSI-Eingang verschaltet. Bei Verwendung dieses Signals im RSI-Kontext müssen dem **Input**-Objekt die folgenden Parameter zugewiesen werden.

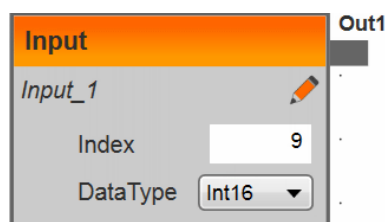


Abb. 6-2: Input-Objektparameter

Parameter	Beschreibung
Index	Nummer des ersten Bit des gruppierten Signals
DataType	Datentyp entsprechend der Bit-Breite des gruppierten Signals

6.2 Ethernet über KLI-Netzwerkconfiguration konfigurieren

Beschreibung

Für den Datenaustausch über Ethernet kann eine Netzwerkverbindung über das KLI der Robotersteuerung hergestellt werden. Hierzu wird ein eigenes Ethernet-Sensor-Netzwerk benötigt, das von anderen Netzwerken der Robotersteuerung unabhängig ist.



Weitere Informationen zu den verfügbaren Ethernet-Schnittstellen sind in der Montageanleitung der Robotersteuerung zu finden.

Voraussetzung

- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.

Vorgehensweise

1. Im Hauptmenü **Inbetriebnahme** > **Netzwerkconfiguration** wählen. Das Fenster **Netzwerkconfiguration** öffnet sich.
2. Die Registerkarte **KLI** wählen und auf **Erweitert...** drücken. Das Fenster für die erweiterte Netzwerkconfiguration öffnet sich.
3. Auf **Interface hinzufügen** drücken. Im Bereich **Konfigurierte Interfaces**: wird automatisch ein neuer Eintrag angelegt.
4. Den neu angelegten Eintrag markieren und im Feld **Interfacebezeichnung**: den Netzwerknamen eintragen, z. B. Ethernet-Sensor-Netzwerk.
5. Im Feld **Adresstyp**: den Typ **Gemischte IP-Adresse** auswählen.
Durch Auswahl des Typs **Gemischte IP-Adresse** werden die notwendigen Empfangstasks automatisch angelegt:
 - **Empfangstask**:
 - **Empfangsfilter**: Ziel-Subnetz
 - **Echtzeit Empfangstask**:
 - **Empfangsfilter**: UDP
6. In den Feldern darunter die IP-Adresse der Robotersteuerung und die Subnetzmaske eintragen.



Die IP-Adresse des Sensors und der Robotersteuerung müssen im gleichen Netzwerkbereich liegen.



Die IP-Adresse muss in einem freien Netzwerkbereich liegen. Interne Netzwerkbereiche der Robotersteuerung oder Netzwerkbereiche vorhandener KLI-/KONI-Netzwerk-Interfaces sind für die Konfiguration gesperrt. Die internen Netzwerkbereiche der Robotersteuerung sind in der Netzwerkconfiguration unter der Registerkarte **Interne Subnetze** konfiguriert.

7. Änderungen speichern und die Robotersteuerung mit den Einstellungen **Kaltstart** und **Dateien neu einlesen** neu starten.
8. Nach dem Neustart prüfen, ob im Diagnosemonitor ein Netzwerkinterface mit der gewählten Bezeichnung vorhanden ist.



Weitere Informationen zur KLI-Netzwerkconfiguration sind in der Bedien- und Programmieranleitung für Systemintegratoren der System Software zu finden.

6.3 Ethernet über KONI-Netzwerkconfiguration konfigurieren

Beschreibung

Für den Datenaustausch über Ethernet kann eine Netzwerkverbindung über das KONI der Robotersteuerung hergestellt werden. Hierzu wird ein eigenes Ethernet-Sensor-Netzwerk benötigt, das von anderen Netzwerken der Robotersteuerung unabhängig ist.



Weitere Informationen zu den verfügbaren Ethernet-Schnittstellen sind in der Montageanleitung der Robotersteuerung zu finden.

Voraussetzung

- KUKA.OptionalNetworkInterface ist installiert.
- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.

Vorgehensweise

1. Im Hauptmenü **Inbetriebnahme** > **Netzwerkconfiguration** wählen. Das Fenster **Netzwerkconfiguration** öffnet sich.
2. Die Registerkarte **KONI** wählen und auf **Erweitert...** drücken. Das Fenster für die erweiterte Netzwerkconfiguration öffnet sich.
3. Auf **Interface hinzufügen** drücken. Im Bereich **Konfigurierte Interfaces**: wird automatisch ein neuer Eintrag angelegt.
4. Den neu angelegten Eintrag markieren und im Feld **Interfacebezeichnung**: den Netzwerknamen eintragen, z. B. Ethernet-Sensor-Netzwerk.
5. Im Feld **Adresstyp**: den Typ **Gemischte IP-Adresse** auswählen.
Durch Auswahl des Typs **Gemischte IP-Adresse** werden die notwendigen Empfangstasks automatisch angelegt:
 - **Empfangstask**:
 - **Empfangsfilter: Ziel-Subnetz**
 - **Echtzeit Empfangstask**:
 - **Empfangsfilter: UDP**
6. In den Feldern darunter die IP-Adresse der Robotersteuerung und die Subnetzmaske eintragen.



Die IP-Adresse des Sensors und der Robotersteuerung müssen im gleichen Netzwerkbereich liegen.



Die IP-Adresse muss in einem freien Netzwerkbereich liegen. Interne Netzwerkbereiche der Robotersteuerung oder Netzwerkbereiche vorhandener KLI-/KONI-Netzwerk-Interfaces sind für die Konfiguration gesperrt. Die internen Netzwerkbereiche der Robotersteuerung sind in der Netzwerkconfiguration unter der Registerkarte **Interne Subnetze** konfiguriert.

7. Änderungen speichern und die Robotersteuerung mit den Einstellungen **Kaltstart** und **Dateien neu einlesen** neu starten.
8. Nach dem Neustart prüfen, ob im Diagnosemonitor ein Netzwerkinterface mit der gewählten Bezeichnung vorhanden ist.



Weitere Informationen zur KONI-Netzwerkconfiguration sind in der Dokumentation zu KUKA.OptionalNetworkInterface zu finden.

6.4 Globale Variablen ändern

Beschreibung

Die folgenden globalen Variablen können vom Benutzer geändert werden:

Variable	Beschreibung
RSIERRMSG	Quittiermeldung, wenn Fehler beim Ausführen von RSI-Befehlen auftreten TRUE = Fehler werden mit einer Quittiermeldung auf der smartHMI angezeigt. FALSE = Keine Quittiermeldung. Zur Fehlerbehandlung müssen die Rückgabewerte der RSI-Befehle im KRL-Programm ausgewertet werden. Default: TRUE
RSITECHIDX	Funktionsgenerator für RSI-Korrekturen im IPO-Modus Default-Wert: 1 Die maximale Anzahl der Funktionsgeneratoren ist in den Maschinendaten definiert (\$TECH_MAX).

Konfigurationsdatei

Verzeichnis	C:\KRC\ROBOTER\KRC\R1\TP\RSI
Datei	RSI_USER.DAT

```

DEFDAT RSI_USER PUBLIC
...
RSI global Variables:
GLOBAL BOOL RSIERRMSG=TRUE
...
; Flag for writing context information
GLOBAL INT RSITECHIDX=1
; Tech Channel used for RSI corrections

ENDDAT

```

Voraussetzung

- Benutzergruppe Experte

Vorgehensweise

1. Datei im Navigator markieren und **Öffnen** drücken. Die Datei wird im Editor angezeigt.
2. Variablen wie benötigt ändern.
3. Datei speichern und schließen.

6.5 RSI-Kontexte über Kontextverwaltung parametrieren

Beschreibung

RSI-Kontexte können über die Kontextverwaltung auf der Robotersteuerung parametrieren werden.

Ein RSI-Kontext, der bereits über KRL initialisiert wurde, wird durch die Parameteränderungen nicht verändert. Die Änderungen werden erst wirksam, wenn der RSI-Kontext neu initialisiert wird.

Voraussetzung

Zum Ändern von Parameterwerten und Kopieren, Löschen oder Umbenennen eines RSI-Kontexts:

- Benutzerrechte der Funktionsgruppe **Konfiguration der Techpakete**

Vorgehensweise

1. Im Hauptmenü **Konfiguration** > **RobotSensorInterface** > **Kontextverwaltung** wählen.

Beim erstmaligen Aufruf der Kontextverwaltung öffnet sich die Ansicht mit der Kontextübersicht.

(>>> *"Kontextübersicht" Seite 46*)

Danach wird immer die Ansicht geöffnet, mit der die Kontextverwaltung geschlossen wurde.

2. Um die Parameter eines RSI-Kontexts anzuzeigen, den gewünschten Kontext in der Kontextübersicht markieren und den Button mit dem Ordner-Symbol drücken.

(>>> *"Parameteransicht" Seite 47*)

Kontextübersicht

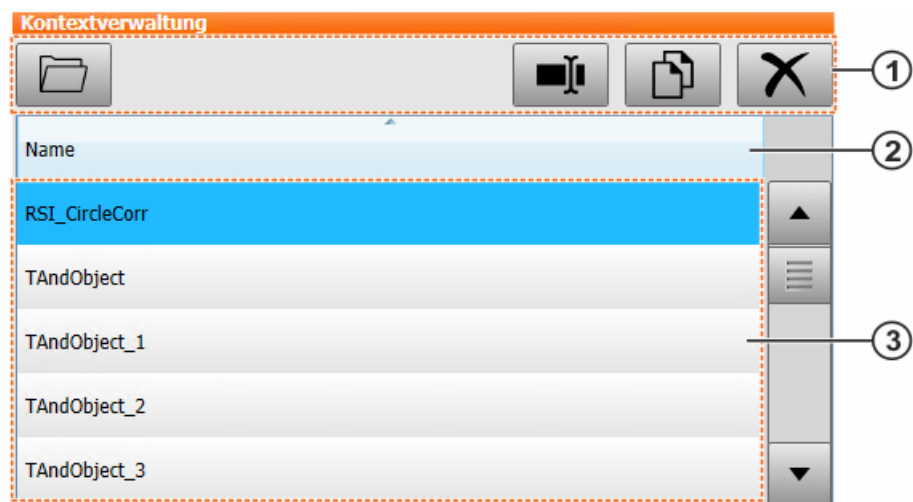


Abb. 6-3: Kontextübersicht

- 1 Buttonleiste
- 2 Schaltfläche: sortiert die RSI-Kontexte alphabetisch auf- oder absteigend
- 3 Liste vorhandener RSI-Kontexte

Buttons

Funktionen der Buttons in der Kontextübersicht:

Button	Beschreibung
	Öffnet die Ansicht mit den Parametern des ausgewählten RSI-Kontexts
	Macht den Namen des ausgewählten RSI-Kontexts editierbar (zum Umbenennen) Erforderliche Benutzerrechte: Funktionsgruppe Konfiguration der Techpakete
	Erstellt eine Kopie des ausgewählten RSI-Kontexts Erforderliche Benutzerrechte: Funktionsgruppe Konfiguration der Techpakete
	Löscht den ausgewählten RSI-Kontext Erforderliche Benutzerrechte: Funktionsgruppe Konfiguration der Techpakete

Parameteransicht

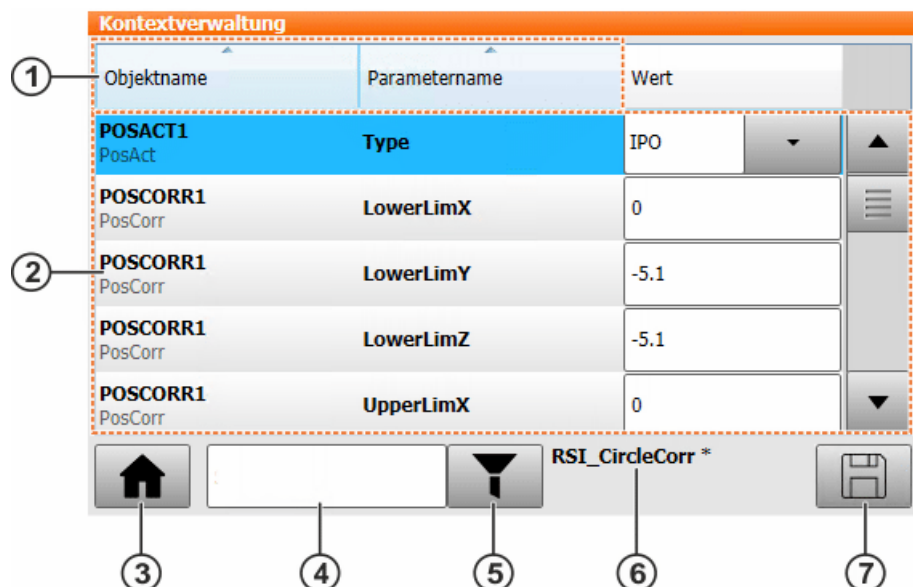


Abb. 6-4: Parameteransicht

Pos.	Beschreibung
1	Schaltflächen Sortieren die Parameter alphabetisch auf- oder absteigend nach dem Objektname oder nach dem Parameternamen.
2	Liste aller vorhandenen Parameter innerhalb des RSI-Kontexts
3	HOME-Button Schaltet zurück zur Kontextübersicht
4	Eingabefeld zur Parametersuche
5	Öffnet die Filtereinstellungen Als Filterkriterien für die Parametersuche können der Objektname, Objekttyp und/oder Parametername eingestellt werden.
6	Name des aktuell geöffneten RSI-Kontexts
7	Speichert die Parameteränderungen Erforderliche Benutzerrechte: Funktionsgruppe Konfiguration der Techpakete

7 Programmierung

7.1 Übersicht RSI-Befehle

Zur Programmierung der Signalverarbeitung stellt RobotSensorInterface Funktionen zur Verfügung. Jede dieser Funktionen, ausgenommen RSI_MOVECORR(), besitzt einen Rückgabewert. Der Rückgabewert kann im KRL-Programm abgefragt und ausgewertet werden.

Die Fehlercodes sind Konstanten, die in folgender Datenliste deklariert sind:

- KRC:\R1\TP\RSI\RSI.DAT

Um zu überprüfen, ob ein RSI-Befehl korrekt ausgeführt wurde, können die in den Funktionsbeschreibungen angegebenen Konstanten verwendet werden.

Funktion	Beschreibung
RSI_CREATE	(>>> 7.1.2 "RSI_CREATE: RSI-Container erstellen" Seite 50)
RSI_DELETE	(>>> 7.1.3 "RSI_DELETE: RSI-Container löschen" Seite 51)
RSI_ON	(>>> 7.1.4 "RSI_ON: Signalverarbeitung einschalten" Seite 51)
RSI_OFF	(>>> 7.1.5 "RSI_OFF: Signalverarbeitung ausschalten" Seite 53)
RSI_MOVECORR	(>>> 7.1.6 "RSI_MOVECORR: Sensorgeführte Bewegung einschalten" Seite 53)
RSI_GETPUBLICPAR	(>>> 7.1.7 "RSI_GETPUBLICPAR: RSI-Objektparameterwert lesen" Seite 54)
RSI_SETPUBLICPAR	(>>> 7.1.8 "RSI_SETPUBLICPAR: RSI-Objektparameter neuen Wert zuweisen" Seite 55)
RSI_RESET	(>>> 7.1.9 "RSI_RESET: Signalverarbeitung und alle RSI-Objekte löschen " Seite 56)
RSI_CHECKID	(>>> 7.1.10 "RSI_CHECKID: RSI-Container-ID prüfen" Seite 56)
RSI_ENABLE	(>>> 7.1.11 "RSI_ENABLE: RSI-Container aktivieren" Seite 57)
RSI_DISABLE	(>>> 7.1.12 "RSI_DISABLE: RSI-Container deaktivieren" Seite 57)

7.1.1 Zeichen und Schriftarten

In den Syntaxbeschreibungen werden folgende Zeichen und Schriftarten verwendet:

Syntax-Element	Darstellung
KRL-Code	• Command-Tag • Großschreibung Beispiele: GLOBAL; ANIN ON; OFFSET
Elemente, die durch programmspezifische Angaben ersetzt werden müssen	• Cite-Tag • Groß-/Kleinschreibung Beispiele: <i>Strecke</i> ; <i>Zeit</i> ; <i>Format</i>
Optionale Elemente	• In spitzen Klammern Beispiel: <STEP <i>Schrittweite</i> >
Elemente, die sich gegenseitig ausschließen	• Getrennt durch das Zeichen " " Beispiel: IN OUT

7.1.2 RSI_CREATE: RSI-Container erstellen

Beschreibung

Die Funktion RSI_CREATE() erstellt einen RSI-Container und lädt einen RSI-Kontext in den Container.

Der mit RSI_CREATE() erstellte Container ist standardmäßig aktiviert. Über die Container-ID kann auf den erstellten Container zugegriffen werden.

Syntax

Ret = RSI_CREATE(*Datei:IN*<, *ContainerID:OUT*><, *Status:IN*>)

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIFILENOTFOUND: Datei mit der Signalkonfiguration nicht gefunden • RSIINVFILE: Ungültige Datei, z. B. ungültiges Dateiformat oder Fehler in der Konfiguration • RSINOMEMORY: Kein freier RSI-Speicher mehr verfügbar • RSIINVOBJTYPE: Unbekanntes Objekt im RSI-Kontext • RSIEXTLIBNOTFOUND: Externe RSI-Objekt-Bibliothek nicht gefunden • RSINOTLINKED: RSI-Objekt mit fehlendem Eingangssignal • RSILNKCIRCLE: Fehler in der Signalverknüpfung
<i>Datei:IN</i>	Typ: CHAR-Feld Dateiname des RSI-Kontexts (ohne Dateierdung .rsix)

Element	Beschreibung
<i>ContainerID:OUT</i>	Typ: INT ID des RSI-Containers
<i>Status:IN</i>	Typ: BOOL TRUE = Aktiviert den RSI-Container FALSE = Deaktiviert den RSI-Container Default: TRUE

7.1.3 RSI_DELETE: RSI-Container löschen

Beschreibung

Die Funktion RSI_DELETE() löscht einen RSI-Container und die darin enthaltenen RSI-Objekte.

Syntax

Ret = RSI_DELETE (*ContainerID:IN*)

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: - RSIOK: Funktion erfolgreich ausgeführt - RSIINVOBJID: Ungültige Container-ID
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers

7.1.4 RSI_ON: Signalverarbeitung einschalten

Beschreibung

Mit der Funktion RSI_ON() wird die Signalverarbeitung in einer bestimmten Konfiguration gestartet.

Standardmäßig wird die Signalverarbeitung im Sensormodus #IPO_FAST und Korrekturmodus #ABSOLUTE ausgeführt. Das Korrekturkoordinatensystem wird in diesem Fall über den jeweiligen RSI-Objektparameter der vorhandenen RSI-Objekte vom Typ POSCORR direkt im RSI-Kontext festgelegt.

Wenn die Signalverarbeitung im Sensormodus #IPO ausgeführt wird, muss das Korrekturkoordinatensystem mit RSI_ON() definiert werden.

Wenn die Signalverarbeitung im Sensormodus #IPO_FAST und Korrekturmodus #RELATIVE ausgeführt wird und eine kartesische Positionskorrektur durch RSI-Objekte vom Typ POSCORR erfolgt, muss das Integrationskoordinatensystem mit RSI_ON() definiert werden.



Wenn die Signale im Sensormodus #IPO verarbeitet werden, kann der Bahnverlauf nur auf der Basis von LIN- und CIRC-Bewegungen korrigiert werden.

Syntax

Ret = RSI_ON(<Korrekturmodus:IN><, Sensormodus:IN, Korrektursystem:IN><, Integrationssystem:IN>)

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIALREADYON: Signalverarbeitung ist bereits eingeschaltet. • RSIINTEGRATIONSYS: Integrationskoordinatensystem im aktuellen Sensorsystem nicht möglich
<i>Korrekturmodus:IN</i>	Typ: ENUM Korrekturmodus: • #ABSOLUTE: Absolutkorrektur • #RELATIVE: Relativkorrektur Default: #ABSOLUTE
<i>Sensormodus:IN</i>	Typ: ENUM Modus der Signalverarbeitung: • #IPO_FAST: 4 ms • #IPO: 12 ms mit Filterung (\$FILTER) Default: #IPO_FAST
<i>Korrektursystem:IN</i>	Optionaler Parameter Typ: ENUM Korrekturkoordinatensystem für die Sensorkorrektur: • #BASE • #ROBROOT • #TCP • #TTS • #WORLD Default: #BASE Hinweis: nur relevant im Sensormodus #IPO

Element	Beschreibung
<i>Integrationssystem:IN</i>	Optionaler Parameter Typ: ENUM Integrationskoordinatensystem für die Sensorkorrektur: • #BASE • #ROBROOT • #TCP • #TTS • #WORLD Default: #WORLD Hinweis: nur relevant im Korrekturmodus #RELATIVE und Sensormodus #IPO_FAST



Weitere Informationen zum Einfluss des Integrationskoordinatensystems auf den Korrekturverlauf: (>>> [2.4.2.2 "Relativkorrektur" Seite 22](#))

7.1.5 RSI_OFF: Signalverarbeitung ausschalten

Beschreibung

Die Funktion RSI_OFF() schaltet die Signalverarbeitung aus.

Syntax

```
Ret = RSI_OFF()
```

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSINOTRUNNING: Keine laufende Signalverarbeitung

7.1.6 RSI_MOVECORR: Sensorgeführte Bewegung einschalten

Beschreibung

Die Funktion RSI_MOVECORR() schaltet die sensorgeführte Bewegung ein. Der Roboter wird rein korrekturgesteuert auf der Basis von Sensordaten verfahren, d. h. mit den Korrekturwerten der RSI-Objekte POSCORR oder AXISCORR.

- Eine sensorgeführte Bewegung kann mithilfe des RSI-Objekts STOP abgebrochen werden.
- Die Korrekturvorgaben können zusätzlich gefiltert werden.

Syntax

```
RSI_MOVECORR(<Stopp-Modus:IN>, <filterLength:IN>)
```

Erläuterung der Syntax

Element	Beschreibung
<i>Stopp-Modus:IN</i>	Optionaler Parameter Typ: ENUM Verhalten nach Abbruch der Bewegung: • #RSIBRAKE: Roboter fährt direkt vom Haltepunkt aus weiter. • #RSIBRAKERET: Roboter fährt zurück zu dem Punkt auf der Bahn, an dem das Stoppsignal eingetroffen ist. Default: #RSIBRAKE
<i>filterLength:IN</i>	Optionaler Parameter Typ: INT Filterwert für die Korrekturvorgaben • -1: Filterlänge der letzten Bewegung • 0: Kein Filter • >0: Filterlänge in ms Default: 0 Hinweis: Nur relevant im Sensormodus #IPO; im Conveyor-Betrieb wird immer die Filterlänge der letzten Bewegung verwendet.

7.1.7 RSI_GETPUBLICPAR: RSI-Objektparameterwert lesen

Beschreibung

Mit der Funktion `RSI_GETPUBLICPAR()` kann der Parameterwert eines RSI-Objekts gelesen werden.

Voraussetzung

- RSI-Objektparameter ist freigeschaltet
(>>> [8.4.2.1 "RSI-Objektparameter freischalten" Seite 75](#))

Syntax

```
Ret = RSI_GETPUBLICPAR(ContainerID:IN, Objekt:IN, Parameter:IN, Wert:OUT)
```

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIINVCONT: Ungültige Container-ID • RSIINPARAMID: Ungültiger RSI-Objekt- oder Parametername oder RSI-Objektparameter ist nicht freigeschaltet.
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers
<i>Objekt:IN</i>	Typ: CHAR-Feld Name des RSI-Objekts
<i>Parameter:IN</i>	Typ: CHAR-Feld Name des RSI-Objektparameters
<i>Wert:OUT</i>	Typ: REAL Wert des RSI-Objektparameters



Um über KRL auf einen Funktionsblockparameter zuzugreifen, muss ein spezieller Objektname verwendet werden.
(>>> [8.7 "Zugriff auf Funktionsblockparameter über KRL" Seite 91](#))

7.1.8 RSI_SETPUBLICPAR: RSI-Objektparameter neuen Wert zuweisen

Beschreibung

Mit der Funktion RSI_SETPUBLICPAR() kann dem Parameter eines RSI-Objekts ein neuer Wert zugewiesen werden.

Voraussetzung

- RSI-Objektparameter ist freigeschaltet
(>>> [8.4.2.1 "RSI-Objektparameter freischalten" Seite 75](#))

Syntax

```
Ret = RSI_SETPUBLICPAR(ContainerID:IN, Objekt:IN, Parameter:IN,  
Wert:IN)
```

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIINVCONT: Ungültige Container-ID • RSIINPARAMID: Ungültiger RSI-Objekt- oder Parametername oder RSI-Objektparameter ist nicht freigeschaltet. • RSIINPARAM: Ungültiger RSI-Objektparameterwert
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers
<i>Objekt:IN</i>	Typ: CHAR-Feld Name des RSI-Objekts
<i>Parameter:IN</i>	Typ: CHAR-Feld Name des RSI-Objektparameters
<i>Wert:IN</i>	Typ: REAL Neuer Wert des RSI-Objektparameters



Um über KRL auf einen Funktionsblockparameter zuzugreifen, muss ein spezieller Objektname verwendet werden.
 (>>> [8.7 "Zugriff auf Funktionsblockparameter über KRL" Seite 91](#))

7.1.9 RSI_RESET: Signalverarbeitung und alle RSI-Objekte löschen

Beschreibung

Die Funktion RSI_RESET() löscht die Signalverarbeitung und alle RSI-Objekte.

Syntax

```
Ret = RSI_RESET()
```

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt

7.1.10 RSI_CHECKID: RSI-Container-ID prüfen

Beschreibung

Mit der Funktion RSI_CHECKID() kann geprüft werden, ob eine gültige RSI-Container-ID verwendet wird.

Syntax

$$Ret = RSI_CHECKID (ContainerID:IN)$$
Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: BOOL Variable für den Rückgabewert der Funktion Mögliche Werte: • TRUE = RSI-Container zur ID vorhanden • FALSE = Kein RSI-Container zur ID vorhanden
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers

7.1.11 RSI_ENABLE: RSI-Container aktivieren**Beschreibung**

Die Funktion RSI_ENABLE() aktiviert einen deaktivierten RSI-Container.

Syntax

$$Ret = RSI_ENABLE (ContainerID:IN)$$
Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIINVOBJID: Ungültige Container-ID
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers

7.1.12 RSI_DISABLE: RSI-Container deaktivieren**Beschreibung**

Die Funktion RSI_DISABLE() deaktiviert einen RSI-Container. RSI-Objekte innerhalb eines deaktivierten RSI-Containers werden von der Signalverarbeitung nicht mehr ausgeführt.

Syntax

$$Ret = RSI_DISABLE (ContainerID:IN)$$

Erläuterung der Syntax

Element	Beschreibung
<i>Ret</i>	Typ: INT Variable für den Rückgabewert der Funktion Mögliche Werte: • RSIOK: Funktion erfolgreich ausgeführt • RSIINVOBJID: Ungültige Container-ID
<i>ContainerID:IN</i>	Typ: INT ID des RSI-Containers

7.2 Verhalten der Befehle RSI_DISABLE und RSI_ENABLE

Beschreibung

Wenn ein RSI-Container mit RSI_DISABLE() deaktiviert wird, werden die RSI-Objekte innerhalb dieses RSI-Containers von der Signalverarbeitung nicht mehr ausgeführt. Die zu diesem Zeitpunkt vorhandenen Signalwerte der RSI-Objekte innerhalb des RSI-Containers bleiben bestehen.

Wenn der RSI-Container mit RSI_ENABLE() wieder aktiviert wird, werden die RSI-Objekte durch die Signalverarbeitung wieder ausgeführt.



Signale (z. B. von einem Sensor) können sich ändern, während ein RSI-Container deaktiviert ist. Beim Fortsetzen der Signalverarbeitung eines RSI-Containers durch RSI_ENABLE() kann es daher zu einem Sprung bei der Verarbeitung dieser Signale kommen. Dies kann zu einem unerwünschten Verhalten führen und muss durch geeignete Programmierung des RSI-Kontexts durch den Anwender behandelt werden.

Beispiel

Pausieren des RSI-Kontexts durch RSI_DISABLE() beim Generieren eines Sinussignals durch ein SOURCE-Objekt.



Abb. 7-1: RSI-Kontext pausiert

```
DEF RSI_DISABLEENABLE()
DECL INT ret,cont
ret=RSI_CREATE("Signals", cont)
ret=RSI_ON()
wait sec 7
ret=RSI_DISABLE(cont)
```

```
wait sec 1
ret=RSI_ENABLE(cont)
wait sec 10
ret=RSI_OFF()
END
```

7.3 Verhalten der Befehle RSI_ON und RSI_OFF

Beschreibung

Mit RSI_OFF() und RSI_ON() werden alle RSI-Kontexte neu gestartet. Mit RSI_OFF() und RSI_ON() kann kein einzelner RSI-Kontext angesprochen werden, sondern nur alle durch RSI_CREATE() erzeugten RSI-Kontexte gemeinsam.

Beispiel

Zurücksetzen des RSI-Kontexts durch RSI_OFF() beim Generieren eines Sinussignals durch ein SOURCE-Objekt.



Abb. 7-2: RSI-Kontext neu gestartet

```
DEF RSI_ONOFF()
DECL INT ret,cont
ret=RSI_CREATE("Signals",cont)
ret=RSI_ON()
wait sec 7
ret=RSI_OFF()
wait sec 1
ret=RSI_ON()
Wait sec 10
ret=RSI_OFF()
END
```

7.4 Signalverarbeitung programmieren

Schritt	Beschreibung
1	RSI-Kontext mit RSIVisual konfigurieren.
2	RSI-Kontext auf die Robotersteuerung übertragen. Zielverzeichnis auf der Robotersteuerung: C:\KRC\ROBOTER\Config\User\Common\SensorInterface

Schritt	Beschreibung
3	RSI-Kontext in KRL-Programm einbinden. (>>> 7.4.1 "RSI-Kontext in KRL-Programm einbinden" Seite 60)
4	RSI-Objektparameter nachträglich in KRL ändern, wenn benötigt. (>>> 7.4.2 "Zugriff auf RSI-Objektparameter über KRL" Seite 60)

7.4.1 RSI-Kontext in KRL-Programm einbinden

Die Signalverarbeitung muss im KRL-Programm initialisiert, eingeschaltet und wieder ausgeschaltet werden.

Aufbau eines Signalverarbeitungsprogramms:

```

1 DEF signal_processing()
2
3 DECL INT ret
4
5
6 ret=RSI_CREATE("myContext")
7 ret=RSI_ON()
8
9
10 movements
11
12
13
14
15 ret=RSI_OFF()
16
17
18
19
20 END

```

Zeile	Beschreibung
3	Deklaration der KRL-Variablen (hier nur Variable "ret" für den Rückgabewert)
6	RSI_CREATE() initialisiert die Signalverarbeitung. Der RSI-Kontext wird in einen RSI-Container geladen.
7	RSI_ON() schaltet die Signalverarbeitung ein.
10	Bewegungsanweisungen oder RSI_MOVECORR() für eine sensorgeführte Bewegung
15	RSI_OFF() schaltet die Signalverarbeitung aus.

7.4.2 Zugriff auf RSI-Objektparameter über KRL

Beschreibung

Parameter von RSI-Objekten können nachträglich über folgende Funktionen im KRL-Programm gelesen oder eingestellt werden:

- RSI_GETPUBLICPAR()
Funktion zum Lesen des Objektparameterwerts
- RSI_SETPUBLICPAR()
Funktion zum Einstellen des Objektparameterwerts

Voraussetzung

- RSI-Objektparameter ist freigeschaltet.
(>>> 8.4.2.1 "RSI-Objektparameter freischalten" Seite 75)
- KRL-Programm wurde gestartet.
- RSI-Kontext wurde mit RSI_CREATE() geladen.

Beispiel

(>>> 9.4 "Beispiel RSI_CircleCorr: sensorgeführte Kreisbewegung" Seite 99)

7.5 XML-Datei für Ethernet-Verbindung konfigurieren

Beschreibung

RobotSensorInterface verwendet das XML-Format, um Daten über Ethernet auszutauschen. Für eine Ethernet-Verbindung muss im Verzeichnis C:\KRC\ROBOTER\Config\User\Common\SensorInterface eine Konfigurationsdatei definiert sein.

Der Name der Konfigurationsdatei muss später im Ethernet-Objekt des RSI-Kontexts angegeben werden und beim Initialisieren der Signalverarbeitung im KRL-Programm eingelesen werden.

Übersicht

Aufbau der Konfigurationsdatei:

```
<ROOT>
  <CONFIG></CONFIG>
  <SEND>
    <ELEMENTS></ELEMENTS>
  </SEND>
  <RECEIVE>
    <ELEMENTS></ELEMENTS>
  </RECEIVE>
</ROOT>
```

Abschnitt	Beschreibung
<CONFIG ... </CONFIG>	Konfiguration der Verbindungsparameter zwischen Sensorsystem und Schnittstelle (>>> 7.5.1 "XML-Struktur für Verbindungseigenschaften" Seite 62)
<SEND> ... </SEND>	Konfiguration der Sendestruktur (>>> 7.5.2 "XML-Struktur für den Datenversand" Seite 62)
<RECEIVE> ... </RECEIVE>	Konfiguration der Empfangsstruktur (>>> 7.5.3 "XML-Struktur für den Datenempfang" Seite 64)

WorkVisual

In WorkVisual steht eine Vorlage zur Konfiguration der Ethernet-Verbindung zur Verfügung.

(>>> 7.5.7 "Ethernet-Verbindung in WorkVisual konfigurieren" Seite 69)

7.5.1 XML-Struktur für Verbindungseigenschaften

Beschreibung

Elemente der XML-Struktur:

Element	Beschreibung
IP_NUMBER	IP-Adresse des Sensorsystems
PORT	Port-Nummer des Sensorsystems • 1 ... 65 534
SENTYPE	Kennung des Sensorsystems (Name frei wählbar) Die Robotersteuerung überprüft die Kennung bei jedem Datenpaket, das sie empfängt.
ONLYSEND	Richtung des Datenaustauschs • TRUE = Die Robotersteuerung sendet Daten und erwartet keine Daten vom Sensorsystem zurück. • FALSE = Die Robotersteuerung sendet und empfängt Daten. Default: FALSE

Beispiel

```
<CONFIG>
<IP_NUMBER>172.1.10.5</IP_NUMBER>
<PORT>49152</PORT>
<SENTYPE>ImFree</SENTYPE>
<ONLYSEND>FALSE</ONLYSEND>
</CONFIG>
```

7.5.2 XML-Struktur für den Datenversand

Beschreibung

Hier werden die Signale aus dem RSI Kontext definiert, die an den Eingängen des ETHERNET Objekts eintreffen und an das Sensorsystem gesendet werden.

Das ETHERNET Objekt verfügt außerdem über eine Lesefunktion, mit der Systeminformationen der Robotersteuerung eingelesen und an das Sensorsystem gesendet werden können. Die Lesefunktion wird über Schlüsselwörter aktiviert.

RobotSensorInterface erstellt aus der konfigurierten XML-Struktur automatisch das XML-Dokument, das die Robotersteuerung sendet.

Signaleingänge

Definition der Signaleingänge in der XML-Struktur:

Attribut	Beschreibung
TAG	Name des Elements Hier wird die XML-Struktur für den Datenversand definiert (XML-Schema). (>>> 7.5.4 "Konfiguration nach XML-Schema" Seite 66)
TYPE	Datentyp des Elements • BOOL • DOUBLE • LONG
INDX	Nummer des ETHERNET Objekteingangs • 1 ... 64 Hinweis: Die Objekteingänge müssen fortlaufend nummeriert sein.

Beispiel Signaleingänge

- Konfigurierte XML-Struktur für den Datenversand:

```
<SEND>
<ELEMENTS>
  <ELEMENT TAG="Out.01" TYPE="BOOL" INDX="1" />
  <ELEMENT TAG="Out.02" TYPE="BOOL" INDX="2" />
  <ELEMENT TAG="Out.03" TYPE="BOOL" INDX="3" />
  <ELEMENT TAG="Out.04" TYPE="BOOL" INDX="4" />
  <ELEMENT TAG="Out.05" TYPE="BOOL" INDX="5" />
  <ELEMENT TAG="FTC.Fx" TYPE="DOUBLE" INDX="6" />
  <ELEMENT TAG="FTC.Fy" TYPE="DOUBLE" INDX="7" />
  <ELEMENT TAG="FTC.Fz" TYPE="DOUBLE" INDX="8" />
  <ELEMENT TAG="FTC.Mx" TYPE="DOUBLE" INDX="9" />
  <ELEMENT TAG="FTC.My" TYPE="DOUBLE" INDX="10" />
  <ELEMENT TAG="FTC.Mz" TYPE="DOUBLE" INDX="11" />
  <ELEMENT TAG="Override" TYPE="LONG" INDX="12" />
</ELEMENTS>
</SEND>
```

- Von der Robotersteuerung gesendetes XML-Dokument:

```
<Rob TYPE="KUKA">
  <Out 01="0" 02="1" 03="1" 04="0" 05="0" />
  <FTC Fx="1.234" Fy="54.75" Fz="345.7
    Mx="2346.6" My="12.0" Mz="3546" />
  <Override>90</Override>
  <IPOC>123645634563</IPOC>
</Rob>
```



Das Schlüsselwort IPOC sendet einen Zeitstempel und wird automatisch erstellt.

Lesefunktion

Aktivieren der Lesefunktion in der XML-Struktur:

Attribut	Beschreibung
TAG	Name des Elements Ein Schlüsselwort gibt an, welche Systeminformation eingelesen wird. (>>> 7.5.5 "Schlüsselwörter zum Daten lesen" Seite 67)
TYPE	Datentyp des Elements • DOUBLE • LONG
INDX	Schlüsselwort zum Einlesen der Systeminformation • INTERNAL

Beispiel Lesefunktion

(>>> "Beispiel Lesefunktion" Seite 67)

7.5.3 XML-Struktur für den Datenempfang

Beschreibung

Hier werden die Signale definiert, die an den Ausgängen des ETHERNET-Objekts vom Sensorsystem empfangen und im RSI-Kontext an die Robotersteuerung weitergeleitet werden.

Das ETHERNET-Objekt verfügt außerdem über eine Schreibfunktion, mit der Informationen in die Robotersteuerung geschrieben oder Meldungen auf der smartHMI ausgegeben werden können. Die Schreibfunktion wird über Schlüsselwörter aktiviert.

RobotSensorInterface erstellt aus der konfigurierten XML-Struktur automatisch das XML-Dokument, das die Robotersteuerung erwartet.

Signalausgänge

Definition der Signalausgänge in der XML-Struktur:

Attribut	Beschreibung
TAG	Name des Elements Hier wird die XML-Struktur für den Datenempfang definiert (XML-Schema). (>>> 7.5.4 "Konfiguration nach XML-Schema" Seite 66)
TYPE	Datentyp des Elements • BOOL • DOUBLE • LONG
INDX	Nummer des ETHERNET-Objektausgangs • 1 ... 64 Hinweis: Die Objektausgänge müssen fortlaufend nummeriert sein.

Attribut	Beschreibung
HOLDON	Verhalten des Objektausgangs bei zu spät eintreffenden Datenpaketen • 0: Der Ausgang wird zurückgesetzt. • 1: Der letzte gültige Wert, der eingetroffen ist, bleibt am Ausgang stehen.

Beispiel Signalausgänge

- Konfigurierte XML-Struktur für den Datenempfang:

```
<RECEIVE>
<ELEMENTS>
  <ELEMENT TAG="RKorr.X" TYPE="DOUBLE" INDX="1" HOLDON="1" />
  <ELEMENT TAG="RKorr.Y" TYPE="DOUBLE" INDX="2" HOLDON="1" />
  <ELEMENT TAG="RKorr.Z" TYPE="DOUBLE" INDX="3" HOLDON="1" />
  <ELEMENT TAG="RKorr.A" TYPE="DOUBLE" INDX="4" HOLDON="1" />
  <ELEMENT TAG="RKorr.B" TYPE="DOUBLE" INDX="5" HOLDON="1" />
  <ELEMENT TAG="RKorr.C" TYPE="DOUBLE" INDX="6" HOLDON="1" />
  <ELEMENT TAG="AK.A1" TYPE="DOUBLE" INDX="7" HOLDON="0" />
  <ELEMENT TAG="AK.A2" TYPE="DOUBLE" INDX="8" HOLDON="0" />
  <ELEMENT TAG="AK.A3" TYPE="DOUBLE" INDX="9" HOLDON="0" />
  <ELEMENT TAG="AK.A4" TYPE="DOUBLE" INDX="10" HOLDON="0" />
  <ELEMENT TAG="AK.A5" TYPE="DOUBLE" INDX="11" HOLDON="0" />
  <ELEMENT TAG="AK.A6" TYPE="DOUBLE" INDX="12" HOLDON="0" />
  <ELEMENT TAG="EK.E1" TYPE="DOUBLE" INDX="13" HOLDON="0" />
  <ELEMENT TAG="EK.E2" TYPE="DOUBLE" INDX="14" HOLDON="0" />
  <ELEMENT TAG="EK.E3" TYPE="DOUBLE" INDX="15" HOLDON="0" />
  <ELEMENT TAG="EK.E4" TYPE="DOUBLE" INDX="16" HOLDON="0" />
  <ELEMENT TAG="EK.E5" TYPE="DOUBLE" INDX="17" HOLDON="0" />
  <ELEMENT TAG="EK.E6" TYPE="DOUBLE" INDX="18" HOLDON="0" />
  <ELEMENT TAG="DiO" TYPE="LONG" INDX="19" HOLDON="1" />
</ELEMENTS>
</RECEIVE>
```

- Vom Sensorsystem empfangenes XML-Dokument:

```
<Sen Type="ImFree">
  <RKorr X="4" Y="7" Z="32" A="6" B="" C="6" />
  <AK A1="2" A2="54" A3="35" A4="76" A5="567" A6="785" />
  <EK E1="67" E2="67" E3="678" E4="3" E5="3" E6="7" />
  <DiO>123</DiO>
  <IPOC>123645634563</IPOC>
</Sen>
```



Der Zeitstempel mit dem Schlüsselwort IPOC wird überprüft. Das Datenpaket ist nur dann gültig, wenn der Zeitstempel mit dem zuvor gesendeten Zeitstempel übereinstimmt.

Schreibfunktion

Aktivieren der Schreibfunktion in der XML-Struktur:

Attribut	Beschreibung
TAG	Name des Elements Ein Schlüsselwort gibt an, welche Information in die Robotersteuerung geschrieben wird oder ob eine Meldung auf der smartHMI ausgegeben wird. (>>> 7.5.6 "Schlüsselwörter zum Daten schreiben" Seite 68)
TYPE	Datentyp des Elements • DOUBLE • STRING
INDX	Schlüsselwort zum Schreiben der Information • INTERNAL
HOLDON	Verhalten des Objektausgangs bei zu spät eintreffenden Datenpaketen • 0: Der Ausgang wird zurückgesetzt. • 1: Der letzte gültige Wert, der eingetroffen ist, bleibt am Ausgang stehen.

Beispiel Schreibfunktion

(>>> "Beispiel Schreibfunktion" Seite 68)

7.5.4 Konfiguration nach XML-Schema

Beschreibung

RobotSensorInterface erstellt aus der konfigurierten XML-Struktur automatisch die XML-Dokumente für den Datenaustausch.

Nach XML-Schema sind folgende Schreibweisen zu unterscheiden:

- Element-Schreibweise
- Attribut-Schreibweise

Element-Schreibweise

- TAGs in der konfigurierten XML-Struktur:

```
...
<ELEMENTS>
  <ELEMENT TAG="Out1" ... />
  <ELEMENT TAG="Out2" ... />
  <ELEMENT TAG="Out3" ... />
</ELEMENTS>
...
```

- TAGs im erstellten XML-Dokument:

```
...
<Out1>...</Out1>
<Out2>...</Out2>
<Out3>...</Out3>
...
```

Attribut-Schreibweise

- TAGs in der konfigurierten XML-Struktur:

```
...
<ELEMENTS>
  <ELEMENT TAG="Out.01" ... />
  <ELEMENT TAG="Out.02" ... />
  <ELEMENT TAG="Out.03" ... />
</ELEMENTS>
...
```

- TAG mit Attributen im erstellten XML-Dokument:

```
...
<Out 01="..." 02="..." 03="..." />
...
```

7.5.5 Schlüsselwörter zum Daten lesen

Schlüsselwörter sind Buchstabenfolgen mit einer fest zugeordneten Bedeutung. Sie dürfen in der XML-Struktur nicht anders als in dieser Bedeutung verwendet werden. Die Groß- und Kleinschreibung ist nicht relevant. Ein Schlüsselwort gilt in jeder Schreibweise als Schlüsselwort.

Schlüsselwörter

Folgende Informationen der Robotersteuerung können über Schlüsselwörter im Attribut TAG eingelesen werden:

Information	Schlüsselwort	Typ
Kartesische Istposition	DEF_RIst	DOUBLE
Kartesische Sollposition	DEF_RSol	DOUBLE
Achsspezifische Istposition der Roboterachsen A1 ... A6	DEF_AIPos	DOUBLE
Achsspezifische Sollposition der Roboterachsen A1 ... A6	DEF_ASPos	DOUBLE
Achsspezifische Istposition der Zusatzachsen E1 ... E6	DEF_EIPos	DOUBLE
Achsspezifische Sollposition der Zusatzachsen E1 ... E6	DEF_ESPos	DOUBLE
Motorströme der Roboterachsen A1 ... A6	DEF_MACur	DOUBLE
Motorströme der Zusatzachsen E1 ... E6	DEF_MECur	DOUBLE
Anzahl der zu spät eingetroffenen Datenpakete	DEF_Delay	LONG
Technologieparameter im Hauptlauf (Funktionsgeneratoren 1 ... 6)	DEF_Tech.C1 ... DEF_Tech.C6	DOUBLE
Technologieparameter im Vorlauf (Funktionsgeneratoren 1 ... 6)	DEF_Tech.T1 ... DEF_Tech.T6	DOUBLE

Beispiel Lesefunktion

- Konfigurierte XML-Struktur für den Datenversand:

```
<SEND>
<ELEMENTS>
```

```

<ELEMENT TAG="DEF_Rist" TYPE="DOUBLE" INDX="INTERNAL" />
<ELEMENT TAG="DEF_AIPos" TYPE="DOUBLE" INDX="INTERNAL" />
<ELEMENT TAG="DEF_MACur" TYPE="DOUBLE" INDX="INTERNAL" />
<ELEMENT TAG="DEF_Delay" TYPE="LONG" INDX="INTERNAL" />
<ELEMENT TAG="DEF_Tech.C1" TYPE="DOUBLE" INDX="INTERNAL" />
</ELEMENTS>
</SEND>

```

- Von der Robotersteuerung gesendetes XML-Dokument:

```

<Rob TYPE="KUKA">
  <Rist X="0.0" Y="0.0" Z="0.0" A="0.0" B="0.0" C="0.0" />
  <AIPos A1="0.0" A2="0.0" A3="0.0" A4="0.0" A5="0.0" A6="0.0" />
  <MACur A1="1.0" A2="1.0" A3="1.0" A4="1.0" A5="1.0" A6="1.0" />
  <Delay D="" />
  <Tech T11="0.0" T12="0.0" T13="0.0" T14="0.0" T15="0.0" T16="0.0" T17="0.0"
  T18="0.0" T19="0.0" T110="0.0" />
  <IPOC>123645634563</IPOC>
</Rob>

```



Das Schlüsselwort IPOC sendet einen Zeitstempel und wird automatisch erstellt.

7.5.6 Schlüsselwörter zum Daten schreiben

Schlüsselwörter sind Buchstabenfolgen mit einer fest zugeordneten Bedeutung. Sie dürfen in der XML-Struktur nicht anders als in dieser Bedeutung verwendet werden. Die Groß- und Kleinschreibung ist nicht relevant. Ein Schlüsselwort gilt in jeder Schreibweise als Schlüsselwort.

Schlüsselwörter

Folgende Informationen können über Schlüsselwörter im Attribut TAG in die Robotersteuerung geschrieben werden:

Information	Schlüsselwort	Typ
Technologieparameter im Hauptlauf (Funktionsgeneratoren 1 ... 6)	DEF_Tech.C1 ... DEF_Tech.C6	DOUBLE
Technologieparameter im Vorlauf (Funktionsgeneratoren 1 ... 6)	DEF_Tech.T1 ... DEF_Tech.T6	DOUBLE

Schlüsselwort im Attribut TAG, um Meldungen auf der smartHMI auszugeben:

Information	Schlüsselwort	Typ
Hinweis- oder Fehlermeldung	DEF_EStr	STRING

Meldungstypen

Im XML-Dokument, das vom Sensorsystem beschrieben und gesendet wird, können folgende Meldungstypen auftreten:

- **<EStr> xxx </EStr>**: Hinweismeldung
- **<EStr>Error: xxx </EStr>**: Quittiermeldung (Stopp des Roboters)
- **<EStr/>**: Keine Meldung bei einem leeren Tag

Beispiel Schreibfunktion

- Konfigurierte XML-Struktur für den Datenempfang:

```
<RECEIVE>
<ELEMENTS>
<ELEMENT TAG="DEF_EStr" TYPE="STRING" INDX="INTERNAL" />
  <ELEMENT TAG="DEF_Tech.T2" TYPE="DOUBLE" INDX="INTERNAL" HOLDON="0" />
</ELEMENTS>
</RECEIVE>
```

- Vom Robotersystem empfangenes XML-Dokument:

```
<Sen Type="ImFree">
  <EStr>Message!</EStr>
  <Tech T21="0.0" T22="0.0" T23="0.0" T24="0.0" T25="0.0" T26="0.0" T27="0.0"
  T28="0.0" T29="0.0" T210="0.0" />
  <IPOC>123645634563</IPOC>
</Sen>
```



Der Zeitstempel mit dem Schlüsselwort IPOC wird überprüft. Das Datenpaket ist nur dann gültig, wenn der Zeitstempel mit dem zuvor gesendeten Zeitstempel übereinstimmt.

7.5.7 Ethernet-Verbindung in WorkVisual konfigurieren

Beschreibung

Die Konfigurationsdatei für die Ethernet-Verbindung kann in WorkVisual erstellt, bearbeitet und auf die Robotersteuerung übertragen werden.

Zielverzeichnis für die Konfigurationsdatei:

- ...\\Config\\User\\Common\\SensorInterface

Voraussetzung

- RobotSensorInterface ist in WorkVisual installiert.
- Das aktive Projekt wurde von der Robotersteuerung geladen und das Projekt enthält den Katalog **KRL Templates**.

ODER:

Zum online Arbeiten mit dem KRC-Explorer: Über den Arbeitsbereich **Programmierung und Diagnose** wurde eine Verbindung mit der Robotersteuerung hergestellt.

Zum Übertragen des Projekts auf die Robotersteuerung:

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Vorgehensweise

1. Wenn das Projekt geladen wurde, im Fenster **Projektstruktur** die Registerkarte **Dateien** wählen.
2. Auf den Ordner **SensorInterface** rechtsklicken und im Kontextmenü **Hinzufügen...** wählen.
3. Die Vorlage **RSI Ethernet Configuration** markieren und hinzufügen.
4. Einen Dateinamen eingeben und mit **OK** bestätigen.
5. Die neu erstellte Konfigurationsdatei öffnen und die Ethernet-Verbindung wie benötigt konfigurieren.
6. Konfigurationsdatei speichern.
7. Das Projekt zurück auf die Robotersteuerung übertragen.

ODER:

Über den Arbeitsbereich **Programmierung und Diagnose** die Daten auf der Robotersteuerung aktualisieren.

8 Expertenprogrammierung mit RSIVisual

8.1 Übersicht RSIVisual

RSIVisual ist ein grafischer Editor in WorkVisual mit folgenden Funktionalitäten:

- RSI-Kontexte innerhalb eines WorkVisual-Projekts erstellen und bearbeiten
- RSI-Kontexte innerhalb eines WorkVisual-Repositories im Arbeitsbereich **Programmierung und Diagnose** erstellen und bearbeiten

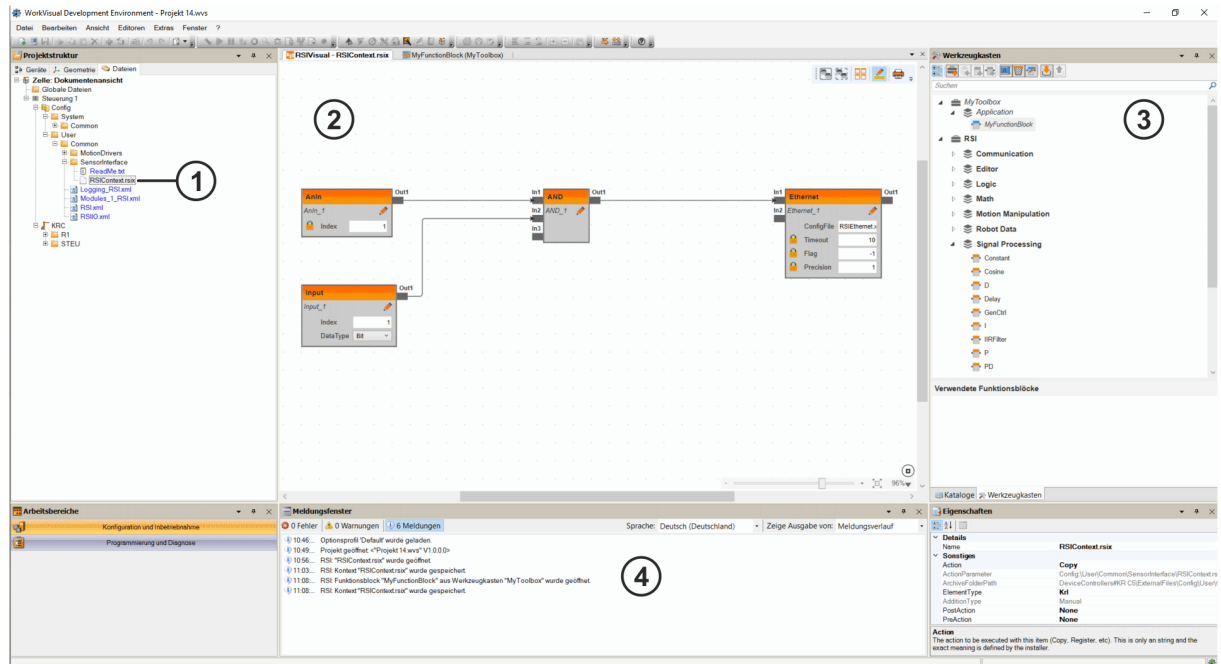


Abb. 8-1: Übersicht RSIVisual

Pos.	Beschreibung
1	RSI-Kontext im Projektverzeichnis (>>> 8.2 "RSI-Kontext anlegen" Seite 71)
2	RSI-Kontext im Kontext-Editor (>>> 8.4 "RSI-Kontext im Kontext-Editor öffnen" Seite 73)
3	Fenster Werkzeugkasten Wenn der Kontext-Editor geöffnet ist, werden die verfügbaren RSI-Objekte im Werkzeugkasten von WorkVisual angezeigt. (>>> 8.5 "Fenster Werkzeugkasten einblenden" Seite 79)
4	Meldungsfenster Fehler beim Erstellen eines RSI-Kontexts werden angezeigt.

8.2 RSI-Kontext anlegen

Beschreibung

RSI-Kontexte müssen in folgendem Zielverzeichnis angelegt werden:

- ...\\Config\\User\\Common\\SensorInterface

Voraussetzung

- RobotSensorInterface ist in WorkVisual installiert.
- Das aktive Projekt wurde von der Robotersteuerung geladen und das Projekt enthält den Katalog **KRL Templates**.

ODER:

Zum online Arbeiten mit dem KRC-Explorer: Über den Arbeitsbereich **Programmierung und Diagnose** wurde eine Verbindung mit der Robotersteuerung hergestellt.

Vorgehensweise

1. Wenn das Projekt geladen wurde, im Fenster **Projektstruktur** die Registerkarte **Dateien** wählen.
2. Auf den Ordner **SensorInterface** rechtsklicken und im Kontextmenü **Hinzufügen...** wählen.
3. Die Vorlage **RSIContext** markieren und hinzufügen.
4. Einen Dateinamen eingeben und mit **OK** bestätigen.

8.3 RSI-Kontextdatei importieren und konvertieren

Beschreibung

RSI-Kontextdateien können in WorkVisual importiert und gleichzeitig konvertiert werden.

- RSI-Kontextdateien, die im alten Dateiformat *.rsi.xml vorliegen, werden in das aktuelle Dateiformat *.rsix konvertiert.
- Wenn in dem Verzeichnis, aus dem eine *.rsi.xml-Datei importiert wird, eine passende *.rsi.diagram-Datei liegt, wird die RSI-Objektanordnung mitkonvertiert. Liegt eine passende *.rsi-Datei in dem Verzeichnis, werden auch die Kommentare importiert.
- Die importierten Dateien werden automatisch in das vorgesehene Zielverzeichnis abgelegt:
 - ...\\Config\\User\\Common\\SensorInterface

Voraussetzung

- RobotSensorInterface ist in WorkVisual installiert.
- Ein Projekt ist geöffnet.
- RobotSensorInterface ist in dem Projekt installiert.

Vorgehensweise

1. Menüfolge **Datei > Import / Export** wählen. Ein Assistent zum Importieren öffnet sich.
2. Den Eintrag **RSI-Kontextdatei importieren** wählen.
3. Auf **Weiter >** klicken und über den Button **Auswählen ...** zu dem Verzeichnis navigieren, in dem die zu importierenden Dateien abgelegt sind.
4. Die zu importierenden Dateien markieren und auf **Öffnen** klicken, um sie in den Assistenten zu laden.
5. Auf **Weiter >** klicken und die gewünschte Zielsteuerung auswählen.
6. Die Option **Vorhandene Datei überschreiben** ist standardmäßig deaktiviert (kein Häkchen). Wenn im Projektverzeichnis bereits eine RSI-Kontextdatei gleichen Namens vorhanden ist, verhindert dies das (unbeabsichtigte) Überschreiben der vorhandenen Datei.

Das Häkchen nur setzen, wenn explizit gewünscht ist, die vorhandene RSI-Kontextdatei zu überschreiben.

- 7. Auf **Fertigstellen** klicken. Die Dateien werden importiert und konvertiert. Es wird angezeigt, ob der Import erfolgreich war. Den Assistenten schließen.

8.4 RSI-Kontext im Kontext-Editor öffnen

Beschreibung

RSI-Kontexte können im Kontext-Editor von RSIVisual bearbeitet werden.

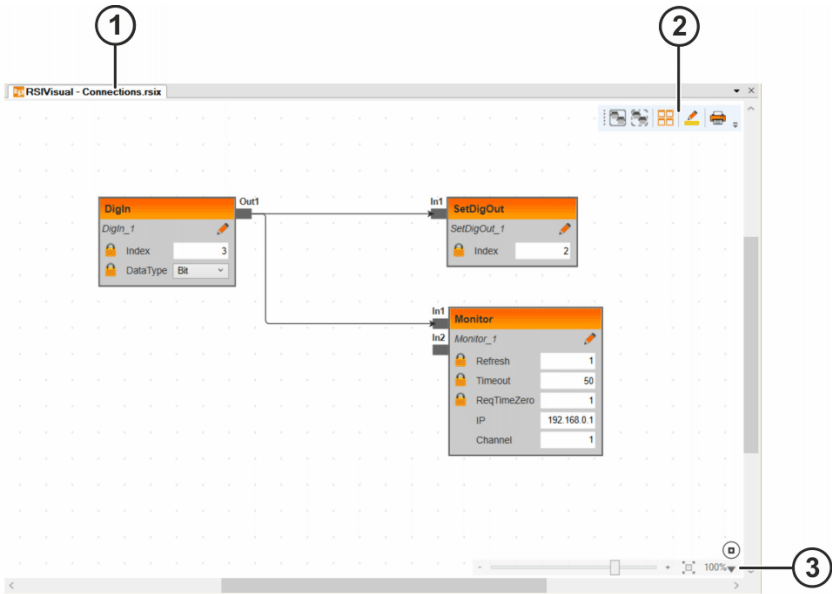


Abb. 8-2: Kontext-Editor

- 1 Name des RSI-Kontexts
- 2 Editor-Funktionen
- 3 Skalierung

Editor-Funktionen

Button	Name/Beschreibung
	Gruppiere markierte Elemente Erstellt eine Gruppe um die im Editor markierten RSI-Elemente. (>>> 8.4.6 "RSI-Elemente gruppieren" Seite 77) Der Button ist aktiv (orange), sobald ein RSI-Element markiert ist.
	Gruppierung aufheben Löst eine vorhandene Gruppe auf. Der Button ist aktiv (orange), sobald eine aufgeklappte Gruppe markiert ist.
	Automatisch anordnen Ordnet die RSI-Elemente im Editor nach Signalffluss an. (>>> 8.4.5 "RSI-Elemente automatisch anordnen" Seite 77) Der Button ist immer aktiv (orange).

Button	Name/Beschreibung
	Markiere angeschlossene Verbindungen Aktiviert das Highlighten von verbundenen Links. Aktueller Status Highlighten inaktiv.
	Markiere angeschlossene Verbindungen Deaktiviert das Highlighten von verbundenen Links. Aktueller Status Highlighten aktiv Wenn das Highlighten aktiv ist, werden die Verbindungen eines markierten RSI-Elements visuell hervorgehoben.
	Kontext drucken Druckt die aktuelle Kontextansicht. Der Druckdialog von Windows wird gestartet.

Skalierung

Button	Name/Beschreibung
	Skalierung verkleinern
	Schieberegler zum Ändern der Skalierung
	Skalierung vergrößern
	An Fenstergröße anpassen Passt die Skalierung so an, dass alle RSI-Elemente im Editor angezeigt werden.
	Skalierung auswählen Zeigt die aktuelle Skalierung an und öffnet ein Menü zur Auswahl einer neuen Skalierung (25 %, 50 %, 75 %, 100 %).

Voraussetzung

- RSI-Kontextdatei ist im Projektverzeichnis vorhanden.
(>>> [8.2 "RSI-Kontext anlegen" Seite 71](#))

Vorgehensweise

- Auf die RSI-Kontextdatei doppelklicken oder rechtsklicken und im Kontextmenü **RSIVisual** wählen.

Alternative Vorgehensweise

- Die RSI-Kontextdatei markieren.
- Menüfolge **Editoren > Optionspakete > RSIVisual** wählen.

8.4.1 RSI-Element in Editor einfügen

Beschreibung

RSI-Elemente, die in den Editor eingefügt werden, erhalten automatisch einen generischen Namen. Dieser ist wie folgt aufgebaut:

- Elementname plus Unterstrich mit laufender Nummer

Werden z. B. 3 RSI-Objekte vom Typ **Input** eingefügt, erhalten diese automatisch folgende Namen:

- Input_1, Input_2, Input_3

Der Name ist bearbeitbar (Stift-Symbol) und kann geändert werden:

- Der neue Name muss eindeutig sein, d. h. er darf nicht bereits im Kontext vorhanden sein.
- Wenn ein Name eingegeben wird, der nicht eindeutig ist, wird die Eingabe rot umrandet. Die Änderung kann dann nicht gespeichert werden und im Meldungsfenster wird eine entsprechende Fehlermeldung angezeigt.

Voraussetzung

- RSI-Kontext ist im Editor geöffnet.
- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

- Das gewünschte Element aus dem Werkzeugkasten per Drag&Drop in den Editor ziehen.

8.4.2 RSI-Objektparameter bearbeiten

Die RSI-Objekte sind gemäß den Objektbeschreibungen in der Hilfedatei zu verwenden und zu parametrieren.

Wird für einen Objektparameter ein ungültiger Wert eingegeben, wird die Eingabe rot umrandet. Der Kontext kann dann nicht gespeichert werden und im Meldungsfenster wird eine entsprechende Fehlermeldung angezeigt.

8.4.2.1 RSI-Objektparameter freischalten

Beschreibung

Wenn der Parameter eines RSI-Objekts freigeschaltet ist, kann sein Wert im KRL-Programm gelesen und neu eingestellt werden.

(>>> [7.4.2 "Zugriff auf RSI-Objektparameter über KRL" Seite 60](#))

Voraussetzung

- Parameter ist mit Schloss-Symbol gekennzeichnet.
- Schloss ist geschlossen.

Vorgehensweise

- Auf das Schloss-Symbol klicken. Schloss öffnet sich.
- Um den Parameter wieder zu sperren, erneut auf das Schloss-Symbol klicken. Schloss schließt sich.

Beispiel

Im RSI-Objekt Sine_1 ist der Parameter **Amp** freigeschaltet. Das Schloss ist offen. Der Amplitudenwert für das Sinussignal kann in KRL gelesen und geschrieben werden.



Abb. 8-3: Parameter Amp freigeschaltet

8.4.3 Ein-/Ausgänge der RSI-Elemente verbinden

Beschreibung

Die Ein-/Ausgänge der RSI-Elemente im Editor können in beide Richtungen verbunden werden, von Eingang zu Ausgang oder umgekehrt. Rot markierte Eingänge sind Pflichteingänge und müssen mit Ausgängen verbunden werden. Wenn gespeichert wird, ohne dass alle Pflichteingänge verbunden sind, wird im Meldungsfenster eine entsprechende Warnung angezeigt.



Sind in einem RSI-Kontext noch unverbundene Pflichteingänge vorhanden, kann der Kontext nicht auf der Robotersteuerung ausgeführt werden.

Ein Kontextmenü für markierte Verbindungen ermöglicht die direkte Navigation im Editor.

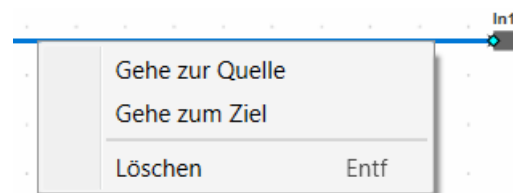


Abb. 8-4: Kontextmenü für markierte Verbindungen

Über das Kontextmenü kann die Ansicht zum Quell- oder Ziel-E/A verschoben werden.

Voraussetzung

- RSIVisual-Editor ist geöffnet.

Vorgehensweise

1. Mit gedrückter linker Maustaste auf Ausgang klicken und in Richtung Eingang bewegen.
ODER:
Mit gedrückter linker Maustaste auf Eingang klicken und in Richtung Ausgang bewegen.
2. Maustaste loslassen, wenn die Verbindung wie gewünscht angezeigt wird. Wurde ein Pflichteingang mit einem Ausgang verbunden, wechselt seine Farbe von Rot nach Grau.

8.4.4 Informationen zu RSI-Elementen anzeigen

Beschreibung

Zu den RSI-Elementen im Editor und im Werkzeugkasten können Informationen angezeigt werden.

Voraussetzung

- RSI-Kontext ist im Editor geöffnet.
- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

- Mit dem Mauszeiger über das Element fahren, zu dem Informationen angezeigt werden sollen.

8.4.5 RSI-Elemente automatisch anordnen

Beschreibung

RSI-Elemente können, basierend auf ihren Verbindungen, automatisch im Editor angeordnet werden. Die Anordnung folgt dabei dem Signalfluss.

- RSI-Elemente ohne verbundene Eingänge werden links angeordnet.
- RSI-Elemente mit verbundenen Eingängen und verbundenen Ausgängen werden in der Mitte angeordnet.
- RSI-Elemente ohne verbundene Ausgänge werden rechts angeordnet.

Buttons

Button	Name/Beschreibung
	Automatisch anordnen Ordnet die RSI-Elemente im Editor nach Signalfluss an. Der Button ist immer aktiv (orange).

Voraussetzung

- RSIVisual-Editor ist geöffnet.

Vorgehensweise

- Auf den Button **Automatisch anordnen** klicken.
- Alternativ: In den Editor rechtsklicken und im Kontextmenü **Automatisch anordnen** wählen.

8.4.6 RSI-Elemente gruppieren

Beschreibung

Im Editor können mehrere RSI-Elemente in eine Gruppe zusammengefasst werden. Gruppen dienen dazu, die Übersichtlichkeit in großen Kontexten zu erhöhen. Außerdem können die erstellten Gruppen im benutzerdefinierten Werkzeugkasten gespeichert und wiederverwendet werden.

(>>> [8.5.3 "Gruppe in benutzerdefiniertem Werkzeugkasten hinzufügen" Seite 82](#))

Gruppen werden mit einem grauen Rahmen gekennzeichnet und erhalten automatisch einen generischen Gruppennamen: Group_1, Group_2 etc. Der Gruppennamen kann geändert werden. Der neue Gruppennamen muss eindeutig sein, d. h. er darf nicht bereits im Werkzeugkasten vorhanden sein. Wird ein Name eingegeben, der nicht eindeutig ist, wird die Eingabe rot umrandet und kann nicht gespeichert werden. Im Meldungsfenster wird eine entsprechende Fehlermeldung angezeigt.

Buttons

Zum Gruppieren von RSI-Elementen stehen folgende Buttons im Editor zur Verfügung:

Button	Name/Beschreibung
	Gruppieren markierte Elemente Erstellt eine Gruppe um die im Editor markierten RSI-Elemente. Der Button ist aktiv (orange), sobald ein RSI-Element markiert ist.
	Gruppierung aufheben Löst eine vorhandene Gruppe auf. Der Button ist aktiv (orange), sobald eine aufgeklappte Gruppe markiert ist.

Voraussetzung

- RSIVisual-Editor ist geöffnet.
- Zum Auflösen einer Gruppe: Gruppe ist aufgeklappt.

Vorgehensweise

Gruppe erstellen:

1. RSI-Elemente, die gruppiert werden sollen, im Editor markieren.
2. Auf den Button **Gruppieren markierte Elemente** klicken.
 Alternativ: In den Editor rechtsklicken und im Kontextmenü **Gruppieren markierte Elemente** wählen.

Gruppe auflösen:

1. Gruppe oder Gruppen, die aufgelöst werden sollen, im Editor markieren.
2. Auf den Button **Gruppierung aufheben** klicken.
 Alternativ: In den Editor rechtsklicken und im Kontextmenü **Gruppierung aufheben** wählen.

Beispiel

Eine Timer-Funktion wurde zu einer Gruppe zusammengefasst. Der Button links neben dem Gruppennamen dient zum Auf- und Zuklappen der Gruppe.

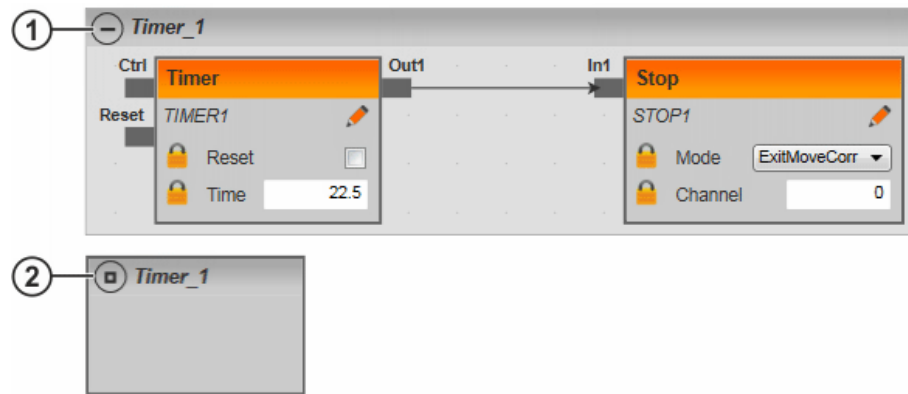


Abb. 8-5: Beispiel für eine Gruppe

1 Zuklappen

2 Aufklappen

8.4.7 Weitere Bearbeitungsfunktionen

Weitere Bearbeitungsfunktionen können über das Menü **Bearbeiten** in WorkVisual aufgerufen werden:

Ausschneiden, Kopieren, Löschen

Voraussetzung:

- RSI-Element ist markiert.

Einfügen

Voraussetzung:

- RSI-Element wurde kopiert.



Zum Ausführen dieser Funktionen können auch die üblichen Tastenkombinationen verwendet werden, z. B. STRG+C zum Kopieren eines markierten Objekts.

8.5 Fenster Werkzeugkasten einblenden

Beschreibung

Sobald ein RSI-Kontext im Editor geöffnet ist, stehen die RSI-Objekte im Werkzeugkasten von WorkVisual zur Verfügung und können per Drag&Drop in den Editor gezogen werden.

Wenn der Werkzeugkasten noch nicht in WorkVisual angezeigt wird, muss das Fenster eingeblendet werden.

Voraussetzung

- RSIVisual-Editor ist geöffnet.

Vorgehensweise

- Menüfolge **Fenster > Werkzeugkasten** wählen.

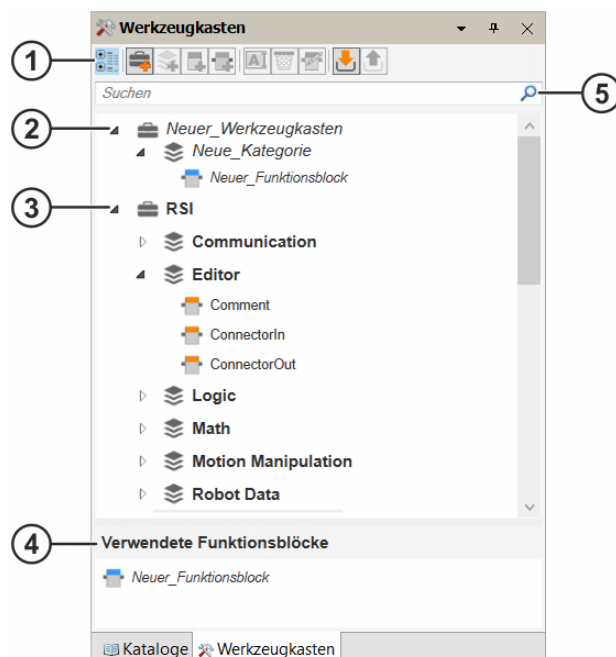


Abb. 8-6: Werkzeugkasten

Pos.	Beschreibung
1	Buttonleiste
2	Benutzerdefinierter Werkzeugkasten
3	Werkzeugkasten mit den RSI-Objekten
4	Anzeige der Funktionsblöcke, die im aktuell geöffneten RSI-Kontext verwendet werden
5	Suchfilter für die Elemente im Werkzeugkasten Die Elemente im Werkzeugkasten können gefiltert werden, indem man einen Teil des Elementnamens eingibt. Die Filterung beginnt mit der Eingabe des ersten Buchstabens. Groß- und Kleinschreibung ist nicht relevant.

Buttons

Button	Name/Beschreibung
	Werkzeugkasten kategorisiert anzeigen. Wechselt die Anzeige der RSI-Objekte im Fenster Werkzeugkasten : - Anzeige nach Kategorien - Anzeige in alphabetischer Reihenfolge
	Neuen benutzerdefinierten Werkzeugkasten erstellen Öffnet einen Dialog zum Erstellen eines neuen benutzerdefinierten Werkzeugkastens.
	Neue Kategorie erstellen Öffnet einen Dialog zum Erstellen einer neuen Kategorie in dem ausgewählten benutzerdefinierten Werkzeugkasten.

Button	Name/Beschreibung
	Neuen Funktionsblock erzeugen Öffnet einen Dialog zum Erstellen eines neuen Funktionsblocks in der ausgewählten Kategorie eines benutzerdefinierten Werkzeugkastens.
	Umbenennen Öffnet einen Dialog zum Umbenennen des ausgewählten Elements: Benutzerdefinierter Werkzeugkasten, Kategorie, Funktionsblock oder Gruppe
	Entfernen Entfernt das ausgewählte Element: Benutzerdefinierter Werkzeugkasten, Kategorie, Funktionsblock oder Gruppe
	Funktionsblock bearbeiten Öffnet den in einem benutzerdefinierten Werkzeugkasten ausgewählten Funktionsblock zur Bearbeitung im Editor.
	Benutzerdefinierten Werkzeugkasten importieren Importiert einen als xml-Datei gespeicherten benutzerdefinierten Werkzeugkasten.
	Exportieren Exportiert den ausgewählten benutzerdefinierten Werkzeugkasten und speichert ihn als xml-Datei.

8.5.1 Benutzerdefinierten Werkzeugkasten erstellen

Beschreibung

In einem benutzerdefinierten Werkzeugkasten können Funktionsblöcke und gruppierte RSI-Elemente (Gruppen) verwaltet und verfügbar gemacht werden.

Jeder benutzerdefinierte Werkzeugkasten enthält mindestens eine Kategorie. Weitere Kategorien können hinzugefügt werden.

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

- Im Werkzeugkasten auf den Button **Neuen benutzerdefinierten Werkzeugkasten erstellen** klicken.
- Standardname mit **OK** übernehmen oder einen neuen Namen für den Werkzeugkasten eingeben und mit **OK** bestätigen.
Der neue Werkzeugkasten wird im Fenster **Werkzeugkasten** erstellt. Er enthält bereits eine erste Kategorie.
- Um eine weitere Kategorie hinzuzufügen, den neu erstellten Werkzeugkasten markieren und auf den Button **Neue Kategorie erstellen** klicken.
Alternativ: Auf den neu erstellten Werkzeugkasten rechtsklicken und im Kontextmenü **Kategorie hinzufügen** wählen.
- Standardname mit **OK** übernehmen oder einen neuen Namen für die Kategorie eingeben und mit **OK** bestätigen.

8.5.2 Kategorie für benutzerdefinierten Werkzeugkasten erstellen

Beschreibung

Für einen benutzerdefinierten Werkzeugkasten können verschiedene Kategorien erstellt werden. Eine Kategorie kann Gruppen und/oder Funktionsblöcke enthalten.

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

1. Gewünschten benutzerdefinierten Werkzeugkasten markieren und auf den Button **Neue Kategorie erstellen** klicken.
Alternativ: Auf den benutzerdefinierten Werkzeugkasten rechtsklicken und im Kontextmenü **Kategorie hinzufügen** wählen.
2. Standardname mit **OK** übernehmen oder einen neuen Namen für die Kategorie eingeben und mit **OK** bestätigen.

8.5.3 Gruppe in benutzerdefiniertem Werkzeugkasten hinzufügen

Beschreibung

Gruppierte RSI-Elemente (Gruppen) können einer der Kategorien, die in einem benutzerdefinierten Werkzeugkasten vorhanden sind, hinzugefügt werden.

Voraussetzung

- RSI-Kontext ist im Editor geöffnet.
- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

1. Auf die gewünschte Kategorie rechtsklicken und im Kontextmenü **Gruppe hinzufügen ...** wählen.
Der Dialog **Gruppe hinzufügen** öffnet sich. Die im aktuellen Editor verfügbaren Gruppen werden angezeigt.
2. Die gewünschte Gruppe auswählen und Auswahl mit **OK** bestätigen.
Die Gruppe wird in der ausgewählten Kategorie hinzugefügt.

8.5.4 Funktionsblock in benutzerdefiniertem Werkzeugkasten erstellen

Beschreibung

Die in einem benutzerdefinierten Werkzeugkasten erstellten Funktionsblöcke können in RSI-Kontexten verwendet werden.

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

1. Gewünschte Kategorie im benutzerdefinierten Werkzeugkasten markieren und auf den Button **Neuen Funktionsblock erzeugen** klicken.
Alternativ: Auf die Kategorie rechtsklicken und im Kontextmenü **Funktionsblock hinzufügen** wählen.
2. Standardname mit **OK** übernehmen oder einen neuen Namen für den Funktionsblock eingeben und mit **OK** bestätigen.

8.5.5 Funktionsblock in RSI-Kontext aktualisieren

Beschreibung

Wenn ein Funktionsblock im Werkzeugkasten bearbeitet wird, ändert sich seine Funktion. Wenn der Funktionsblock vor seiner Bearbeitung bereits in einem RSI-Kontext verwendet wurde, ist im RSI-Kontext die ursprüngliche Version gespeichert. Damit die funktionale Änderung im RSI-Kontext übernommen wird, muss der Funktionsblock aktualisiert werden.

Voraussetzung

- RSI-Kontext, in dem der geänderte Funktionsblock verwendet wird, ist im Editor geöffnet.

Vorgehensweise

- Auf den geänderten Funktionsblock im Werkzeugkasten rechtsklicken und im Kontextmenü **In Kontext aktualisieren** wählen.

8.5.6 Elemente im benutzerdefinierten Werkzeugkasten kopieren, ausschneiden und einfügen

Beschreibung

Funktionsblöcke, Kategorien und Gruppen können innerhalb eines benutzerdefinierten Werkzeugkastens kopiert, ausgeschnitten und wieder eingefügt werden. Es ist auch möglich, diese Elemente in einen anderen benutzerdefinierten Werkzeugkasten zu kopieren oder zu verschieben.

Vorgehensweise

- Auf ein Element im Werkzeugkasten rechtsklicken und im Kontextmenü den gewünschten Befehl wählen.
- Alternativ den gewünschten Befehl mithilfe der gängigen Shortcuts (STRG+C, STRG+X oder STRG+V) ausführen.

8.5.7 Benutzerdefinierten Werkzeugkasten exportieren

Beschreibung

Ein benutzerdefinierter Werkzeugkasten kann aus WorkVisual exportiert und als xml-Datei gespeichert werden.

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

1. Benutzerdefinierten Werkzeugkasten, der exportiert werden soll, markieren und auf den Button **Exportieren** klicken.
Alternativ: Auf den benutzerdefinierten Werkzeugkasten rechtsklicken und im Kontextmenü **Exportieren ...** wählen.
2. Das Fenster **Speichern unter** öffnet sich. Einen Speicherort für den Werkzeugkasten wählen.
3. Bei Bedarf im Feld **Dateiname** einen neuen Namen für den Werkzeugkasten vergeben.
4. Auf den Button **Speichern** klicken.

8.5.8 Benutzerdefinierten Werkzeugkasten importieren

Beschreibung

Ein als xml-Datei gespeicherter benutzerdefinierter Werkzeugkasten kann in den Werkzeugkasten von WorkVisual importiert werden.

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> [8.5 "Fenster Werkzeugkasten einblenden" Seite 79](#))

Vorgehensweise

1. Im Werkzeugkasten auf den Button **Benutzerdefinierten Werkzeugkasten importieren** klicken. Ein Fenster öffnet sich.
2. Zu dem Verzeichnis navigieren, in dem sich die Datei befindet, die importiert werden soll.
3. Die Datei markieren und auf **Öffnen** klicken.
4. Wenn bereits ein benutzerdefinierter Werkzeugkasten mit demselben Namen vorhanden ist, öffnet sich ein Dialog zum Umbenennen des Werkzeugkastens, der importiert werden soll.
Einen neuen Namen eingeben und mit **OK** bestätigen.

8.5.9 Benutzerdefinierten Werkzeugkasten im Optionspaketeditor verwenden

Beschreibung

Ein benutzerdefinierter Werkzeugkasten kann im Optionspaketeditor einem benutzerspezifischen Optionspaket hinzugefügt werden. Wird das benutzerspezifische Optionspaket in WorkVisual installiert, steht der benutzerdefinierte Werkzeugkasten im Werkzeugkasten von WorkVisual zur Verfügung.

Voraussetzung

- OptionPackageEditor ist installiert.

Vorgehensweise

1. Benutzerdefinierten Werkzeugkasten aus WorkVisual exportieren.
(>>> [8.5.7 "Benutzerdefinierten Werkzeugkasten exportieren" Seite 83](#))
2. Optionspaketeditor starten.
3. Benutzerspezifisches Optionspaket im Optionspaketeditor öffnen oder neu erstellen.



Weitere Informationen zum Erstellen und Bearbeiten von Optionspaketen sind in der Dokumentation **OptionPackageEditor** zu finden.

4. In der Struktur des benutzerspezifischen Optionspakets im Fenster **Optionspaketstruktur** ein neues Verzeichnis mit dem Namen **Rsi-Toolbox** unter **WorkVisual > Plugins** einfügen
5. Den benutzerdefinierten Werkzeugkasten (xml-Datei) in dem neuen Verzeichnis hinzufügen.

Beispiel

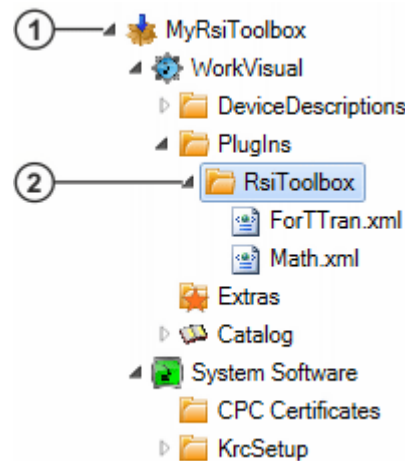


Abb. 8-7: Werkzeugkasten im Optionspaketeditor

- 1 Benutzerspezifisches Optionspaket im Fenster **Optionspaketstruktur**
- 2 Verzeichnis mit 2 benutzerdefinierten Werkzeugkästen

8.6 Funktionsblockprogrammierung mit RSIVisual

Ein Funktionsblock stellt eine wiederverwendbare funktionale Einheit eines RSI-Kontexts dar. Während Gruppen vor allem dazu dienen, einen Kontext übersichtlicher zu machen, soll ein Funktionsblock wie ein Standard-RSI-Element verwendet werden können, ohne dass der Anwender Informationen über den Inhalt des Funktionsblocks erhält oder benötigt.

8.6.1 Funktionsblock im Funktionsblock-Editor öffnen

Beschreibung

Ein neu erstellter Funktionsblock besitzt noch keinen Inhalt. Zur Bearbeitung steht in RSIVisual ein eigener Funktionsblock-Editor zur Verfügung. Der Name des Funktionsblocks und der Name des benutzerdefinierten Werkzeugkastens, in dem der Funktionsblock erstellt wurde, werden im Editor angezeigt.

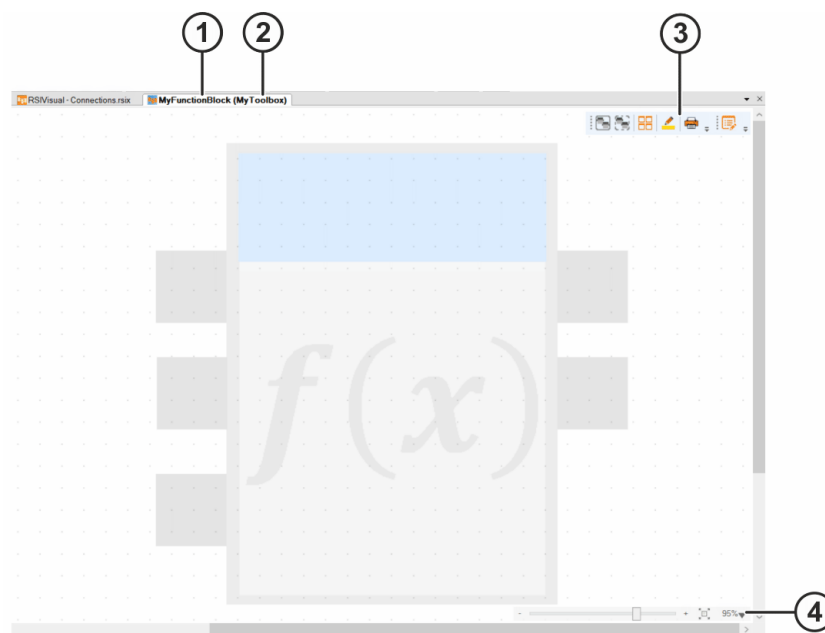


Abb. 8-8: Funktionsblock-Editor

- | | | | |
|---|--------------------------|---|-------------------|
| 1 | Name des Funktionsblocks | 3 | Editor-Funktionen |
| 2 | Name des Werkzeugkastens | 4 | Skalierung |

Editor-Funktionen

Button	Name/Beschreibung
	Gruppieren markierte Elemente Erstellt eine Gruppe um die im Editor markierten RSI-Elemente. (>>> 8.4.6 "RSI-Elemente gruppieren" Seite 77) Der Button ist aktiv (orange), sobald ein RSI-Element markiert ist.
	Gruppierung aufheben Löst eine vorhandene Gruppe auf. Der Button ist aktiv (orange), sobald eine aufgeklappte Gruppe markiert ist.
	Automatisch anordnen Ordnet die RSI-Elemente im Editor nach Signalfluss an. (>>> 8.4.5 "RSI-Elemente automatisch anordnen" Seite 77) Der Button ist immer aktiv (orange).
	Markiere angeschlossene Verbindungen Aktiviert das Highlighten von verbundenen Links. Aktueller Status Highlighten inaktiv.
	Markiere angeschlossene Verbindungen Deaktiviert das Highlighten von verbundenen Links. Aktueller Status Highlighten aktiv Wenn das Highlighten aktiv ist, werden die Verbindungen eines markierten RSI-Elements visuell hervorgehoben.

Button	Name/Beschreibung
	Kontext drucken Druckt die aktuelle Kontextansicht. Der Druckdialog wird gestartet.
	Bearbeiten der Funktionsblockeigenschaften. Öffnet das Fenster Funktionsblockeigenschaften . (>>> 8.6.4 "Funktionsblockeigenschaften definieren" Seite 89)

Skalierung

Button	Name/Beschreibung
	Skalierung verkleinern
	Schieberegler zum Ändern der Skalierung
	Skalierung vergrößern
	An Fenstergröße anpassen Passt die Skalierung so an, dass alle RSI-Elemente im Editor angezeigt werden.
	Skalierung auswählen Zeigt die aktuelle Skalierung an und öffnet ein Menü zur Auswahl einer neuen Skalierung (25 %, 50 %, 75 %, 100 %).

Voraussetzung

- Fenster **Werkzeugkasten** ist eingeblendet.
(>>> 8.5 "Fenster Werkzeugkasten einblenden" Seite 79)

Vorgehensweise

- Gewünschten Funktionsblock im benutzerdefinierten Werkzeugkasten markieren und auf den Button **Funktionsblock bearbeiten** klicken.
- Alternativ: Auf den Funktionsblock rechtsklicken und im Kontextmenü **Funktionsblock bearbeiten** wählen.

8.6.2 Funktionsblock programmieren

Beschreibung

Ein Funktionsblock kann wie ein RSI-Kontext programmiert werden. Alle RSI-Objekte und Gruppen aus dem Werkzeugkasten stehen hierfür zur Verfügung. Andere Funktionsblöcke können nicht in einem Funktionsblock verwendet werden.

Voraussetzung

- Funktionsblock ist im Editor geöffnet.

Vorgehensweise

- Gewünschte Funktion mithilfe der RSI-Objekte und Gruppen programmieren.
- Funktionsblockeigenschaften wie benötigt bearbeiten.

(>>> 8.6.4 "Funktionsblockeigenschaften definieren" Seite 89)

3. Funktionsblock speichern.

8.6.3 Ein- und Ausgänge von Funktionsblöcken definieren

Beschreibung

Die Ein- und Ausgänge eines Funktionsblocks können über spezielle Verbindungselemente definiert werden.

Ein Eingang eines RSI-Objekts kann im Funktionsblockeditor mit einem Verbindungselement ConnectorIn und ein Ausgang eines RSI-Objekts mit einem Verbindungselement ConnectorOut verknüpft werden.

Bei Verwendung des Funktionsblocks in einem RSI-Kontext wird ein mit ConnectorIn verknüpfter Eingang des RSI-Objekts als Eingang des Funktionsblocks dargestellt. Ein mit ConnectorOut verknüpfter Ausgang des RSI-Objekts wird als Ausgang des Funktionsblocks dargestellt. Der Funktionsblock kann somit im RSI-Kontext mit anderen RSI-Objekten oder Funktionsblöcken verbunden werden.

Die Verbindungselemente können nur im Funktionsblockeditor verwendet werden.

Verbindungselemente

Die Verbindungselemente sind im Werkzeugkasten unter dem Element **Editor** zu finden:

RSI-Objekt	Name/Beschreibung
ConnectorIn ConnectorIn_1 	Verbindungselement für Eingänge im Funktionsblock Beim Einfügen in den Funktionsblock wird automatisch ein Name und eine Nummer für das Verbindungsobjekt vergeben: • Der Name kann geändert werden. Dieser wird bei Verwendung des Funktionsblocks als Name des Funktionsblockeingangs dargestellt. • Die Nummer entspricht der Reihenfolge in der Darstellung der Funktionsblockeingänge beginnend von oben. Das Verbindungselement muss im Funktionsblockeditor mit dem Eingang eines RSI-Objekts verbunden werden. Hinweis: Alle Eingänge, die ein Funktionsblock mitbringt, müssen bei Verwendung in einem RSI-Kontext verschaltet werden.
ConnectorOut ConnectorOut_1 	Verbindungselement für Ausgänge im Funktionsblock Beim Einfügen in den Funktionsblock wird automatisch ein Name und eine Nummer für das Verbindungsobjekt vergeben: • Der Name kann geändert werden. Dieser wird bei Verwendung des Funktionsblocks als Name des Funktionsblockausgangs dargestellt. • Die Nummer entspricht der Reihenfolge in der Darstellung der Funktionsblockausgänge beginnend von oben. Das Verbindungselement muss im Funktionsblockeditor mit dem Ausgang eines RSI-Objekts verbunden werden.

8.6.4 Funktionsblockeigenschaften definieren

Beschreibung

Im Funktionsblockeditor können folgende Funktionsblockeigenschaften definiert werden:

- **Funktionsblockbeschreibung**

Für den Funktionsblock kann eine Beschreibung eingegeben werden. Diese Beschreibung wird als Hilfetext angezeigt, wenn man im RSI-Kontext oder im Werkzeugkasten mit dem Mauszeiger über den Funktionsblock fährt.

- **Funktionsblockparameter**

Die Parameter der RSI-Objekte im Funktionsblock können als Funktionsblockparameter definiert werden. Die Funktionsblockparameter werden mit dem Funktionsblock im Werkzeugkasten gespeichert.

Bei der Verwendung des Funktionsblocks im RSI-Kontext können die Funktionsblockparameter eingestellt werden:

- Wird ein Funktionsblockparameter mehreren RSI-Objektparametern zugewiesen, ist Folgendes zu beachten:
 - Die Objektparameter müssen vom gleichen Datentyp sein.
 - Der erste Objektparameter, der dem Funktionsblockparameter zugewiesen wird, legt den Typ der weiteren Objektparameter fest.
- Die eingestellten Funktionsblockparameter werden im RSI-Kontext gespeichert.

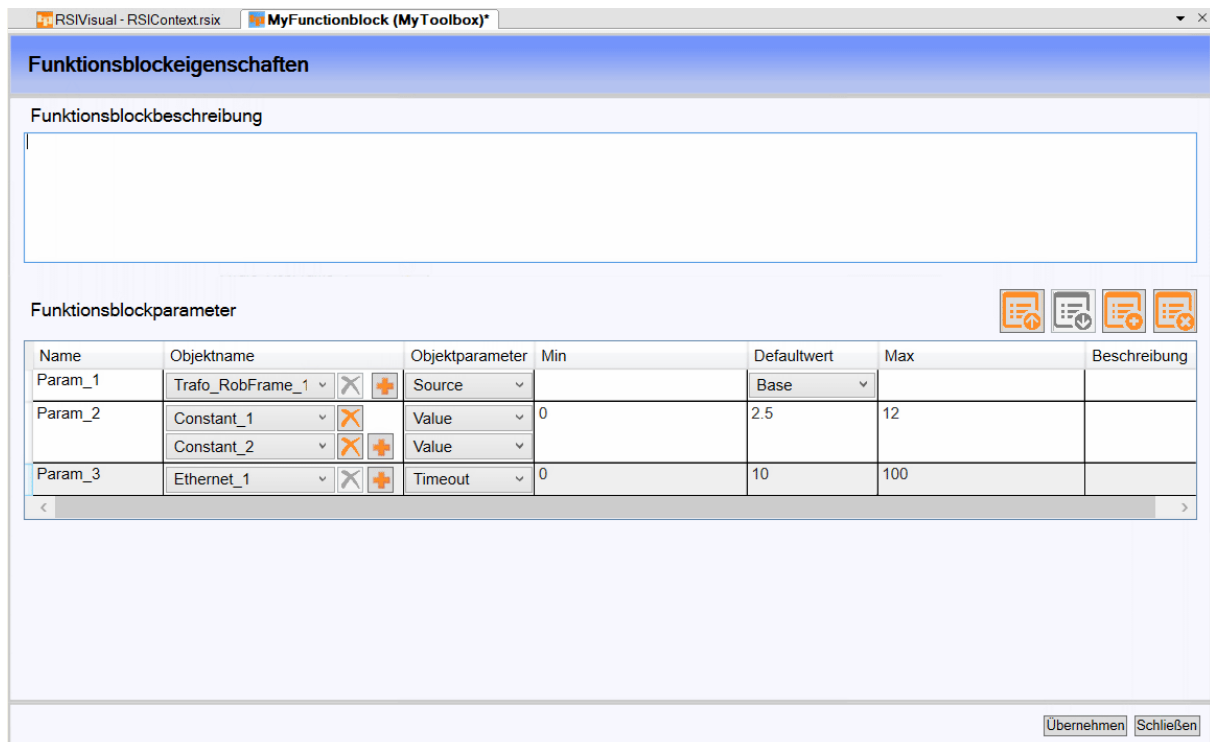


Abb. 8-9: Funktionsblockeigenschaften

Element	Beschreibung
Name	Name des Funktionsblockparameters
Objektname	Name des RSI-Objekts, dessen Parameter mit dem Funktionsblockparameter beschrieben werden soll Es stehen alle parametrierbaren RSI-Objekte zur Auswahl, die im Funktionsblock verwendet werden.
Objektparameter	Parameter des ausgewählten RSI-Objekts, der mit dem Funktionsblockparameter beschrieben werden soll Es stehen alle Parameter des ausgewählten RSI-Objekts zur Auswahl.
Min	Minimaler Wert des Funktionsblockparameters
Defaultwert	Default-Wert des Funktionsblockparameters Dieser Parameterwert kommt standardmäßig zur Anzeige, wenn der Funktionsblock in einem RSI-Kontext verwendet wird.
Max	Maximaler Wert des Funktionsblockparameters
Beschreibung	Beschreibung des Funktionsblockparameters Diese Beschreibung wird als Hilfetext angezeigt, wenn man im RSI-Kontext mit dem Mauszeiger über den Funktionsblockparameter fährt.

Buttons

Zum Erstellen der Funktionsblockparameter stehen folgende Buttons zur Verfügung:

Button	Name/Beschreibung
	Den markierten Parameter nach oben verschieben. Verschiebt den markierten Funktionsblockparameter eine Zeile nach oben.
	Den markierten Parameter nach unten verschieben. Verschiebt den markierten Funktionsblockparameter eine Zeile nach unten.
	Hinzufügen eines neuen Parameters. Fügt einen neuen Funktionsblockparameter ein.
	Den markierten Parameter löschen. Löscht den markierten Funktionsblockparameter.
	Parameter hinzufügen Fügt einen neuen Objektparameter zum Funktionsblockparameter hinzu.
	Parameter löschen Löscht einen Objektparameter. Der Button ist nur aktiv, wenn dem Funktionsblockparameter mehr als 1 Objektparameter zugewiesen ist.

Voraussetzung

- Funktionsblock ist im Editor geöffnet.

Vorgehensweise

1. Im Funktionsblockeditor auf den Button **Bearbeiten der Funktionsblockeigenschaften**. klicken. Das Fenster **Funktionsblockeigenschaften** öffnet sich.
2. Funktionsblockeigenschaften definieren.
3. Änderungen übernehmen und Fenster schließen.

8.7 Zugriff auf Funktionsblockparameter über KRL

Beschreibung

Auf Funktionsblockparameter kann nicht direkt aus KRL heraus zugegriffen werden. Es ist jedoch möglich, speziell die Parameter von RSI-Objekten innerhalb des Funktionsblocks freizuschalten, die auch als Funktionsblockparameter konfiguriert wurden.

(>>> [8.4.2.1 "RSI-Objektparameter freischalten" Seite 75](#))

Über die Funktionen RSI_GETPUBLICPAR() und RSI_SETPUBLICPAR() kann auf diese freigeschalteten Parameter zugegriffen werden, und zwar über einen speziellen Objektname. Dieser spezielle Objektname setzt sich aus 2 Teilen zusammen, die über einen Punkt aneinandergereiht werden:

- Spezieller Objektname = *Teil_1.Teil_2*
 - *Teil_1*: Name des verwendeten Funktionsblocks im RSI-Kontext
 - *Teil_2*: Objektname des RSI-Objekts innerhalb des Funktionsblocks

Voraussetzung

- RSI-Objektparameter ist im Funktionsblock freigeschaltet.
- Funktionsblock wird in einem RSI-Kontext verwendet.

Vorgehensweise

- KRL-Methode für den Funktionsblockparameter erstellen.

Beispiel

Freigeschalteter RSI-Objektparameter im Funktionsblock **Fb_Param**:

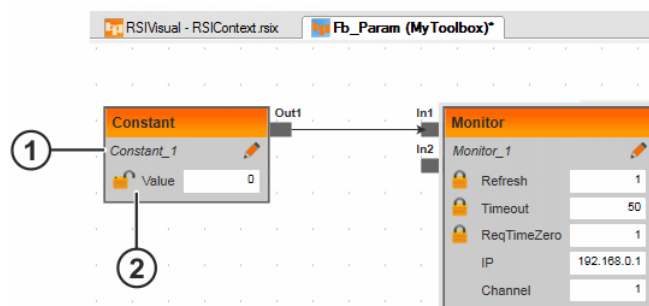


Abb. 8-10: RSI-Objektparameter im Funktionsblock **Fb_Param**

- 1 RSI-Objekt mit dem Namen **Constant_1**
- 2 RSI-Objektparameter **Value** freigeschaltet

Funktionsblock **Fb_Param** im RSI-Kontext:

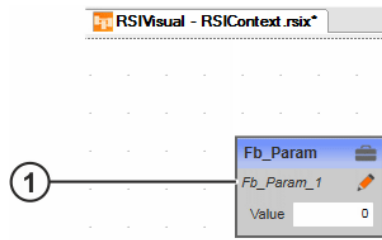


Abb. 8-11: Funktionsblock Fb_Param im RSI-Kontext

1 Funktionsblock mit dem Namen **Fb_Param_1**

KRL-Methode zum Setzen des Funktionsblockparameters:

```

1 DEF FbParam ( )
2
3 INT Ret
4 INT ContId
5
6 Ret = RSI_Create("RSIContext", ContId)
7 IF (Ret <> RSIOK) THEN
8   ;error handling
9   halt
10 ENDIF
11
12 Ret = SetMyFbParam(ContId, 5.67)
13 IF (Ret <> RSIOK) THEN
14   ;error handling
15   halt
16 ENDIF
17
18 ;do rest
19
20 DEFFCT INT SetMyFbParam(ContId :in, Value :in)
21
22 INT ContID
23 REAL Value
24
25 RETURN RSI_SetPublicPar(ContId, "Fb_Param_1.Constant_1",
    "Value", Value)
26
27 ENDFCT

```

Zeile	Beschreibung
6	RSI_Create() initialisiert die Signalverarbeitung. Der RSI-Kontext wird in einen RSI-Container geladen.
25	RSI_SetPublicPar() weist dem im RSI-Kontext verwendeten Funktionsblockparameter einen neuen Wert zu.

8.8 Hilfedatei

Die Hilfedatei mit den RSI-Objektbeschreibungen befindet sich im Katalog **Optionen** im Ordner **Dokumentation** der Option RobotSensorInterface.

9 Beispielkonfigurationen und Beispielprogramme

9.1 Server-Programm und Beispieldateien einbinden

Beschreibung

Im Lieferumfang der Software sind ein Server-Programm sowie verschiedene Beispieldateien enthalten:

- Auf dem Datenträger mit den Installationsdateien der Software unter .DOC\Examples\...
- Nach Installation in WorkVisual im Katalog **Optionen** im Ordner **Dokumentation\Examples\...** der Option

Server-Programm und Beispieldateien zum Herstellen einer Ethernet-Kommunikation mit der Robotersteuerung:

Ordner	Dateien
...\Ethernet\Server	Server-Programm • TestServer.exe
...\Ethernet	RSI-Kontext mit KRL-Programm • RSI_Ethernet.rsix • RSI_Ethernet.src
	XML-Datei für Ethernet-Verbindung • RSI_EthernetConfig.xml

Weitere Beispieldateien:

Ordner	Dateien
...\CircleCorr	RSI-Kontext mit KRL-Programm • RSI_CircleCorr.rsix • RSI_CircleCorr.src
...\DistanceCtrl	RSI-Kontext mit KRL-Programm • RSI_DistanceCtrl.rsix • RSI_DistanceCtrl.src
...\Transformations	RSI-Kontext mit KRL-Programm • RSI_SigTransformation.rsix • RSI_SigTransformation.src

Zielverzeichnisse

Die Beispieldateien müssen auf die Robotersteuerung kopiert werden, z. B.:

- Direkt über USB-Stick
- Über Projekt in WorkVisual
- Über den KRC-Explorer in WorkVisual (online arbeiten)

Zielverzeichnis für KRL-Programme:

- C:\KRC\ROBOTER\KRC\R1\Program

Zielverzeichnis für RSI-Kontexte und XML-Datei für die Ethernet-Verbindung:

- C:\KRC\ROBOTER\Config\User\Common\SensorInterface

Voraussetzung

Externes System für Server-Programm:

- Windows-Betriebssystem mit installiertem .NET-Framework 3.5 oder höher

Robotersteuerung:

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Zum Kopieren/Übertragen der Beispieldateien:

- USB-Stick mit den Beispieldateien
- ODER:
- Bei Verwendung von WorkVisual: Netzwerkverbindung zur realen Robotersteuerung

Vorgehensweise

1. Server-Programm auf externes System kopieren.
2. Beispieldateien über den USB-Stick in das jeweilige Zielverzeichnis auf der Robotersteuerung kopieren oder über WorkVisual.
(>>> 9.1.1 "Dateien über Projekt in WorkVisual auf Steuerung übertragen" Seite 94)
(>>> 9.1.2 "Dateien über KRC-Explorer von WorkVisual auf Steuerung übertragen" Seite 94)
3. Server-Programm auf dem externen System starten.
4. Menü-Button drücken. Das Fenster **Server Properties** öffnet sich.
5. Unter **Available Network Interfaces** werden die verfügbaren IP-Adressen des Server-PCs angezeigt.
6. Die IP-Adresse der Verbindung zwischen Roboter und PC in der XML-Datei für die Ethernet-Verbindung einstellen.

9.1.1 Dateien über Projekt in WorkVisual auf Steuerung übertragen

Vorgehensweise

1. Das aktive Projekt von der Robotersteuerung in WorkVisual laden.
2. Beispieldateien aus dem Katalog **Optionen** per Drag&Drop in das jeweilige Zielverzeichnis im Projekt einfügen.
3. Das Projekt zurück auf die Robotersteuerung übertragen.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

9.1.2 Dateien über KRC-Explorer von WorkVisual auf Steuerung übertragen

Vorgehensweise

1. Das aktive Projekt von der Robotersteuerung in WorkVisual laden.
2. In den Arbeitsbereich **Programmierung und Diagnose** wechseln und eine Verbindung zur Robotersteuerung herstellen.
3. Das Fenster **Kataloge** über das Menü von WorkVisual einblenden.
4. Beispieldateien aus dem Katalog **Optionen** per Drag&Drop in das jeweilige Zielverzeichnis auf der Robotersteuerung einfügen.
5. Die Daten auf der Robotersteuerung aktualisieren.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

9.2 Bedienoberfläche Server-Programm

Das Server-Programm ermöglicht es, die Verbindung zwischen einem externen System und der Robotersteuerung zu testen, indem eine stabile Kommunikation zur Robotersteuerung hergestellt wird.

Dazu werden die empfangenen Daten ausgewertet und der aktuelle Zeitstempel des Paketes in das zu sendende XML-Dokument kopiert. Das XML-Dokument kann mit Korrekturdaten oder mit Nullwerten gesendet werden.

Das Server-Programm enthält folgende Funktionalitäten:

- Senden und Empfangen von Daten im Sensortakt
- Freie Bewegungskorrektur, kartesisch über Bedienelemente
- Anzeige der empfangenen Daten
- Anzeige der gesendeten Daten



Das Testserver-Programm und das Betriebssystem Windows sind nicht echtzeitfähig. Es können keine Rückschlüsse auf das zeitliche Verhalten oder die Stabilität eines Gesamtsystems gezogen werden.

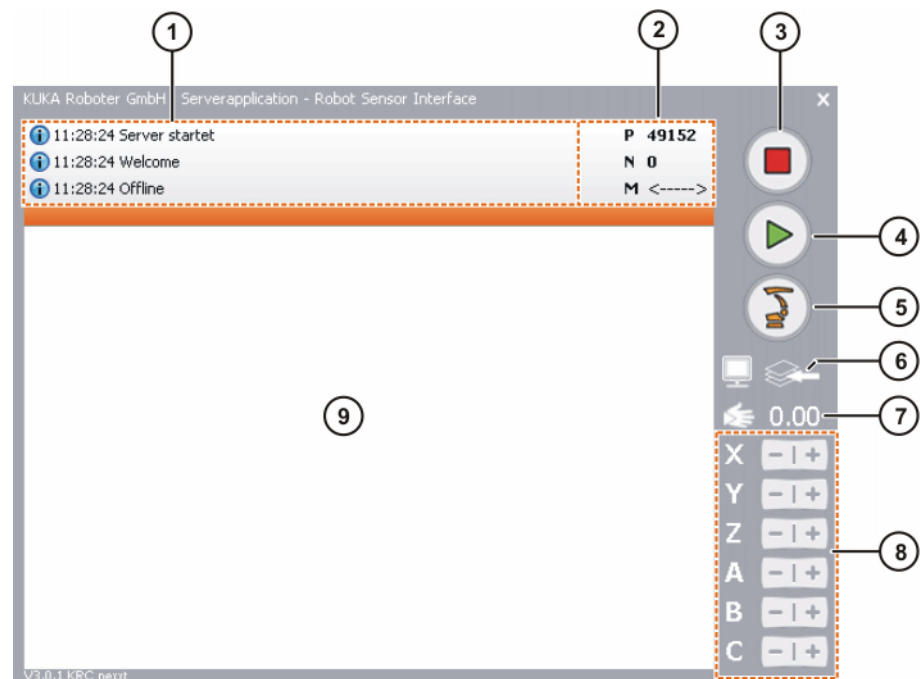


Abb. 9-1: Serverprogramm

Pos.	Beschreibung
1	Meldungsfenster
2	Anzeige der eingestellten Kommunikationsparameter • P: Port-Nummer • N: Netzwerkkarten-Index • M: Kommunikationsmodus – <---->: Der Server kann Daten empfangen und senden. – <-----: Der Server kann nur Daten empfangen.
3	Stopp-Button Die Kommunikation mit der Robotersteuerung wird beendet und der Server wird zurückgesetzt.
4	Start-Button Der Datenaustausch zwischen Server-Programm und Robotersteuerung wird ausgewertet. Die erste eingehende Verbindungsanfrage wird gebunden und als Kommunikationsadapter benutzt.
5	Menü-Button zum Einstellen der Kommunikationsparameter (>>> 9.2.1 "Kommunikationsparameter im Server-Programm einstellen" Seite 96)
6	Anzeigeoptionen • Pfeil zeigt nach links: Die empfangenen Daten werden angezeigt. (Default) • Pfeil zeigt nach rechts: Die gesendeten Daten werden angezeigt.
7	Hand-Symbol Über einen Regler kann die Schrittweite für die Bewegungskorrektur pro Sensortakt eingestellt werden. • 0.00 ... 3.33
8	Buttons für die inkrementelle Bewegungskorrektur pro Sensortakt Die Schrittweite wird über das Hand-Symbol eingestellt.
9	Anzeigefenster Je nach eingestellter Anzeigeoption werden die gesendeten oder die empfangenen Daten angezeigt. Die angezeigten Daten werden im Sensortakt aktualisiert.

9.2.1 Kommunikationsparameter im Server-Programm einstellen

Vorgehensweise

1. Im Server-Programm auf den Menü-Button klicken.
Das Fenster **Server Properties** öffnet sich.
2. Kommunikationsparameter einstellen.
3. Fenster schließen.

Beschreibung

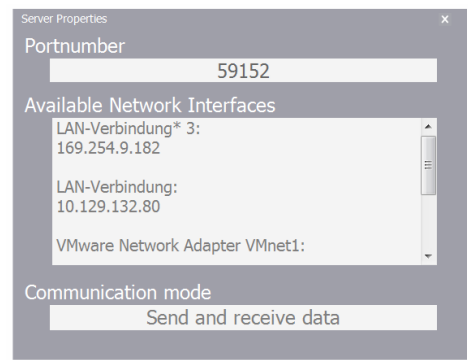


Abb. 9-2: Fenster Server Properties

Element	Beschreibung
Portnummer	Port-Nummer der Socket-Verbindung eingeben. An diesem Port erwartet das externe System die Verbindungsanfrage der Robotersteuerung. Es muss eine freie Nummer gewählt werden, die nicht als Standarddienst belegt ist. Default-Wert: 59152 Hinweis: Bei der Wahl des Ports ist darauf zu achten, dass dieser nicht durch andere Dienste, z. B. vom Betriebssystem verwendet wird. In diesem Fall kann keine Verbindung über diesen Port aufgebaut werden.
Available Network Interfaces	Zeigt die verfügbaren IP-Adressen an, die auf dem verwendeten PC definiert sind.
Communication mode	Kommunikationsmodus auswählen. • Send and receive data: Der Server kann Daten empfangen und senden. • Only receive data: Der Server kann nur Daten empfangen. Default-Wert: Send and receive data

9.3 Beispiel RSI_Ethernet: kartesische Korrektur über Ethernet

Die Robotersteuerung empfängt kartesische Korrekturdaten von einem Sensor und schickt diese an den Roboter. Der Roboter wird rein korrekturgesteuert auf Basis von relativen Korrekturwerten verfahren. Referenzkoordinatensystem für die Korrekturaufschaltung ist das BASE-Koordinatensystem.

Programm

```
1 DEF RSI_Ethernet( )
2 ; =====
3 ;
4 ; RSI EXAMPLE: ETHERNET communication
5 ; Realtime UDP data exchange with server application
6 ;
7 ; =====
8
```

```

 9 ; Declaration of KRL variables
10 DECL INT ret          ; Return value for RSI commands
11 DECL INT CONTID       ; ContainerID
12
13 INI
14
15 ; Move to start position
16 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
17
18 ; Create RSI Context
19 ret = RSI_CREATE("RSI_Ethernet",CONTID,TRUE)
20 IF (ret <> RSIOK) THEN
21   HALT
22 ENDIF
23
24 ; Start RSI execution
25 ret = RSI_ON(#RELATIVE)
26 IF (ret <> RSIOK) THEN
27   HALT
28 ENDIF
29
30 ; Sensor guided movement
31 RSI_MOVECORR()
32
33 ; Turn off RSI
34 ret = RSI_OFF()
35 IF (ret <> RSIOK) THEN
36   HALT
37 ENDIF
38
39 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
40
41 END

```

Zeile	Beschreibung
16	Startposition der sensorgeführten Bewegung
19	RSI_CREATE() lädt den RSI-Kontext in einen RSI-Container.
25	RSI_ON() schaltet die Signalverarbeitung ein. Korrekturmodus: Relativkorrektur
31	RSI_MOVECORR() schaltet die sensorgeführte Bewegung ein.
34	RSI_OFF() schaltet die Signalverarbeitung aus.
39	Zurück zur Startposition

RSI-Kontext

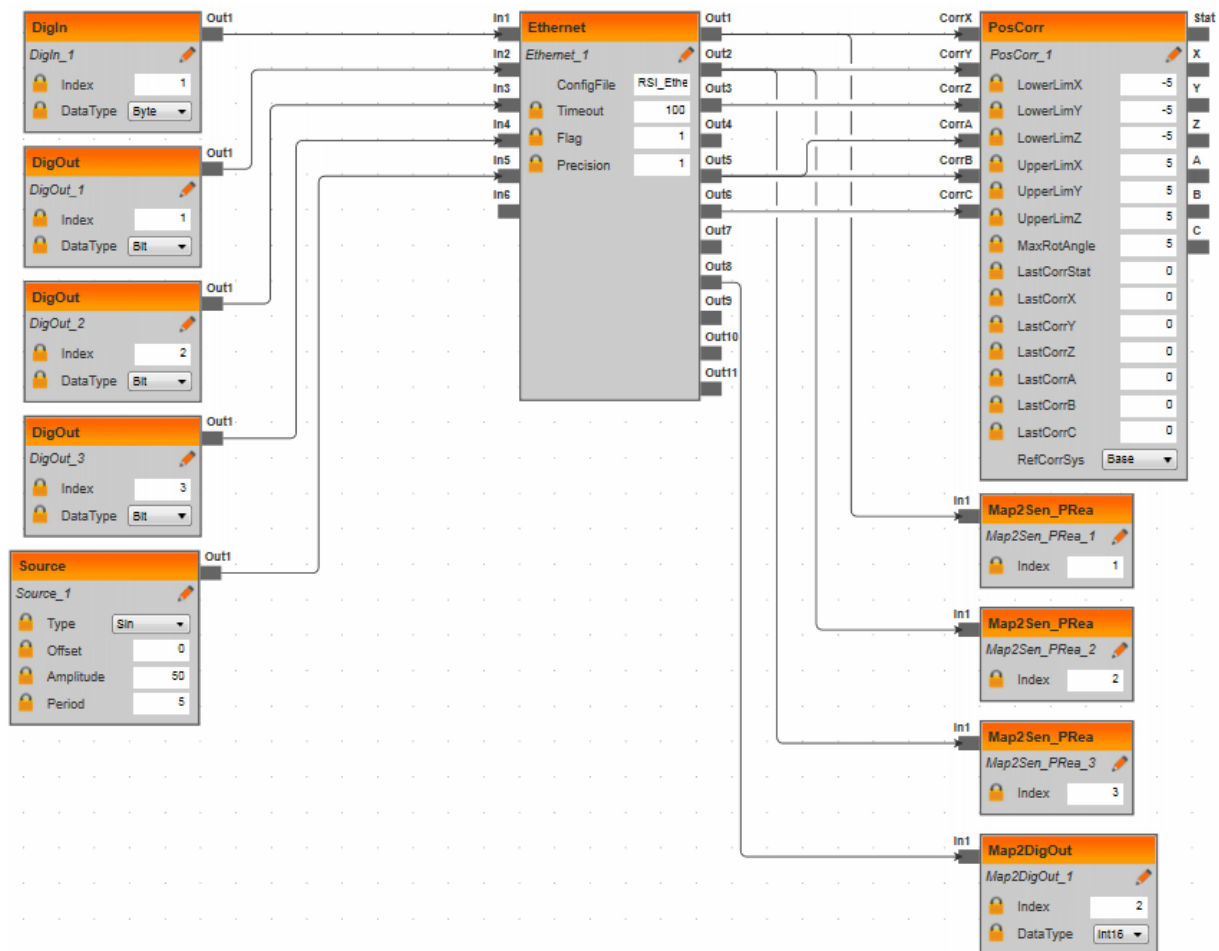


Abb. 9-3: Beispiel RSI_Ethernet: kartesische Korrektur über Ethernet

RSI-Objekt	Beschreibung
Digin_1	Liest über 8 digitale Eingänge die Sensordaten ein und übergibt diese an die Ethernet-Schnittstelle (Eingang 1).
DigOut_1 ... DigOut_3	Liest über 3 digitale Ausgänge Roboterdaten ein und übergibt diese an die Ethernet-Schnittstelle (Eingang 2 ... 4).
Source_1	Liefert ein sinusförmiges Signal mit der Amplitude 50, periodisch alle 5 s.
Ethernet_1	Sendet die über Eingang 2 ... 5 eingehenden Signale an das Sensorsystem und erhält Sensordaten über Eingang 1 zurück. Die Sensordaten stehen an Ausgang 1... 6 zur weiteren Verarbeitung zur Verfügung.
PosCorr_1	Liest die Sensordaten ein, die an Ausgang 1... 6 der Ethernet-Schnittstelle zur Verfügung stehen und ermittelt die kartesischen Korrekturdaten.
Map2Sen_PRea_1 ... Map2Sen_PRea_3	Schreiben die kartesischen Korrekturdaten in die Systemvariable \$SEN_PREA.
Map2DigOut_1	Greift auf die verarbeiteten Signale zu und setzt 16 digitale Ausgänge.

9.4 Beispiel RSI_CircleCorr: sensorgeführte Kreisbewegung

Es wird eine sensorgeführte Kreisbewegung konfiguriert. Dazu wird ein sinusförmiges Signal erzeugt, das als Sinus in Z-Richtung und noch einmal

zeitverzögert als Sinus in Y-Richtung in das Korrekturobjekt POSCORR eingelesen wird. Nach dem ersten Durchlauf der Signalverarbeitung wird die Amplitude des Signals im KRL-Programm nachträglich geändert. Bei erneutem Start der Signalverarbeitung mit halbiertem Amplitude erhält man eine kleinere Kreisbewegung.

Der Roboter wird rein korrekturgesteuert auf Basis der absoluten Korrekturwerte in Y- und Z-Richtung verfahren. Referenzkoordinatensystem für die Korrekturaufschaltung ist das BASE-Koordinatensystem. Über einen Timer wird die sensorgeführte Bewegung nach einer definierten Zeit abgebrochen.

Programm

```

1 DEF RSI_CircleCorr( )
2 ; =====
3
4 ; RSI EXAMPLE: Lissajous circle
5 ; Create a circle movement with two sine corrections
6 ;
7 ; =====
8
9 ; Declaration of KRL variables
10 DECL INT ret          ; Return value for RSI commands
11 DECL INT CONTID       ; ContainerID
12
13 INI
14
15 ; Move to start position
16 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
17
18 ; Base in actual position
19 $BASE.X=$POS_ACT.X
20 $BASE.Y=$POS_ACT.Y
21 $BASE.Z=$POS_ACT.z
22
23 ; Create RSI Context
24 ret=RSI_CREATE("RSI_CircleCorr",CONTID)
25 IF (ret <> RSIOK) THEN
26   HALT
27 ENDIF
28
29 ; Start RSI execution
30 ret=RSI_ON(#ABSOLUTE)
31 IF (ret <> RSIOK) THEN
32   HALT
33 ENDIF
34
35 ; Sensor guided movement
36 RSI_MOVECORR()
37
38 ; Turn off RSI
39 ret=RSI_OFF()
40 IF (ret <> RSIOK) THEN
41   HALT
42 ENDIF
43
44 ; Modify RSI parameter
45 ret=RSI_GETPUBLICPAR(CONTID,"SOURCE1","Amplitude", fVar)
46
47 ...

```

```

49 ret=RSI_SETPUBLICPAR(CONTID,"SOURCE1","Amplitude", fVar/2)

...

54 ; Start RSI execution
55 ret=RSI_ON(#ABSOLUTE)

...

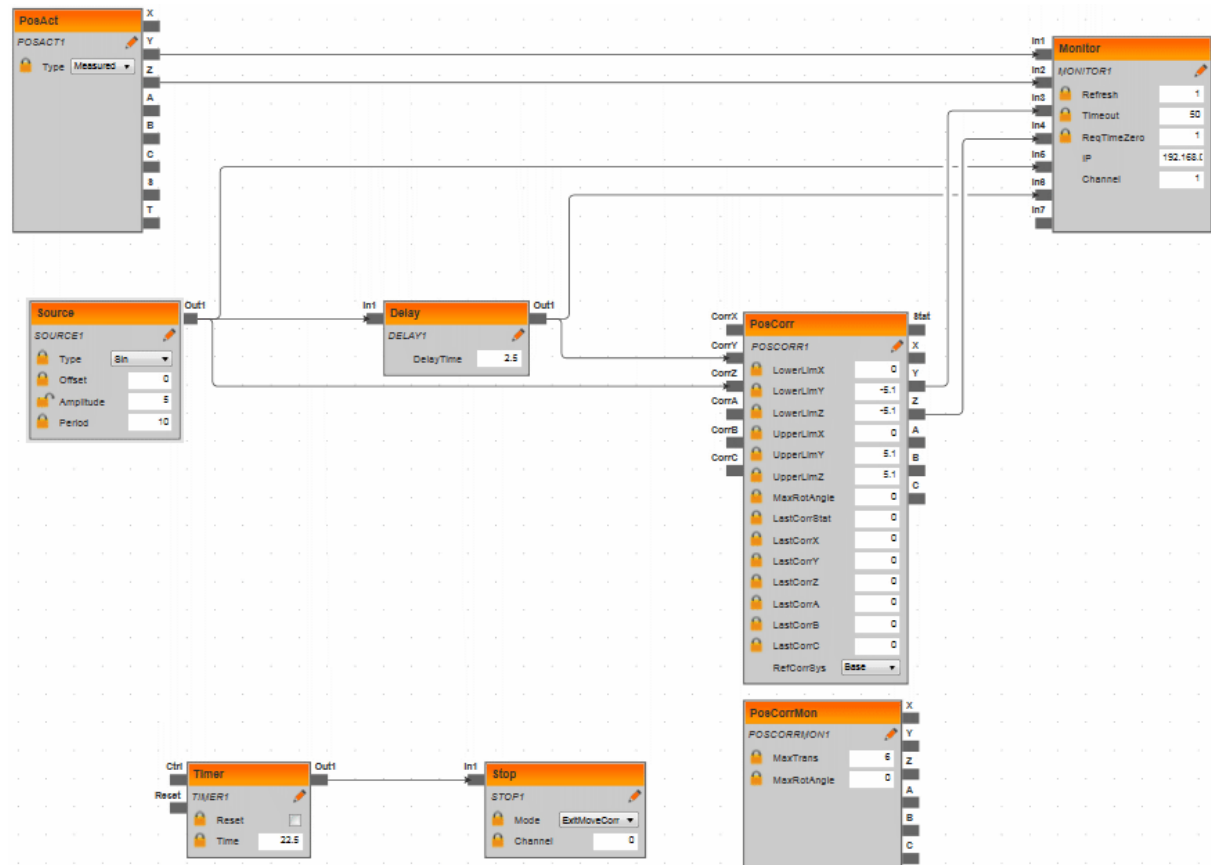
60 ; Sensor guided movement
61 RSI_MOVECORR()
62
63 ; Turn off RSI
64 ret=RSI_OFF()

...

69 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
70
71 END

```

Zeile	Beschreibung
16	Startpunkt der sensorgeführten Bewegung
19 ... 21	Aktuelle Roboterposition bezogen auf die Basis
24	RSI_CREATE() lädt den RSI-Kontext in einen RSI-Container.
30	RSI_ON() schaltet die Signalverarbeitung ein. Korrekturmodus: Absolutkorrektur
36	RSI_MOVECORR() schaltet die sensorgeführte Bewegung ein.
39	RSI_OFF() schaltet die Signalverarbeitung aus.
45	RSI_GETPUBLICPAR() liest die aktuell eingestellte Amplitude des Signals (SOURCE1).
49	RSI_SETPUBLICPAR() weist der Amplitude des Signals (SOURCE1) einen neuen Wert zu. Die Amplitude wird halbiert.
55	RSI_ON() schaltet die Signalverarbeitung ein. Korrekturmodus: Absolutkorrektur
61	RSI_MOVECORR() schaltet die sensorgeführte Bewegung ein.
64	RSI_OFF() schaltet die Signalverarbeitung aus.

RSI-Kontext**Abb. 9-4: Beispiel RSI_CircleCorr: sensorgeführte Kreisbewegung**

RSI-Objekt	Beschreibung
TIMER1	Nach Ablauf der im Timer eingestellten Zeit wird die sensorgeführte Bewegung abgebrochen.
STOP1	
POSCORRMON1	Begrenzt die maximale kartesische Gesamtkorrektur. Maximale translatorische Auslenkung in X, Y, Z: 6 mm
SOURCE1	Liefert ein sinusförmiges Signal mit der Amplitude 5.0, periodisch alle 10 s.
DELAY1	Das Signal wird um 2.5 s verzögert.
POSCORR1	Liest den Sinus-Korrekturwert in Z-Richtung und zeitverzögert den Sinus-Korrekturwert in Y-Richtung ein.
POSACT1	Liest die kartesische Istposition des Roboters in Y- und Z-Richtung ein. Referenzkoordinatensystem für die Korrekturaufschaltung: BASE
MONITOR1	Folgende Signale sind mit dem MONITOR-Objekt verknüpft und können mit RSI-Monitor auf der Robotersteuerung angezeigt werden: Kartesische Istposition des Roboters in Y- und Z-Richtung [mm]

9.5 Beispiel RSI_DistanceCtrl: Bahnkorrektur zur Abstandssteuerung

Es soll ein definierter Abstand zu einem Bauteil eingehalten werden. Bei eingeschalteter Signalverarbeitung misst ein Sensor den Abstand zum Bauteil und verfährt mit einer LIN-Bewegung 100 mm in Y-Richtung. Paral-

Bei einer LIN-Bewegung wird ein relativer Korrekturwert ermittelt und der Bahnverlauf der LIN-Bewegung in Z-Richtung korrigiert.

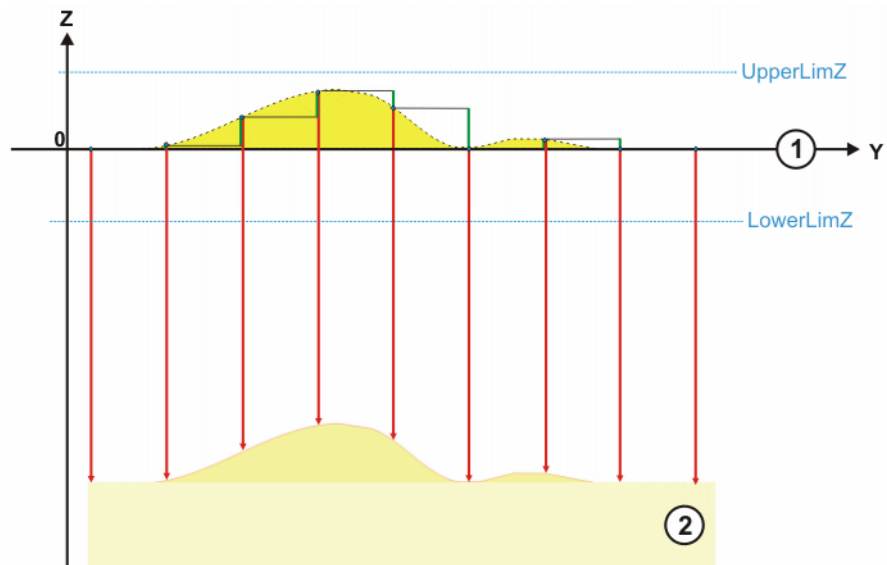


Abb. 9-5: Bahnkorrektur zur Abstandssteuerung

1 Bauteil

2 Programmierte Bahn

Programm

```

1 DEF RSI_DistanceCtrl( )
2 ; =====
3 ;
4 ; RSI EXAMPLE: Distance Ctrl
5 ; Move on a LIN path with superimposed corrections
6 ; Deviation from programmed path is controlled with
7 ; a analog input $ANIN[1]
8 ;
9 ; =====
10
11 ; Declaration of KRL variables
12 DECL INT ret          ; Return value for RSI commands
13 DECL INT CONTID       ; ContainerID
14
15 INI
16
17 ; Move to start position
18 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
19 $BASE=$POS_ACT
20
21 ; Create signal processing
22 ret=RSI_CREATE("RSI_DistanceCtrl")
23 IF (ret <> RSIOK) THEN
24   HALT
25 ENDIF
26
27 ; Start signal processing in relative correction mode
28 ret=RSI_ON(#RELATIVE)
29 IF (ret <> RSIOK) THEN
30   HALT
31 ENDIF
32
33 LIN_REL {Y 100

```

```

34
35 ; Turn off RSI
36 ret=RSI_OFF()
37 IF (ret <> RSIOK) THEN
38   HALT
39 ENDIF
40
41 END

```

Zeile	Beschreibung
18	Startpunkt der Sensorkorrektur
19	Position des BASE-Koordinatensystems im aktuellen TCP
22	RSI_CREATE() lädt den RSI-Kontext in einen RSI-Container.
28	RSI_ON() schaltet die Signalverarbeitung ein. Korrekturmodus: Relativkorrektur
33	Relative LIN-Bewegung in Y-Richtung (100 mm)
36	RSI_OFF() schaltet die Signalverarbeitung aus.

RSI-Kontext

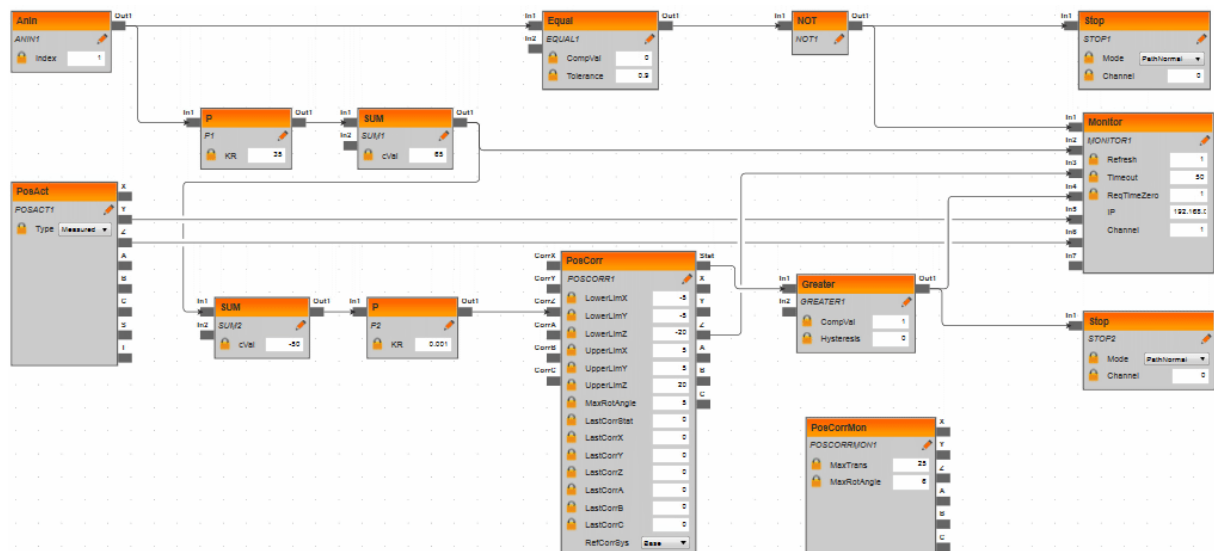


Abb. 9-6: Beispiel RSI_DistanceCtrl: Bahnkorrektur zur Abstandssteuerung

RSI-Objekt	Beschreibung
ANIN1	Liest über einen analogen Eingang das Sensorsignal ein.
EQUAL1	Mit EQUAL wird überprüft, ob das Sensorsignal innerhalb einer Toleranzgrenze liegt. Wenn dies nicht der Fall ist (NOT), bremst der Roboter bahntreu.
NOT1	
STOP1	
P1	Mit P1 wird das Sensorsignal umgerechnet, z. B. ergeben 5 V einen Abstand von 10 cm (= Istabstand). Der Istabstand (SUM1) wird mit dem Sollabstand (SUM2) addiert. Das Ergebnis ist der Korrekturwert in Z-Richtung in cm. Mit P2 wird der Korrekturwert in mm umgerechnet.
SUM1	
SUM2	
P2	
POSCORR1	Liest den berechneten Korrekturwert in Z-Richtung ein, der am Ausgang des Objekts P2 zur Verfügung steht. Referenzkoordinatensystem für die Korrekturaufschaltung: BASE

RSI-Objekt	Beschreibung
POSCORRMON1	Begrenzt die maximale kartesische Gesamtkorrektur. • Maximale translatorische Auslenkung in X, Y, Z: 25 mm • Maximale rotatorische Auslenkung der Drehwinkel: 6°
GREATER1	Der Korrekturstatus, der am Ausgang "Stat" des Korrekturobjekts POSCORR zur Verfügung steht, wird überprüft. Wenn der Korrekturstatus >1 ist, d. h. die zulässige Korrektur wurde überschritten und automatisch auf die Maximalkorrektur ± 20 mm begrenzt, bremst der Roboter bahntreu.
STOP2	
POSACT1	Liest die aktuelle kartesische Istposition des Roboters in Y- und Z-Richtung ein.
MONITOR1	Folgende Signale sind mit dem MONITOR-Objekt verknüpft und können mit RSI-Monitor auf der Robotersteuerung angezeigt werden: • Analoges eingelesenes Sensorsignal [V] • Berechneter Istabstand [cm] • Korrekturwert in Z-Richtung [mm] • Korrekturbegrenzung (true, false) • Kartesische Istposition des Roboters in Y- und Z-Richtung [mm]

9.6 Beispiel RSI_SigTransformation: Transformation in neues Koordinatensystem

Hier wird die Programmierung einer Transformation von Positionsdaten beschrieben, die von einem Sensor erfasst werden.

Am Anbauflansch des Roboters ist neben dem Werkzeug ein Sensor angebracht, der die Position eines Werkstücks erfasst, z. B. eine Kamera. Die Sensordaten müssen ausgehend vom Sensor-Koordinatensystem in das BASE-Koordinatensystem der Robotersteuerung transformiert werden.

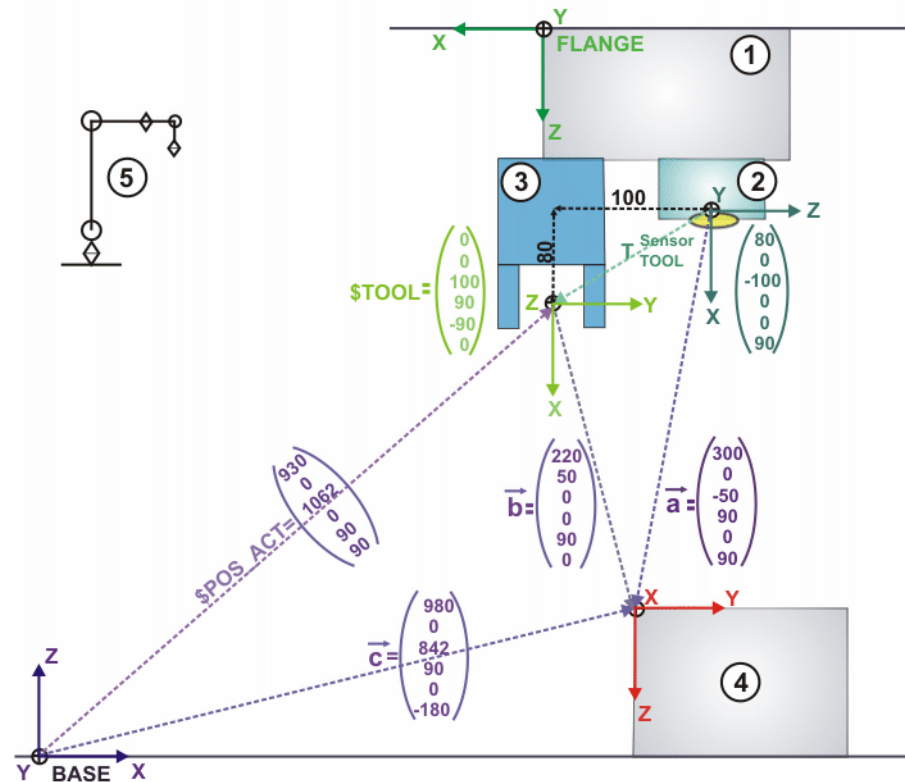


Abb. 9-7: Beispiel einer Transformation

- | | |
|----------------|-------------------|
| 1 Anbauflansch | 4 Werkstück |
| 2 Sensor | 5 Roboterstellung |
| 3 Werkzeug | |

Der Sensor erfasst die Position und die Orientierung eines Werkstücks im Sensor-Koordinatensystem (Vektor a). Im RSI-Objekt TRAFO_USERFRAME wird die Verschiebung und Verdrehung des Sensors zum Tool angegeben (T_Sensor/Tool). TRAFO_USERFRAME transformiert die Sensordaten in das TOOL-Koordinatensystem (Vektor b). Um die Sensordaten im BASE-Koordinatensystem zu erhalten, wird das RSI-Objekt TRAFO_ROBFRAME verwendet. TRAFO_ROBFRAME transformiert die Tool-Koordinaten in das BASE-Koordinatensystem (Vektor c).

Mit dem Beispielprogramm kann das in der Abbildung angegebene Zahlenbeispiel überprüft werden. Dazu muss ein KR 16 eingestellt sein. Wenn die mit dem MONITOR-Objekt verknüpften Signale mit RSI-Monitor auf der Robotersteuerung angezeigt werden, erhält man Position und Orientierung des Werkstücks im BASE-Koordinatensystem der Robotersteuerung (Vektor c).

Programm

```

1 DEF RSI_SigTransformation( )
2 ; =====
3 ;
4 ; RSI EXAMPLE: Transformation of coordinates
5 ; Simulate a sensorsignal in relationship to
6 ; a flange mounted sensor. Transform the SIGNAL
7 ; to $BASE coordinates. Show the transformed
8 ; position in RSIMONITOR
9 ; =====
10

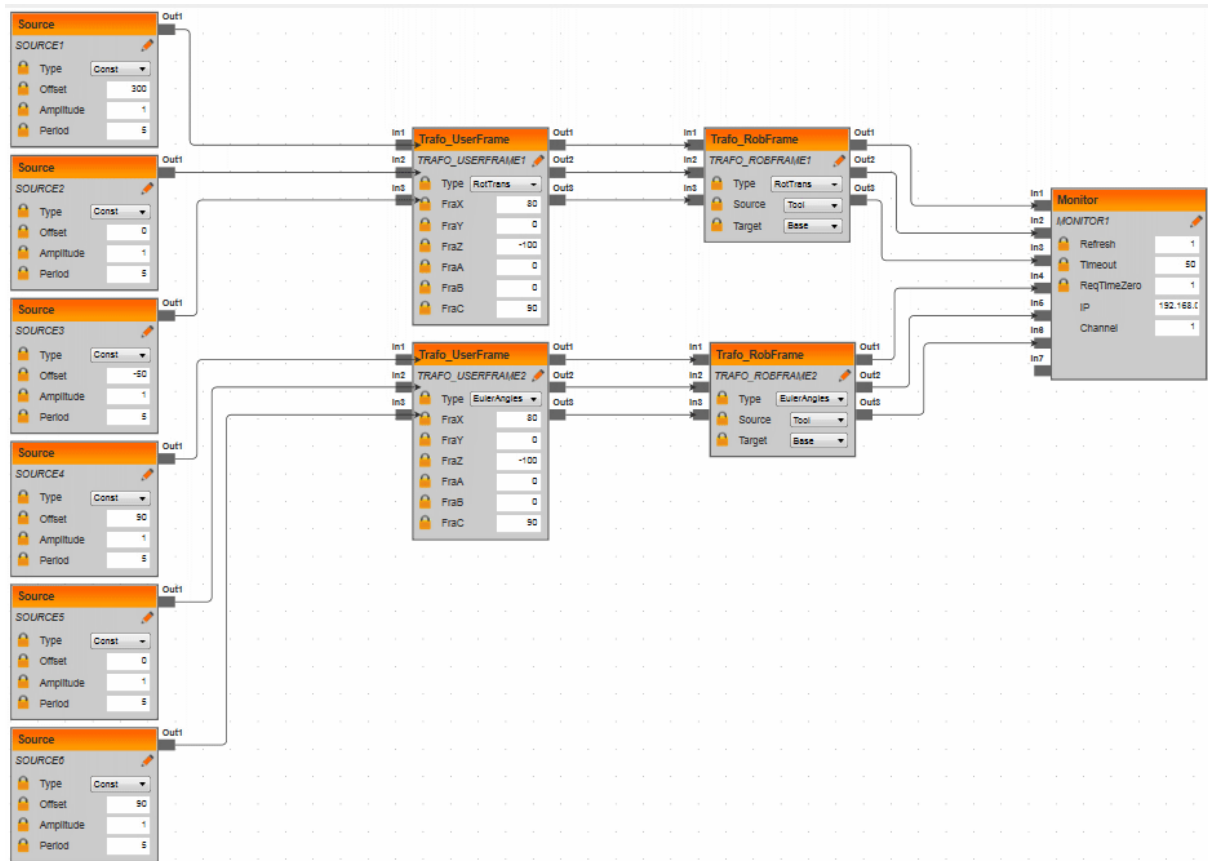
```

```

11 ; Declaration of KRL variables
12 DECL INT CONTID          ; ContainerID
13
14 INI
15
16 ; Move to start position
17 PTP {A1 0, A2 -90, A3 90, A4 0, A5 90, A6 0}
18 $TOOL = {X 0, Y 0, Z 100, A 90 ,B -90, C 0}
19 $BASE = $NULLFRAME
20
21 ; Create signal processing
22 IF (RSI_CREATE("RSI_SigTransformation") <> RSIOK) THEN
23   HALT
24 ENDIF
25
26 ; Start signal processing
27 IF (RSI_ON() <> RSIOK) THEN
28   HALT
29 ENDIF
30
31 wait sec 0.012
32
33 ; Turn off RSI
34 IF (RSI_OFF() <> RSIOK) THEN
35   HALT
36 ENDIF
37
38 END

```

Zeile	Beschreibung
17	Startposition der Transformation
18	Position des TOOL-Koordinatensystems
19	Position des BASE-Koordinatensystems (NULLFRAME)
22	RSI_CREATE() lädt den RSI-Kontext in einen RSI-Container.
27	RSI_ON() schaltet die Signalverarbeitung ein. Korrekturmodus: Relativkorrektur
31	In der Wartezeit werden die Transformationsdaten berechnet.
34	RSI_OFF() schaltet die Signalverarbeitung aus.

RSI-Kontext**Abb. 9-8: Beispiel RSI_SigTransformation: Transformation in neues Koordinatensystem**

RSI-Objekt	Beschreibung
SOURCE1 ... SOURCE3	Liefern die Position des Werkstücks im Sensor-Koordinatensystem (Vektor a) und übergeben die Daten an TRAFO_USERFRAME1.
TRAFO_USERFRAME1	Transformiert die Positionsdaten des Werkstücks im Sensor-Koordinatensystem in das TOOL-Koordinatensystem der Robotersteuerung (Vektor b). Die Daten stehen an den Ausgängen des Objekts zur Verfügung.
TRAFO_ROBFRAME1	Transformiert die Positionsdaten des Werkstücks im TOOL-Koordinatensystem in das BASE-Koordinatensystem der Robotersteuerung (Vektor c). Die Daten stehen an den Ausgängen des Objekts zur Verfügung.
SOURCE4 ... SOURCE6	Liefern die Orientierung des Werkstücks im Sensor-Koordinatensystem (Vektor a) und übergeben die Daten an TRAFO_USERFRAME2.
TRAFO_USERFRAME2	Transformiert die Orientierungswinkel des Werkstücks im Sensor-Koordinatensystem in das TOOL-Koordinatensystem der Robotersteuerung (Vektor b). Die Daten stehen an den Ausgängen des Objekts zur Verfügung.
TRAFO_ROBFRAME2	Transformiert die Orientierungswinkel des Werkstücks im TOOL-Koordinatensystem in das BASE-Koordinatensystem der Robotersteuerung (Vektor c). Die Daten stehen an den Ausgängen des Objekts zur Verfügung.

RSI-Objekt	Beschreibung
MONITOR1	Folgende Signale sind mit dem MONITOR-Objekt verknüpft und können mit RSI-Monitor auf der Robotersteuerung angezeigt werden: • Ergebnis der Transformation (Vektor c): Position und Orientierung des Werkstücks im BASE-Koordinatensystem der Robotersteuerung

10 Diagnose

10.1 RSI-Monitor anzeigen

Beschreibung

Der RSI-Monitor kann bis zu 24 Signale aus dem RSI-Kontext anzeigen und aufzeichnen.

(>>> 10.1.2 "Signalverlauf anzeigen und aufzeichnen" Seite 113)

Im RSI-Kontext wird hierzu das RSI-Objekt MONITOR verwendet. Im RSI-Kontext müssen die Signale, die angezeigt werden sollen, mit den Eingängen des MONITOR-Objekts verknüpft sein.

Vorgehensweise

- Im Hauptmenü **Anzeige** > **RobotSensorInterface** > **RSI Monitor** wählen.



Abb. 10-1: Übersicht Bedienoberfläche

Schaltflächen

Schaltfläche	Beschreibung
Setup	Die Signaleigenschaften für die Signalaufzeichnung können definiert werden. (>>> 10.1.1 "Signaleigenschaften definieren" Seite 112)
Datei	Der aufgezeichnete Signalverlauf kann in einer Datei gespeichert und wieder geladen werden. (>>> 10.1.3 "Signalaufzeichnung speichern" Seite 114) (>>> 10.1.4 "Signalaufzeichnung in den RSI-Monitor laden" Seite 114)

Schaltfläche	Beschreibung
Einstellung	Folgende Einstellungen können vorgenommen werden: • RSI Kanal Kanalnummer des MONITOR RSI-Objekts: – 1 ... 8 Default: 1 Relevant, wenn im RSI-Kontext mehrere MONITOR-Objekte verwendet werden • Verbindung halten – Häkchen bei Halten gesetzt: Die Signalaufzeichnung wird fortgesetzt, wenn der Monitor geschlossen wird. – Kein Häkchen bei Halten gesetzt: Die Signalaufzeichnung wird unterbrochen, wenn der Monitor geschlossen wird. Erforderliche Benutzerrechte: Funktionsgruppe Konfiguration der Techpakete
Zoom	Über einen Regler kann das angezeigte Zeitfenster vergrößert oder verkleinert werden. Das sichtbare Zeitfenster kann durch horizontales Schieben auf dem Monitor-Anzeigefenster verschoben werden.

10.1.1 Signaleigenschaften definieren

Beschreibung

Die Signaleigenschaften für die Signalaufzeichnung im RSI-Monitor können definiert werden.

Vorgehensweise

1. Im Hauptmenü **Anzeige** > **RobotSensorInterface** > **RSI Monitor** wählen.
2. Auf **Setup** drücken und die Signaleigenschaften definieren.



Abb. 10-2: Signaleigenschaften

Pos.	Beschreibung
1	Listenfeld mit den Signalen 1 ... 24
2	Linienstärke des Signals • Linienstärke1 ... 4 Default: Linienstärke 1
3	Signalfarbe des Signals
4	Ordinate, an dem das Signal skaliert wird • Links: Skalierung an der linken Ordinate • Rechts: Skalierung an der rechten Ordinate Default: Links
5	Die Checkbox muss aktiviert sein (Häkchen gesetzt), um ein Signal im Monitor anzuzeigen. Default: Checkbox für Signal 1 ... 6 aktiviert.

Schaltflächen

Schaltfläche	Beschreibung
Alle aktivieren	Aktiviert alle Signale.
Alle deaktivieren	Deaktiviert alle Signale.
Stärke zurücksetzen	Setzt die Linienstärken der Signale auf die voreingestellte Linienstärke 1 zurück.
Farben zurücksetzen	Setzt die Signalfarben auf die voreingestellten Farben zurück.
Zeichnungsfläche zurücksetzen	Setzt das Zeitfenster auf die voreingestellte Größe zurück (Zoom aufheben)
Alles Zurücksetzen	Setzt alle Signaleigenschaften auf die Voreinstellung zurück.

10.1.2 Signalverlauf anzeigen und aufzeichnen

Beschreibung

Jedes MONITOR-Objekt verwendet einen eigenen Kanal zum RSI-Monitor. Wenn im RSI-Kontext mehrere MONITOR-Objekte verwendet werden, muss die Kanalnummer des gewünschten MONITOR-Objekts für die Signalaufzeichnung eingestellt werden. Der RSI-Monitor zeigt nur die über den eingestellten Kanal empfangenen Signale an.

Voraussetzung

- Zum Einstellen der Kanalnummer des MONITOR-Objekts:
Benutzerrechte der Funktionsgruppe **Konfiguration der Techpakete**
- IP-Adresse im MONITOR-Objekt: 192.168.0.1

Vorgehensweise

1. Im Hauptmenü **Anzeige** > **RobotSensorInterface** > **RSI Monitor** wählen.
2. Auf **Setup** drücken und die Signaleigenschaften für die Aufzeichnung definieren.
(>>> **10.1.1 "Signaleigenschaften definieren" Seite 112**)

3. Bei Bedarf auf **Einstellung** drücken, um die Kanalnummer des MONITOR-Objekts einzustellen.
4. Programm anwählen und abfahren.
Die Aufzeichnung startet mit dem Einschalten der Signalverarbeitung und endet mit dem Ausschalten der Signalverarbeitung.



Eine Signalaufzeichnung wird im RSI-Monitor erst gelöscht, wenn im KRL-Programm ein MONITOR-Objekt neu angelegt wird. Nach dem Zurücksetzen des Programms oder dem Löschen der Signalverarbeitung bleibt die Signalaufzeichnung im RSI-Monitor stehen.

10.1.3 Signalaufzeichnung speichern

Beschreibung

Ein mit dem RSI-Monitor aufgezeichneter Signalverlauf kann in folgendem Verzeichnis gespeichert werden:

- C:\KRC\ROBOTER\LOG\SensorInterface\MONITOR

Voraussetzung

- Benutzerrechte: Funktionsgruppe **Konfiguration der Techpakete**

Vorgehensweise

1. Im Hauptmenü **Anzeige** > **RobotSensorInterface** > **RSI Monitor** wählen.
2. In der Checkbox **Datei** das Häkchen setzen.
3. Im Feld **Datei speichern** einen Dateinamen für die Aufzeichnung vergeben und auf **Speichern** drücken.

10.1.4 Signalaufzeichnung in den RSI-Monitor laden

Beschreibung

Alle Signalaufzeichnungen, die im Verzeichnis C:\KRC\ROBOTER\LOG\SensorInterface\MONITOR gespeichert sind, können wieder in den RSI-Monitor geladen werden.

Vorgehensweise

1. Im Hauptmenü **Anzeige** > **RobotSensorInterface** > **RSI Monitor** wählen.
2. In der Checkbox **Datei** das Häkchen setzen.
3. Im Feld **Datei laden** die gewünschte Datei auswählen und auf **Laden** drücken.

10.2 Diagnosedaten anzeigen

Voraussetzung

- Benutzerrechte: Funktionsgruppe **Diagnose-Funktionen**

Vorgehensweise

1. Im Hauptmenü **Diagnose** > **Diagnosemonitor** wählen.
2. Im Feld **Modul** das Modul **Robot Sensor Interface (RSI2)** auswählen.

Beschreibung

Diagnosedaten zur RSI:

Name	Beschreibung
Status	Status der Signalverarbeitung • Aktiv IPO: Signalverarbeitung im Sensormodus #IPO • Aktiv IPO_FAST: Signalverarbeitung im Sensormodus #IPO_FAST • Inaktiv: Keine Signalverarbeitung
Sensor Takt	Zykluszeit der Signalverarbeitung
Zähler	Anzahl der Berechnungszyklen seit dem Start der Signalverarbeitung
Ausführungszeit	Zeit, die zur Berechnung des aktuellen RSI-Kontextes benötigt wird
Ausführungszeit (Min)	Minimale Zeit zur Berechnung des aktuellen RSI-Kontextes
Ausführungszeit (Max)	Maximale Zeit zur Berechnung des aktuellen RSI-Kontextes
Ausführungszeit (Mittel)	Durchschnittliche Zeit zur Berechnung des aktuellen RSI-Kontextes
Objekt Zähler	Anzahl der angelegten RSI-Objekte
Speicher	Ingesamt verfügbarer Speicher für RSI (Byte)
Benutzter Speicher	Benutzer Speicher (Byte)
Kommunikationszyklen	Erfolgreiche Kommunikationszyklen seit dem Start der Signalverarbeitung
Gesamtverlust	Anzahl der Paketverluste seit dem Start der Signalverarbeitung
Verbindungsqualität	Qualität der Signalverarbeitung • 0 ... 100 % 100 % = Alle Pakete sind erfolgreich eingetroffen. 0 % = Kein Paket ist erfolgreich eingetroffen.
Maximal zusammenhängender Verlust	Größter zusammenhängender Verlust von Paketen seit Beginn der Signalverarbeitung

10.3 Fehlerprotokoll (LOG-Datei)

Die Fehlermeldungen des RSI werden standardmäßig in einer LOG-Datei unter C:\KRC\ROBOTER\LOG\SensorInterface protokolliert.

10.3.1 LOG-Level konfigurieren

Beschreibung

Standardmäßig werden nur Fehlermeldungen in die RSI-LOG-Datei geschrieben. Der LOG-Level kann so geändert werden, dass zusätzlich Hinweismeldungen protokolliert werden.

LOG-Level	Beschreibung
error	Fehlermeldungen der Schnittstelle werden protokolliert.
info	Fehlermeldungen und Hinweismeldungen der Schnittstelle werden protokolliert.

Konfigurationsdatei

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common
Datei	Logging_RSI.xml

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.

Vorgehensweise

1. Konfigurationsdatei Logging_RSI.xml öffnen.
2. In dieser Zeile den LOG-Level ändern:

```
<Class Name="RSILogger1" LogLevel="error" />
```

3. Änderung speichern.

11 Meldungen

11.1 Informationen zu den Meldungen

Das Kapitel "Meldungen" enthält ausgewählte Meldungen. Es behandelt nicht alle Meldungen, die im Meldungsfenster angezeigt werden.

11.2 Systemmeldungen aus Modul: CrossMeld (KSS)

11.2.1 KSS29000

Meldungscode	KSS29000
Meldungstext	{Type} Zulässige Gesamtkorrektur überschritten: RSI wird angehalten
Meldungstyp	Quittiermeldung
	
Auswirkung	Drehzahlstopp Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	Ursache: Zulässige Korrektur durch RSI-Korrekturobjekt zu hoch (>>> Seite 117) Lösung: Zulässige Korrektur in RSI-Korrekturobjekt weiter eingrenzen (>>> Seite 118) Ursache: Zulässige Gesamtkorrektur im RSI-Kontext zu niedrig (>>> Seite 118) Lösung: Zulässige Gesamtkorrektur erhöhen (>>> Seite 118)

Ursache: Zulässige Korrektur durch RSI-Korrekturobjekt zu hoch

Beschreibung

Die maximal zulässige kartesische oder achsspezifische Gesamtkorrektur wird mit den RSI-Objekten POSCORRMON oder AXISCORRMON festgelegt. Die an den RSI-Kontext übergebenen Korrekturvorgaben des Sensors haben die maximal zulässige Gesamtkorrektur überschritten.

Dies kann daran liegen, dass in den RSI-Korrekturobjekten zu hohe Korrekturen zugelassen werden:

- AXISCORR
- POSCORR
- AXISCORREXT

Prüfanweisung

1. RSI-Kontext mit RSIVisual öffnen und prüfen, auf welche Werte die Korrektur in den RSI-Objekten AXISCORR, POSCORR oder AXISCORREXT begrenzt wird (LowerLim/UpperLim).
2. Prüfen, ob die Applikation es zulässt, die in den RSI-Objekten eingestellten Werte weiter einzugrenzen.



Informationen zu den einzelnen RSI-Objekten und ihren Objektparametern sind in der Hilfe von RSIVisual zu finden.

Lösung: Zulässige Korrektur in RSI-Korrekturobjekt weiter eingrenzen

Beschreibung

Die zulässige Korrektur im RSI-Korrekturobjekt weiter eingrenzen, wenn die Applikation dies zulässt, sodass die Maximalkorrektur nicht mehr überschritten wird.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Vorgehensweise

1. RSI-Kontext mit RSIVisual öffnen und wie benötigt ändern.
2. RSI-Kontext speichern und auf Robotersteuerung übertragen.

Ursache: Zulässige Gesamtkorrektur im RSI-Kontext zu niedrig

Beschreibung

Die maximal zulässige kartesische oder achsspezifische Gesamtkorrektur wird mit den RSI-Objekten POSCORRMON oder AXISCORRMON festgelegt. Die an den RSI-Kontext übergebenen Korrekturvorgaben des Sensors haben die maximal zulässige Gesamtkorrektur überschritten.

Prüfanweisung

1. RSI-Kontext mit RSIVisual öffnen und prüfen, welche Werte für die maximal zulässige Gesamtkorrektur in den RSI-Objekten POSCORRMON oder AXISCORRMON eingestellt sind.
2. Prüfen, ob die in den RSI-Objekten eingestellten Werte für die Applikation ausreichend sind.



Informationen zu den einzelnen RSI-Objekten und ihren Objektparametern sind in der Hilfe von RSIVisual zu finden.

Lösung: Zulässige Gesamtkorrektur erhöhen

Beschreibung

Die zulässige Gesamtkorrektur im RSI-Objekt so weit erhöhen, wie die Applikation es zulässt.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Vorgehensweise

1. RSI-Kontext mit RSIVisual öffnen und wie benötigt ändern.
2. RSI-Kontext speichern und auf Robotersteuerung übertragen.

11.2.2 KSS29001

Meldungscode	KSS29001
Meldungstext	{Typ} Korrektur außerhalb des zulässigen Bereichs: {Wert}
Meldungstyp	Hinweismeldung 
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: Korrekturvorgabe des Sensors überschreitet zulässigen Bereich (>>> Seite 119) Lösung: Systeminformation: keine Bedienaktion notwendig (>>> Seite 119)

Ursache: Korrekturvorgabe des Sensors überschreitet zulässigen Bereich

Beschreibung

Die kommandierte Korrekturvorgabe des Sensors überschreitet den für das Korrekturobjekt (POSCORR, AXISCORR oder AXISCORREXT) definierten maximal zulässigen Bereich. Die Korrektur wird auf den zulässigen Bereich eingeschränkt.

Lösung: Systeminformation: keine Bedienaktion notwendig

Beschreibung

Diese Meldung ist eine Systeminformation für den Bediener und erfordert keine Aktion.

11.2.3 KSS29002

Meldungscode	KSS29002
Meldungstext	Signalfluss ({Mode}): Objekt {ObjName} liefert Fehler {ErrorCode}
Meldungstyp	Quittiermeldung 
Auswirkung	Bahntreuer NOT-HALT Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	Ursache: RSI-Objekt falsch konfiguriert (>>> Seite 120) Lösung: RSI-Kontext ändern (>>> Seite 120)

Ursache: RSI-Objekt falsch konfiguriert**Beschreibung**

Das in der Meldung angegebene RSI-Objekt liefert einen Fehler und konnte nicht korrekt abgearbeitet werden.

Prüfanweisung

1. RSI-Kontext mit RSIVisual öffnen.
2. RSI-Objekt prüfen, das den Fehler ausgelöst hat, sowie die Eingangsdaten, die am RSI-Objekt anliegen.



Informationen zu den einzelnen RSI-Objekten und ihren Objektparametern sind in der Hilfe von RSIVisual zu finden.

Lösung: RSI-Kontext ändern**Voraussetzung**

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Vorgehensweise

1. RSI-Kontext mit RSIVisual öffnen und wie benötigt ändern.
2. RSI-Kontext speichern und auf Robotersteuerung übertragen.

11.2.4 KSS29004

Meldungscode	KSS29004
Meldungstext	Interner RSI Fehler
Meldungstyp	Zustandsmeldung
	
Auswirkung	Bahntreuer NOT-HALT Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	Ursache: Interner Fehler beim Ausführen von RSI (>>> Seite 120) Lösung: Programm zurücksetzen oder RSI-LOG-Datei an KUKA Service (>>> Seite 121)

Ursache: Interner Fehler beim Ausführen von RSI**Beschreibung**

Beim Ausführen von RSI ist ein interner Fehler aufgetreten.

Alle RSI-Fehlermeldungen werden in einer LOG-Datei protokolliert.

LOG-Datei

Verzeichnis	C:\KRC\ROBOTER\LOG\SensorInterface\
Datei	RSIyyyy-MM-dd_HH-mm-ss.log LOG-Dateien werden mit Angabe von Datum und Uhrzeit gespeichert.

Lösung: Programm zurücksetzen oder RSI-LOG-Datei an KUKA Service**Beschreibung**

Wenn der Fehler nach dem Zurücksetzen des Programms weiterhin auftritt, Kontakt mit dem zuständigen KUKA Customer Support aufnehmen. Der Customer Support benötigt die passende RSI-LOG-Datei.

Voraussetzung

- Programm ist angewählt.

Vorgehensweise

- Menüfolge **Bearbeiten > Programm zurücksetzen** wählen.

Alternative Vorgehensweise

- In der Statusleiste die Statusanzeige **Roboter-Interpreter** berühren. Ein Fenster öffnet sich.
Programm zurücksetzen wählen.

11.2.5 KSS29006

Meldungscode	KSS29006
Meldungstext	RSI: Zeitüberschreitung der Signalberechnung {CalcTime} usec
Meldungstyp	Quittiermeldung 
Auswirkung	Rampenstopp Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	Ursache: RSI-Kontext ist zu umfangreich (>>> Seite 122) Lösung: Nicht benötigte Objekte aus RSI-Kontext entfernen (>>> Seite 122) Ursache: RSI-Kontext doppelt gestartet (>>> Seite 122) Lösung: RSI-Kontext vor Neustart zunächst beenden (>>> Seite 122)

Ursache: RSI-Kontext ist zu umfangreich**Beschreibung**

Der RSI-Kontext ist zu umfangreich und verbraucht zu viel Rechenzeit. Die Signalberechnung konnte nicht innerhalb eines Sensortakts abgeschlossen werden.

Prüfanweisung

- RSI-Kontext mit RSIVisual öffnen und prüfen, ob der RSI-Kontext nicht benötigte RSI-Objekte enthält.

Lösung: Nicht benötigte Objekte aus RSI-Kontext entfernen**Beschreibung**

Nicht benötigte RSI-Objekte können aus dem RSI-Kontext entfernt werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2

Vorgehensweise

1. RSI-Kontext mit RSIVisual öffnen und wie benötigt ändern.
2. RSI-Kontext speichern und auf Robotersteuerung übertragen.

Ursache: RSI-Kontext doppelt gestartet**Beschreibung**

Der RSI-Kontext wird mit den RSI-Funktionen `RSI_CREATE()` und `RSI_ON()` gestartet, obwohl er bereits einmal gestartet und dazwischen nicht beendet wurde.

So kann man prüfen, ob der RSI-Kontext doppelt gestartet wurde:

Prüfanweisung

- Prüfe, ob im Programmcode `RSI_CREATE()` und `RSI_ON()` doppelt aufgerufen werden, ohne dass dazwischen `RSI_RESET()` und `RSI_OFF()` aufgerufen werden.

Lösung: RSI-Kontext vor Neustart zunächst beenden**Beschreibung**

Vor einem Neustart des RSI-Kontexts mit der RSI-Funktion `RSI_CREATE()` und `RSI_ON()` den RSI-Kontext zuerst mit `RSI_RESET()` und `RSI_OFF()` beenden.

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.

3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

11.2.6 KSS29007

Meldungscode	KSS29007
Meldungstext	Fehler beim Aktualisieren von RSI-Signal {Signalname}
Meldungstyp	Quittiermeldung
	
Auswirkung	Drehzahlstopp Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	Ursache: Verbindung zum Feldbus unterbrochen (>>> Seite 123) Lösung: Feldbusverbindung wiederherstellen und E/A-Treiber rekonfigurieren (>>> Seite 124) Ursache: Verbindung zum Feldbus unterbrochen (>>> Seite 124) Lösung: Korrekte Verbindung herstellen (>>> Seite 125)

Ursache: Verbindung zum Feldbus unterbrochen

Beschreibung

Die Verbindung zum Feldbus ist unterbrochen.

So kann man prüfen, ob eine Verbindungsleitung defekt ist:

Prüfanweisung

1. Prüfe, ob die Stecker korrekt angeschlossen sind. Dabei ist besonders zu achten auf:
 - eingedrückte Pins
 - Korrosion
 - verschmorte Kontakte
 - zurückgedrückter Steckereinsatz
 - zurückgedrückte Buchse
 - Stecker auf richtigem Steckplatz
2. Prüfe, ob die Leitung mechanisch beschädigt ist. Ursachen für gequetschte Leitungen oder Adern können sein:
 - zu feste Kabelbinder
 - zu feste Schellen
 - einklemmen beim Schließen einer Abdeckung
 - zu enger Biegeradius
3. Prüfe, ob die elektrische Funktion der Leitung gegeben ist. Dabei ist besonders zu achten auf:
 - Querschluß einzelner Adern gegeneinander
 - Kurzschluß einzelner Adern gegen Masse
 - Korrekte Verdrahtung nach Schaltplan

Lösung: Feldbusverbindung wiederherstellen und E/A-Treiber rekonfigurieren

Beschreibung

Nachdem die Feldbusverbindung wiederhergestellt worden ist, muss der E/A-Treiber rekonfiguriert werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.

Vorgehensweise

1. Im Hauptmenü **Konfiguration** > **Ein-/Ausgänge** > **E/A Treiber** wählen.
2. Die Schaltfläche **Rekonfigurieren** berühren.
Es spielt keine Rolle, ob man sich dazu in der Registerkarte **Zustand** oder **Konfiguration** befindet.
3. Die Sicherheitsabfrage *Wollen Sie wirklich alle E/A-Treiber rekonfigurieren?* mit **Ja** beantworten.
Die Meldung *Rekonfiguration wird durchgeführt ...* wird angezeigt.
Wenn sich die Meldung wieder ausblendet, ist der Rekonfiguration abgeschlossen.

Ursache: Verbindung zum Feldbus unterbrochen

Beschreibung

Die Verbindung zum Feldbus ist unterbrochen.

So kann man prüfen, ob eine Verbindungsleitung defekt ist:

Prüfanweisung

1. Prüfe, ob die Stecker korrekt angeschlossen sind. Dabei ist besonders zu achten auf:
 - eingedrückte Pins
 - Korrosion
 - verschmorte Kontakte
 - zurückgedrückter Steckereinsatz
 - zurückgedrückte Buchse
 - Stecker auf richtigem Steckplatz
2. Prüfe, ob die Leitung mechanisch beschädigt ist. Ursachen für gequetschte Leitungen oder Adern können sein:
 - zu feste Kabelbinder
 - zu feste Schellen
 - einklemmen beim Schließen einer Abdeckung
 - zu enger Biegeradius
3. Prüfe, ob die elektrische Funktion der Leitung gegeben ist. Dabei ist besonders zu achten auf:
 - Querschluss einzelner Adern gegeneinander
 - Kurzschluss einzelner Adern gegen Masse
 - Korrekte Verdrahtung nach Schaltplan

Lösung: Korrekte Verbindung herstellen

Voraussetzung

- Robotersteuerung muss ausgeschaltet und gegen unbefugtes Wiedereinschalten gesichert sein.
- Netzzuleitung ist spannungsfrei geschaltet.

Arbeitssicherheit



WARNUNG

Lebensgefahr durch anliegende Netzspannung

Leitungen, die vom Netzanschluss X1 zum Hauptschalter geführt werden, stehen auch im ausgeschalteten Zustand unter Spannung. Tod, schwere Verletzungen oder Sachschäden können die Folge sein.

- Netzzuleitung vor Beginn der Arbeiten spannungsfrei schalten.
- Spannungsfreiheit vor Beginn der Arbeiten sicherstellen.

HINWEIS

Bauteilschädigung oder -zerstörung durch elektrostatische Entladung (ESD)

Elektrostatische Entladungen bei Ein- und Ausbauarbeiten können zur Zerstörung oder Teilschädigung von Elektronikbauteilen führen.

- Nach ESD-Richtlinien arbeiten.

Vorgehensweise

- Die defekte Leitung tauschen oder Kontaktproblem beseitigen.

11.2.7 KSS29008

Meldungscode	KSS29008
Meldungstext	RSI I/Os Konfigurationsdatei nicht gefunden.
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: RSI-E/A-Konfigurationsdatei nicht gefunden (>>> Seite 125) Lösung: WorkVisual-Projekt neu aufspielen (>>> Seite 126)

Ursache: RSI-E/A-Konfigurationsdatei nicht gefunden

Beschreibung

Die Konfigurationsdatei mit den RSI-E/As konnte nicht gefunden werden.

Beim Übertragen des WorkVisual-Projekts mit den verschalteten RSI-E/As auf die Robotersteuerung wird die Konfigurationsdatei korrekt erstellt.

Konfigurationsdatei

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\
Datei	RSIIIO.xml

So kann man prüfen, ob die Datei vorhanden ist:

Voraussetzung

- Benutzergruppe Experte

Prüfanweisung

1. Im Navigator in das Verzeichnis navigieren, in dem sich die Datei befinden soll.
2. Prüfen, ob die Datei im Verzeichnis vorhanden ist.

Lösung: WorkVisual-Projekt neu aufspielen**Beschreibung**

Das WorkVisual-Projekt muss neu auf die Robotersteuerung übertragen werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Netzwerkverbindung zur Robotersteuerung

Vorgehensweise

1. Das aktive Projekt von der Robotersteuerung laden.
2. Projekt von WorkVisual zurück auf die Robotersteuerung übertragen und aktivieren.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

11.2.8 KSS29009

Meldungscode	KSS29009
Meldungstext	Konfigurationsdatei der RSI I/Os ist ungültig.
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: RSI-E/A-Konfigurationsdatei ungültig (>>> Seite 127) Lösung: WorkVisual-Projekt neu aufspielen (>>> Seite 127)

Ursache: RSI-E/A-Konfigurationsdatei ungültig**Beschreibung**

Die Konfigurationsdatei mit den RSI-E/As ist ungültig.

Beim Übertragen des WorkVisual-Projekts mit den verschalteten RSI-E/As auf die Robotersteuerung wird die Konfigurationsdatei korrekt erstellt.

Konfigurationsdatei

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\
Datei	RSIIIO.xml

Lösung: WorkVisual-Projekt neu aufspielen**Beschreibung**

Das WorkVisual-Projekt muss neu auf die Robotersteuerung übertragen werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Netzwerkverbindung zur Robotersteuerung

Vorgehensweise

1. Das aktive Projekt von der Robotersteuerung laden.
2. Projekt von WorkVisual zurück auf die Robotersteuerung übertragen und aktivieren.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

11.2.9 KSS29010

Meldungscode	KSS29010
Meldungstext	Falsche Versionsnummer der RSI I/O Konfigurationsdatei.
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: Versionsnummer der RSI-E/A-Konfigurationsdatei ungültig (>>> Seite 128) Lösung: WorkVisual-Projekt neu aufspielen (>>> Seite 128)

Ursache: Versionsnummer der RSI-E/A-Konfigurationsdatei ungültig**Beschreibung**

Die Konfigurationsdatei mit den RSI-E/As besitzt eine ungültige Versionsnummer.

Beim Übertragen des WorkVisual-Projekts mit den verschalteten RSI-E/As auf die Robotersteuerung wird die Konfigurationsdatei korrekt erstellt.

Konfigurationsdatei

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\
Datei	RSIIIO.xml

Lösung: WorkVisual-Projekt neu aufspielen**Beschreibung**

Das WorkVisual-Projekt muss neu auf die Robotersteuerung übertragen werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Netzwerkverbindung zur Robotersteuerung

Vorgehensweise

1. Das aktive Projekt von der Robotersteuerung laden.
2. Projekt von WorkVisual zurück auf die Robotersteuerung übertragen und aktivieren.



Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu KUKA.WorkVisual zu finden.

11.2.10 KSS29011

Meldungscode	KSS29011
Meldungstext	Konfigurierter Bus wurde nicht gefunden.
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: Konfigurierter Feldbus nicht auf Robotersteuerung vorhanden oder installiert (>>> Seite 129) Lösung: Feldbus im WorkVisual-Projekt konfigurieren und korrekt verschalten (>>> Seite 129)

Ursache: Konfigurierter Feldbus nicht auf Robotersteuerung vorhanden oder installiert**Beschreibung**

Der konfigurierte Feldbus ist nicht auf der Robotersteuerung vorhanden oder installiert.

Lösung: Feldbus im WorkVisual-Projekt konfigurieren und korrekt verschalten**Beschreibung**

Feldbus gemäß Anleitung in der Dokumentation des Optionspakets KUKA.RobotSensorInterface konfigurieren und korrekt mit den RSI-E/As verschalten.

Vorgehensweise

1. Feldbus im WorkVisual-Projekt konfigurieren.
2. RSI-E/As mit den Feldbus-E/As verschalten.
3. WorkVisual-Projekt auf Robotersteuerung übertragen.

11.2.11 KSS29012

Meldungscode	KSS29012
Meldungstext	Fehler bei der Initialisierung der RSI I/Os.
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: Initialisierung der RSI-E/As fehlgeschlagen (>>> Seite 129) Lösung: KUKA Support kontaktieren, wenn Fehlerbehebung nicht möglich (>>> Seite 130)

Ursache: Initialisierung der RSI-E/As fehlgeschlagen**Beschreibung**

Die Initialisierung der RSI-E/As ist fehlgeschlagen.

LOG-Datei

Die Fehlermeldungen des RSI werden unter C:\KRC\ROBOTER\LOG\SensorInterface protokolliert.

Prüfanweisung

- LOG prüfen, um die Fehlerquelle zu lokalisieren.

Lösung: KUKA Support kontaktieren, wenn Fehlerbehebung nicht möglich

Beschreibung

Wenn der Fehler nicht behoben werden kann, zur Fehlerbehebung den zuständigen KUKA Support kontaktieren.

Vorgehensweise

- Support-Anfrage per E-Mail an lokale Niederlassung senden.

11.2.12 KSS29013

Meldungscode	KSS29013
Meldungstext	RSI kann nicht mit Feldbus kommunizieren: {Feldbusinstanz}
Meldungstyp	Zustandsmeldung
	
Auswirkung	Keine Bremsreaktion Keine Verriegelung von Bewegungen oder Kommandos
Mögliche Ursache(n)	Ursache: Mit RSI verbundener Feldbus meldet einen Fehler (>>> Seite 130) Lösung: Fehlerquelle lokalisieren und Feldbusfehler beheben (>>> Seite 130)

Ursache: Mit RSI verbundener Feldbus meldet einen Fehler

Beschreibung

Der mit RSI verbundene Feldbus meldet einen Fehler. Zur Fehlersuche und -diagnose kann die Feldbusdokumentation zu Rate gezogen werden.

Prüfanweisung

- Prüfen, ob zusätzlich zu dieser Meldung Fehlermeldungen des Feldbusses angezeigt werden.

Lösung: Fehlerquelle lokalisieren und Feldbusfehler beheben

Beschreibung

Nach Lokalisierung der Fehlerquelle den Feldbusfehler entsprechend beheben.

12 Anhang

12.1 Speicher erhöhen



Der Speicher darf nur nach Rücksprache mit der KUKA Deutschland GmbH erhöht werden. (>>> [13 "KUKA Service" Seite 137](#))

Beschreibung

Wenn der zur Verfügung stehende Speicher nicht ausreicht, wird empfohlen, zuerst die Programmierweise in KRL und die konfigurierte Signalverarbeitung im RSI-Kontext zu überprüfen.

Voraussetzung

- Windows-Ebene

Vorgehensweise

1. Datei C:\KRC\ROBOTER\Config\User\Common\RSI.XML öffnen.
2. Im Abschnitt <Interface> im Element <MemSize> die gewünschte Speicherkapazität in Byte eintragen.

```
<Interface>
  <MemSize>500000</MemSize>
</Interface>
```

3. Änderung speichern und Datei schließen.

12.2 RSI-Objektbibliothek

12.2.1 RSI-Objekte zur Kommunikation

Typ	Beschreibung
AnIn	Liefert den Wert eines analogen Eingangs \$ANIN.
AnOut	Liefert den Wert eines analogen Ausgangs \$ANOUT.
DigIn	Liefert den Wert eines Bereichs von digitalen Eingängen \$IN.
DigOut	Liefert den Wert eines Bereichs von digitalen Ausgängen \$OUT
Ethernet	UDP-Ethernet-Kommunikation im XML-Datenformat Bis zu 64 Ein-/Ausgänge können in der Konfigurationsdatei definiert werden. Signale an den Eingängen werden an den Kommunikationspartner gesendet. An den Ausgängen stehen die vom Kommunikationspartner empfangenen Daten zur Verfügung.
Input	Liefert den Wert eines RSI-Eingangs oder eines Bereichs von RSI-Eingängen.
Map2AnOut	Beschreibt einen analogen Ausgang \$ANOUT.
Map2DigOut	Beschreibt einen digitalen Ausgang \$OUT oder einen Bereich von digitalen Ausgängen.
Map2Sen_PInt	Ändert den Wert der Systemvariablen \$SEN_PINT.
Map2Sen_PRea	Ändert den Wert der Systemvariablen \$SEN_PREA.

Typ	Beschreibung
Monitor	RSI-Monitor Visualisierung von bis zu 24 RSI-Signalen
Output	Beschreibt einen RSI-Ausgang oder einen Bereich von RSI-Ausgängen.
ResetDigOut	Setzt bei positiver Flanke einen digitalen Ausgang \$OUT zurück. Der zurückgesetzte Ausgang wird auch bei negativer Flanke gehalten.
Sen_PInt	Liefert den Wert der Systemvariablen \$SEN_PINT.
Sen_PRea	Liefert den Wert der Systemvariablen \$SEN_PREA.
SetDigOut	Setzt bei positiver Flanke einen digitalen Ausgang \$OUT. Der gesetzte Ausgang wird auch bei negativer Flanke gehalten.

12.2.2 RSI-Objekte für logische Verknüpfungen

Typ	Beschreibung
AND	UND-Verknüpfung Es können bis zu 10 Eingangssignale angeschlossen werden.
BAND	Binäre UND-Verknüpfung Verknüpft den Signaleingang 1 mit einem konstanten Wert. Sind mehrere Signaleingänge verlinkt, werden diese miteinander verknüpft. Es können bis zu 10 Eingangssignale angeschlossen werden.
BCOMPL	Binäres Komplement
BOR	Binäre ODER-Verknüpfung Verknüpft den Signaleingang 1 mit einem konstanten Wert. Sind mehrere Signaleingänge verlinkt, werden diese miteinander verknüpft. Es können bis zu 10 Eingangssignale angeschlossen werden.
NOT	Logische Negierung
OR	ODER-Verknüpfung Es können bis zu 10 Eingangssignale angeschlossen werden.
RS FlipFlop	RS-Flip-Flop Rücksetzdominantes Flip-Flop
SignalSwitch	Umschalten zwischen 2 Signalpfaden mittels Steuersignal
SR FlipFlop	SR-Flip-Flop Setzdominantes Flip-Flop
XOR	Exklusive ODER-Verknüpfung Es können bis zu 10 Eingangssignale angeschlossen werden.

12.2.3 RSI-Objekte für mathematische Operationen

Typ	Beschreibung
ABS	Betragfunktion
ACOS	Arkuskosinusfunktion
ASIN	Arkussinusfunktion

Typ	Beschreibung
ATAN	Arkustangensfunktion
ATAN2	Arkustangens des Quotienten von Eingang 1 und 2 Der Quadrant des Ergebnisses wird aus den Vorzeichen der Eingangssignale ermittelt.
CEIL	Kleinste Integer-Zahl größer oder gleich Eingangssignal
COS	Kosinusfunktion
EXP	Exponentialfunktion
FLOOR	Größte Integer-Zahl größer oder gleich Eingangssignal
Limit	Beschränkt ein Signal auf Werte innerhalb einer unteren und oberen Grenze (LowerLimit, UpperLimit).
LOG	Logarithmusfunktion
MinMax	Liefert das aktuell kleinste und größte Signal über alle Eingangssignale. Es können bis zu 10 Eingangssignale angeschlossen werden.
MULTI	Multiplikation von Signalen
NORM	Betrag des Vektors aus bis zu 10 Signaleingängen
POW	Potenzfunktion
ROUND	Rundungsfunktion
SIN	Sinusfunktion
SUB	Subtraktion von Signalen Es können bis zu 5 Eingangssignale angeschlossen werden. Mit dem Parameter Value kann ein konstanter Wert subtrahiert werden.
SUM	Addition von Signalen Es können bis zu 5 Eingangssignale angeschlossen werden. Mit dem Parameter Value kann ein konstanter Wert addiert werden.
TAN	Tangensfunktion

12.2.4 RSI-Objekte für mathematische Vergleiche

Typ	Beschreibung
Equal	Vergleich auf Gleichheit Vergleicht Signaleingang 1 mit einem konstanten Wert oder vergleicht Signaleingang 1 und 2 miteinander.
Greater	Vergleich auf Größer Vergleicht Signaleingang 1 mit einem konstanten Wert oder vergleicht Signaleingang 1 und 2 miteinander.
Less	Vergleich auf Kleiner Vergleicht Signaleingang 1 mit einem konstanten Wert oder vergleicht Signaleingang 1 und 2 miteinander.

12.2.5 RSI-Objekte zur Koordinatentransformation

Typ	Beschreibung
Trafo_RobFrame	Transformiert einen Vektor aus den Eingängen 1 ... 3 von einem Roboter-Bezugskoordinatensystem in ein anderes.
Trafo_UserFrame	Transformiert einen Vektor aus den Eingängen 1 ... 3 in ein neues Bezugskoordinatensystem mit definierter Verschiebung und Verdrehung.

12.2.6 RSI-Objekte zur Bewegungskorrektur

Typ	Beschreibung
AxisCorr	Achswise Korrekturaufschaltung mit Begrenzung Roboterachsen A1 ... A6
AxisCorrExt	Achswise Korrekturaufschaltung mit Begrenzung Zusatzachsen E1 ... E6
Map2OV_PRO	Ändert den Programm-Override (\$OV_PRO).
PosCorr	Kartesische Korrekturaufschaltung mit Begrenzung
Stop	Anhalten einer Bewegung bei positiver Signalfanke Eine rein sensorgeführte Bewegung mit RSI_MOVECORR kann mit dem Modus ExitMoveCorr abgebrochen werden.

12.2.7 RSI-Objekte zur Korrekturüberwachung

Typ	Beschreibung
AxisCorrMon	Begrenzung für die achsspezifische Gesamtkorrektur Bei Überschreitung muss das Roboterprogramm zurückgesetzt werden. Die Ausgänge des Objekts liefern die aktuelle Gesamtkorrektur.
PosCorrMon	Begrenzung für die kartesische Gesamtkorrektur Bei Überschreitung muss das Roboterprogramm zurückgesetzt werden. Die Ausgänge des Objekts liefern die aktuelle Gesamtkorrektur.

12.2.8 RSI-Objekte zum Lesen von Roboterdaten

Typ	Beschreibung
AxisAct	Liefert die aktuellen Achswinkel der Roboterachsen A1... A6.
AxisActExt	Liefert die aktuellen Positionen der Zusatzachsen E1... E6.
GearTorque	Liefert die Getriebemomente der Roboterachsen A1... A6.
GearTorqueExt	Liefert die Getriebemomente der Zusatzachsen E1... E6.
MotorCurrent	Liefert die Motorströme der Roboterachsen A1... A6.
MotorCurrentExt	Liefert die Motorströme der Zusatzachsen E1... E6.
OV_PRO	Liefert den aktuellen Programm-Override \$OV_PRO.
PosAct	Liefert die aktuelle kartesische Roboterposition.
Status	Liefert Statusinformationen der Robotersteuerung, z. B. aktueller Status von Submit- oder Roboter-Interpreter, aktuelle Betriebsart etc.

12.2.9 RSI-Objekte zur Signalverarbeitung

Typ	Beschreibung
Constant	Konstante Setzt einen konstanten Wert
Cosine	Signalgenerator für eine Cosinusschwingung
D	Differenzierungsobjekt $y(k) = B0 * (x(k) - x(k-1))$ $B0 = KD / \langle \text{Sensortakt} \rangle$
Delay	Verzögert das Eingangssignal um eine bestimmte Zeit.
GenCtrl	Generisches Signalverarbeitungsobjekt bis zur 8. Ordnung $y(z) = B0*u(z) + B1*u(z-1) + B2*u(z-2) + \dots + B8*u(z-8) - A1*y(z-1) - A2*y(z-2) - \dots - A8*y(z-8)$
I	Integrationsobjekt (Trapez-Algorithmus) $y(k) = B0 * (x(k) + x(k-1)) + y(k-1)$ $B0 = \langle \text{Sensortakt} \rangle / (2 * TI)$
IIRFilter	IIR-FILTER
P	Verstärkung des Signals
PD	Proportional-Differential-Objekt $y(k) = B0 * x(k) + B1 * x(k-1)$ $B0 = KR * (1 + (TV / \langle \text{Sensortakt} \rangle))$ $B1 = -KR * (TV / \langle \text{Sensortakt} \rangle)$
PID	PID-Objekt $y(k) = y(k-1) + B0 * x(k) + B1 * x(k-1) + B2 * x(k-2)$ $B0 = KR * (1 + TV / \langle \text{Sensortakt} \rangle)$ $B1 = -KR * (1 - \langle \text{Sensortakt} \rangle / TN + 2 * TV / \langle \text{Sensortakt} \rangle)$ $B2 = KR * TV / \langle \text{Sensortakt} \rangle$
PT1	Verzögerungsobjekt 1. Ordnung $y(k) = -A0 * y(k-1) + B0 * x(k)$ $A0 = -\exp(-\langle \text{Sensortakt} \rangle / T1)$ $B0 = KR * (1 - \exp(-\langle \text{Sensortakt} \rangle / T1))$

Typ	Beschreibung
PT2	Verzögerungsobjekt 2. Ordnung $y(k) = -A0 * y(k-1) - A1 * y(k-2) + B0 * x(k) + B1 * x(k-1)$ Fall 1: $T1 \neq T2$ • $Z1 = \exp(-\langle \text{Sensortakt} \rangle / T1)$ • $Z2 = \exp(-\langle \text{Sensortakt} \rangle / T2)$ • $A0 = -Z1 - Z2$ $A1 = Z1 * Z2$ • $B0 = (KP / (T1 - T2)) / (T1 * (1 - Z1) - T2 * (1 - Z2))$ • $B1 = (KP / (T1 - T2)) / (T2 * Z1 * (1 - Z2) - T1 * Z2 * (1 - Z1))$ Fall 2: $T1 == T2$ • $Z0 = \exp(-\langle \text{Sensortakt} \rangle / T1)$ • $B0 = KP * (1 - Z0 * (\langle \text{Sensortakt} \rangle / T1) + 1)$ • $B1 = KP * Z0 * (Z0 + (\langle \text{Sensortakt} \rangle / T1) - 1)$
Rectangle	Signalgenerator für eine Rechteckschwingung
Sawtooth	Signalgenerator für eine Sägezahnschwingung
Sine	Signalgenerator für eine Sinusschwingung
Source	Signalgenerator Erzeugt einen definierten Signalverlauf, z. B. über ein konstantes Signal, ein sinus- oder cosinusförmiges Signal etc.
Timer	Nach Ablauf der eingestellten Zeit wird am Signalausgang Out1 eine positive Flanke ausgegeben.
Triangle	Signalgenerator für eine Dreiecksschwingung

13 KUKA Service

13.1 Support-Anfrage

Einleitung

Diese Dokumentation bietet Informationen zu Betrieb und Bedienung und unterstützt Sie bei der Behebung von Störungen. Für weitere Anfragen steht Ihnen die lokale Niederlassung zur Verfügung.

Informationen

Zur Abwicklung einer Anfrage werden folgende Informationen benötigt:

- Problembeschreibung inkl. Angaben zu Dauer und Häufigkeit der Störung
- Möglichst umfassende Informationen zu den Hardware- und Software-Komponenten des Gesamtsystems

Die folgende Liste gibt Anhaltspunkte, welche Informationen häufig relevant sind:

- Typ und Seriennummer der Kinematik, z. B. des Manipulators
- Typ und Seriennummer der Steuerung
- Typ und Seriennummer der Energiezuführung
- Bezeichnung und Version der Systemsoftware
- Bezeichnungen und Versionen weiterer/anderer Software-Komponenten oder Modifikationen
- Diagnosepaket der Systemsoftware

Für KUKA Sunrise zusätzlich: Vorhandene Projekte inklusive Applikationen

Für Versionen der KUKA System Software älter als V8: Archiv der Software (Diagnosepaket steht hier noch nicht zur Verfügung.)

- Vorhandene Applikation
- Vorhandene Zusatzachsen

13.2 KUKA Customer Support

Die Kontaktdaten der lokalen Niederlassungen finden Sie unter:
www.kuka.com/customer-service-contacts

Index

#IPO, Sensormodus.....	9, 18, 25, 51, 52
#IPO_FAST, Sensormodus.....	9, 18, 52

A

ABS (RSI-Objekt).....	132
Absolutkorrektur.....	22
Achsweise Korrektur.....	20
ACOS (RSI-Objekt).....	132
AND (RSI-Objekt).....	132
Anhang.....	131
AnIn (RSI-Objekt).....	131
AnOut (RSI-Objekt).....	131
Arbeitsraumbegrenzung, Sicherheit.....	31
ASIN (RSI-Objekt).....	132
ATAN (RSI-Objekt).....	133
ATAN2 (RSI-Objekt).....	133
AxisAct (RSI-Objekt).....	134
AxisActExt (RSI-Objekt).....	134
AxisCorr (RSI-Objekt).....	134
AxisCorrExt (RSI-Objekt).....	134
AxisCorrMon (RSI-Objekt).....	134

B

BAND (RSI-Objekt).....	132
BCOMPL (RSI-Objekt).....	132
Bedienung.....	39
Begriffe, verwendet.....	8
Beispieldateien, einbinden.....	93
Beispielkonfigurationen.....	93
Beispielprogramme.....	93
Benutzerrechte, anzeigen.....	39
Bestimmungsgemäße Verwendung.....	29
BOR (RSI-Objekt).....	132

C

CCS.....	8
CEIL (RSI-Objekt).....	133
Constant (RSI-Objekt).....	135
COS (RSI-Objekt).....	133
Cosine (RSI-Objekt).....	135

D

D (RSI-Objekt).....	135
Deinstallieren	
über smartHMI.....	37
über WorkVisual.....	35
Delay (RSI-Objekt).....	135
Diagnose.....	111
Diagnosedaten, anzeigen.....	114
Diagnosemonitor (Menüpunkt).....	114
Diagnosepaket.....	137
DigIn (RSI-Objekt).....	131
DigOut (RSI-Objekt).....	131
Dokumentation, Industrieroboter.....	7

E

E/A-Verschaltung, RSI-E/As.....	41
Einleitung.....	7
Equal (RSI-Objekt).....	133
Ethernet.....	8
Ethernet-Sensor-Netzwerk.....	43
Ethernet-Sensor-Netzwerk.....	44
Ethernet-Verbindung, in WorkVisual konfigurieren.....	69
Ethernet-Verbindung, XML-Datei.....	61
Ethernet (RSI-Objekt).....	131
EXP (RSI-Objekt).....	133
Expertenprogrammierung, RSIVisual.....	71
Exportieren, Werkzeugkasten.....	83

F

Fehlanwendung.....	29
Fehlerbehandlung.....	45
Fehlerprotokoll.....	115
FLOOR (RSI-Objekt).....	133
Funktionen.....	13
Funktionsblock, Ein- und Ausgänge definieren.....	88
Funktionsblock, erstellen.....	82
Funktionsblock, im Editor öffnen.....	85
Funktionsblock, in RSI-Kontext aktualisieren.....	83
Funktionsblock, programmieren.....	87
Funktionsblockeigenschaften, definieren.....	89
Funktionsblockparameter, Zugriff über KRL.....	91
Funktionsgenerator.....	45
Funktionsgenerator, verwendet.....	33
Funktionsgruppen.....	39

G

GearTorque (RSI-Objekt).....	134
GearTorqueExt (RSI-Objekt).....	134
GenCtrl (RSI-Objekt).....	135
Greater (RSI-Objekt).....	133
Gruppe, im Werkzeugkasten hinzufügen.....	82

H

Hilfdatei.....	92
Hinweise.....	7

I

I (RSI-Objekt).....	135
IIRFilter (RSI-Objekt).....	135
Importieren, Werkzeugkasten.....	84
Input (RSI-Objekt).....	131
Installation.....	33
über smartHMI.....	35
über WorkVisual.....	34
IP.....	8
IPO, Sensormodus.....	33, 45

K

Kartesische Korrektur.....	20
Kategorie, für Werkzeugkasten erstellen.....	82
Kenntnisse, benötigt.....	7
KLI.....	8
KLI, Netzwerkkonfiguration.....	43
Konfiguration.....	41
KONI.....	8
KONI, Netzwerkkonfiguration.....	44
Kontextverwaltung.....	45
Korrekturmethode.....	24
Korrekturmodus.....	22
Korrekturtyp.....	19
KR C.....	8
KRL-Ressourcen.....	33
KUKA Customer Support.....	137
KUKA Service.....	137

L

Less (RSI-Objekt).....	133
Limit (RSI-Objekt).....	133
Lizenzbedingungen.....	10
LOG-Datei.....	115
LOG-Level, konfigurieren.....	115
LOG (RSI-Objekt).....	133
Logging_RSI.xml.....	116

M

Map2AnOut (RSI-Objekt).....	131
Map2DigOut (RSI-Objekt).....	131
Map2OV_PRO (RSI-Objekt).....	134
Map2Sen_PInt (RSI-Objekt).....	131
Map2Sen_PRea (RSI-Objekt).....	131
Marken.....	10
Meldungen.....	117
MinMax (RSI-Objekt).....	133
Monitor (RSI-Objekt).....	132
MotorCurrent (RSI-Objekt).....	134
MotorCurrentExt (RSI-Objekt).....	134
MULTI (RSI-Objekt).....	133

N

NORM (RSI-Objekt).....	133
NOT (RSI-Objekt).....	132

O

Objektparameter, Zugriff über KRL.....	60
Open-Source.....	10
Optionspaketeditor.....	84
OR (RSI-Objekt).....	132
Output (RSI-Objekt).....	132
OV_PRO (RSI-Objekt).....	134

P

P (RSI-Objekt).....	135
PD (RSI-Objekt).....	135
PID (RSI-Objekt).....	135
PosAct (RSI-Objekt).....	134
PosCorr (RSI-Objekt).....	134
PosCorrMon (RSI-Objekt).....	134
POW (RSI-Objekt).....	133
Produktbeschreibung.....	13
Programmierung.....	49
PT1 (RSI-Objekt).....	135
PT2 (RSI-Objekt).....	136

R

Rectangle (RSI-Objekt).....	136
Relativkorrektur.....	22
ResetDigOut (RSI-Objekt).....	132
ROUND (RSI-Objekt).....	133
RS FlipFlop (RSI-Objekt).....	132
RSI.....	10
RSI-Befehle, Übersicht.....	49
RSI-Container.....	10
RSI-Container-ID.....	10
RSI-Container-ID, prüfen.....	56
RSI-Container, aktivieren.....	57
RSI-Container, deaktivieren.....	57
RSI-Container, erstellen.....	50
RSI-Container, löschen.....	51
RSI-Element, in Editor einfügen.....	74
RSI-Elemente, automatisch anordnen.....	77
RSI-Elemente, Ein-/Ausgänge verbinden.....	76
RSI-Elemente, gruppieren.....	77
RSI-Kontext.....	10
RSI-Kontext, anlegen.....	71
RSI-Kontext, im Kontext-Editor öffnen.....	73
RSI-Kontext, in Programm einbinden.....	60
RSI-Kontextdatei, importieren und konvertieren.....	72
RSI-Monitor.....	10
RSI-Monitor, anzeigen.....	111
RSI-Objekt.....	10
RSI-Objekt, Parameter bearbeiten.....	75
RSI-Objekt, Parameter freischalten.....	75
RSI-Objektbibliothek.....	10, 131
RSI-Objektparameter.....	10
RSI-Objektparameter, neuen Wert zuweisen.....	55
RSI-Objektparameterwert, lesen.....	54
RSI, Ein-/Ausgänge verschalten.....	41
RSI_CHECKID.....	56
RSI_CREATE.....	50
RSI_DELETE.....	51
RSI_DISABLE.....	57
RSI_ENABLE.....	57
RSI_GETPUBLICPAR.....	54
RSI_MOVECORR.....	53
RSI_OFF.....	53
RSI_ON.....	51
RSI_RESET.....	56
RSI_SETPUBLICPAR.....	55
RSI_USER.DAT.....	45

RSIERRMSG.....	45
RSITECHIDX.....	45
RSIVisual.....	10

S

Sawtooth (RSI-Objekt).....	136
Schlüsselwörter, Daten lesen.....	67
Schlüsselwörter, Daten schreiben.....	68
Schriftarten.....	49
Schulungen.....	7
Sen_PInt (RSI-Objekt).....	132
Sen_PRea (RSI-Objekt).....	132
Sensorbetrieb, Sicherheit.....	31
Sensorgeführte Bewegung, einschalten.....	53
Sensorkorrektur, RoboTeam-Betrieb.....	26
Sensorkorrektur, Sicherheit.....	31
Sensorkorrektur, Übersicht.....	18
Sensormodus.....	9, 18
Server-Programm.....	93
Server-Programm, Bedienoberfläche.....	95
Server-Programm, einbinden.....	93
Server-Programm, Kommunikationsparameter einstellen.....	96
SetDigOut (RSI-Objekt).....	132
Sicherheit.....	31
Sicherheitshinweise.....	7
Signalaufzeichnung, in RSI-Monitor laden...	114
Signalaufzeichnung, speichern.....	114
Signaleigenschaften, definieren.....	112
SignalSwitch (RSI-Objekt).....	132
Signalverarbeitung.....	17
Signalverarbeitung, ausschalten.....	53
Signalverarbeitung, einschalten.....	51
Signalverlauf, anzeigen und aufzeichnen...	113
SIN (RSI-Objekt).....	133
Sine (RSI-Objekt).....	136
smarHMI.....	9
smartPAD.....	9
Software-Endschalter.....	31
Source (RSI-Objekt).....	136
Speicher, erhöhen.....	131
SR FlipFlop (RSI-Objekt).....	132
Status (RSI-Objekt).....	134
Stop (RSI-Objekt).....	134
SUB (RSI-Objekt).....	133
SUM (RSI-Objekt).....	133
Support-Anfrage.....	137
Systemvoraussetzungen	
Hardware.....	33
Kompatibilität.....	33
Software.....	33

T

TAN (RSI-Objekt).....	133
TCP.....	9
TestServer.exe.....	93
Timer (RSI-Objekt).....	136
Trafo_RobFrame (RSI-Objekt).....	134

Trafo_UserFrame (RSI-Objekt).....	134
Triangle (RSI-Objekt).....	136
TTS.....	9

U

Übersicht, RSI-Befehle.....	49
Übersicht, Sensorkorrektur.....	18
UDP.....	9
Update, über smarHMI.....	35
Update, über WorkVisual.....	34

V

Verwendete Begriffe.....	8
--------------------------	---

W

Werkzeugkasten.....	79
Werkzeugkasten, benutzerdefiniert.....	81

X

XML.....	9
XML-Datei, Ethernet-Verbindung.....	61
XML-Schema.....	66
XOR (RSI-Objekt).....	132

Z

Zeichen.....	49
Zielgruppe.....	7